SUP'COM

المدرسة العليا للمواصلات بتونس
ÉCOLE SUPÉRIEURE DES COMMUNICATIONS DE TUNIS

Telecommunications Engineering Degree

Major:

Applications of Multimodal Information (AIM)

End of study internship report

Theme:

Data-driven Approaches to Forecasting the Evolution of Pandemics and Mitigating their Spread

Realized by:

Hamdi Friji

Supervisors:

Dr. Hichem Besbes, Sup'Com, Tunisia
Dr. Hakim Ghazzai, Stevens Institute of Technology, USA
Dr. Yehia Massoud, Stevens Institute of Technology, USA

Work proposed and realized within:

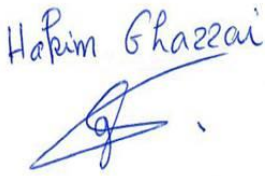
STEVENS INSTITUTE of TECHNOLOGY
THE INNOVATION UNIVERSITY®

Academic year: 2019-2020

Research Supervisor

Dr. Hakim GHAZZAI
Research Scientist at the School of
Systems and Enterprises,
Stevens Institute of Technology, USA

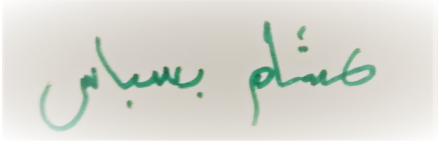
Signature:



Academic Supervisor

Hichem BESBES
Professor at the Higher School of
Communication of Tunis, Supcom,
University of Carthage, TUNISIA

Signature:



Abstract

Since early 2020, the world has been afflicted with an unprecedented global pandemic. The SARS-CoV-19 (COVID-19) has levied massive economic and public health costs across many countries. Due to its virulence, the pathogen is rapidly propagating throughout the world in such a way that makes it incredibly challenging for officials to contain its spread. Therefore, there is a pressing need for national and local authorities to have tools that aid in their ability to assess and extrapolate the future trends of the spread of COVID-19, so they may make rational and informed decisions that minimize public harm. Additionally, the unprecedented worldwide spread of corona virus disease has significantly sped up the development of technology-based solutions to prevent, combat, monitor, or predict pandemics and/or its evolution. The omnipresence of smart Internet-of-things (IoT) devices can play a predominant role in designing advanced techniques helping in minimizing the risk of contamination. In this study, we propose solutions to analyze, forecast, and prevent the spread of pandemics, taking as target the COVID-19.

Firstly, we tackle the problem of assessing and forecasting the spread of the virus to assist the governments in the decision making during pandemics. Indeed, in this part, we present a generalized mechanistic model characterizing the COVID-19 pandemic evolution from a susceptible state to a discharged state, i.e., deceased or recovered states while passing by quarantined and hospitalized states. The model parameters are determined by solving a fitting optimization problem with three observed inputs: the number of infected, deceased, and reported cases. The model's objective function is weighted over the training days so as to guide the fitting algorithm towards the latest pandemic period and lead to more accurate trend predictions for a stronger forecast. We solve the fitting problem with the Levenberg-Marquardt

algorithm, we compare the performance of the model generated from this algorithm to one of another state of the art fitting algorithm. We test the model on the COVID-19 data from four highly afflicted countries. The fitting algorithm has been validated graphically and numerically using, respectively, forecast figures and validation metrics. Indeed, the presented results shows accurate results for most of the countries. Once the model parameters are estimated, forecasting results are derived and uncertainty regions of the expected scenarios are provided.

Secondly, a practical framework that employs the Social IoT (SIoT) concept, computer vision approach, and graph theory is proposed to ensure a safe navigation of pedestrians' through a real-world map of a smart city. The objective is to mitigate the risks of exposure to the virus in high-dense areas where social distancing might not be well-practiced. The proposed routing approach recommends pedestrians' routes in a real-time manner while considering other devices' mobility, other framework users, and the offline pedestrians. First, the IoT devices are clustered into communities according to two SIoT relations that consider the devices' locations and the friendship levels among their owners. Accordingly, the city map roads are assigned weights representing their risky levels, which are based on the SIoT communities and additional information extracted from Closed-Circuit Tele-Vision (CCTV) data input using computer vision models. Afterward, a Dijkstra-based multi-user navigation algorithms, SFOA and OUFIA, are applied to recommend the safest route to follow. Simulation results applied on a real-world IoT data-set have shown the ability of the proposed approach in achieving trade-offs between both safest and shortest paths according to the pedestrian preference.

Keywords: COVID-19, Curve-Fitting, Mechanistic Modeling, Forecasting, Social IoT, Navigation Algorithm, Community Detection, Graph Theory.

Acknowledgments

At the fulfillment of this internship, I would like to thank and express my gratitude to my supervisor Dr. Hakim Ghazzai for his warm welcome upon my arrival, for his kindness and availability. He encouraged and guided me through this work and never failed to provide anything I needed.

I would like to thank my supervisor at SUP'COM Dr. Hichem Besbes for his precious time, support and dedication. He steered me in the right directions whenever I faced problems and helped me successfully conduct my work with constructive advice.

Special thanks and appreciation to Mr. Yehia Massoud, Dean of the School of Systems and Enterprises, who provided constant support to the team.

I thank my fellow lab-mates for the stimulating discussions, for the sleepless nights we were working together, for their support during the pandemic and for all the fun we have had during this internship period.

Last but not the least, my warm thanks to the member of the jury for accepting to evaluate this work.

Dedication

I dedicate my work to my family and my friends.

A special feeling of gratitude to my loving parents, Hamid and Latifa whose words of encouragement and push for tenacity ring in my ears. My sisters Soumayya and Asma have never left my side and are very special.

I also dedicate this work to my many friends and family members who have supported me throughout the process. I dedicate this work and give special thanks to all my friends for being there for me throughout the entire internship program and especially during this pandemic.

Preface

This internship is a part of the mandatory graduation fellowship required for engineering students of the Higher School of Communication of Tunis (SUP'COM), as the final project of a 3-year engineering cycle. The students are required to take theoretical and academic learning into practice to sharpen their skills in the area of expertise. This report is submitted to obtain the National Diploma in Telecommunication Engineering. This project is a collaboration between:

- SUP'COM which is the main school of educating engineers in Telecommunications in Tunisia. It is affiliated to the University of Carthage and occupies the advanced ranks in the competitive examination for the access to the engineering studies.

- Stevens Institute of Technology which is a private research university in Hoboken, New Jersey. Incorporated in 1870, and it is one of the oldest technological universities in the United States.

Precisely, this work was conducted at the School of Systems and Enterprises (SSE) which is the home of the Systems Engineering Research Center (SERC), a University-Affiliated Research Center (UARC) of the U.S. Department of Defense. The SERC is a national resource providing a critical mass of systems engineering researchers. The center leverages the research and expertise of senior lead researchers from 22 collaborator universities throughout the United States. The SERC is unprecedented in the depth and breadth of its reach, leadership and citizenship in systems engineering through its conduct of vitally important research and the education of future systems engineering leaders.

Initially, this internship was supposed to investigate the car-following approach for autonomous driving. To this end, a car-following framework was proposed in a conference paper, attached in Appendix section, which was accepted in the IEEE Global conference on the Artificial Intelligence and IoT (GCAIOT'20). However, due to some problems of access to the university during the pandemic and the need of solutions for mitigating the spread of the virus, we decide to change the topic and work on this emerging issue and hence, in the current study, we propose to use technologies to help governments in assessing, forecasting, and preventing the spread of the COVID-19 pandemic.

Table of contents

Abstract iii

Acknowledgments v

Dedication vi

Preface vii

List of Figures xii

List of Tables xiv

List of Acronyms xv

General Introduction 1

0.1 Background & motivation 1

0.2 Contributions 3

0.3 Report outline 5

I Assessing and Forecasting the Evolution of the COVID-19 Pandemic 7

1 Introduction & Preliminaries 8

1.1 Introduction 8

1.2 Preliminaries 9

1.2.1 Mechanistic models 9

1.2.2 SEIR model 9

1.2.3 Non Linear Least Square (NLLS) Problem 10

1.2.4 NLLS Optimization Algorithm: Levenberg-Marquardt (LM) 10

2 Related Work 13

3 A Generalized Mechanistic Model for Assessing and Forecasting the Spread of COVID-19 Pandemic 16

3.1 Introduction 16

3.2 Proposed Mechanistic Model 16

3.3 Curve Fitting Model . . . 21
3.3.1 Model Weights . . . 23
3.3.2 Optimization Problem . . . 24
3.3.3 Fitting Algorithm . . . 25
3.3.4 Fitting Algorithm Validation . . . 26
3.4 Conclusion . . . 28

4 Results & Discussion **29**
4.1 Introduction . . . 29
4.2 Simulation Parameters . . . 29
4.3 Training Phase and Weights Impact: The Case of Russia . . . 31
4.4 Fitting Performance and Validation . . . 35
4.5 Expected Spread and Forecasting Validation . . . 39
4.6 Limitation of the Study . . . 46
4.7 Conclusion . . . 48

Summary **49**

II Mitigating the Spread of the COVID-19 Pandemic **50**

1 Introduction & Preliminaries **51**
1.1 Introduction . . . 51
1.2 Preliminaries . . . 52
1.2.1 Internet of Things (IoT) . . . 52
1.2.2 Social IoT (SIoT) . . . 52
1.2.3 Object Detection algorithm: YOLOv3 . . . 52

2 Related work **53**

3 A Social IoT-driven Pedestrian Routing Framework to Mitigate the Spread of the COVID-19 **56**
3.1 Introduction . . . 56
3.2 PROPOSED FRAMEWORK . . . 56
3.2.1 Data-set . . . 57
3.3 Framework Architecture . . . 57
3.3.1 Street Network Graph Structure . . . 61
3.3.2 SIoT Relation and Community Detection . . . 61
3.3.3 Computer Vision Phase . . . 64
3.3.4 Weight Link Computation . . . 66
3.3.5 Real-Time Navigation . . . 69

3.3.6 Multi-User Navigation 69
3.4 Conclusion 73

4 Results & Discussion **74**
4.1 Introduction 74
4.1.1 Data-set and Map Description 74
4.1.2 Clustering and Community Detection 75
4.1.3 Computer Vision & Weight Link Prediction 76
4.1.4 Navigation Results 77
4.1.5 Monte Carlo Simulation 79
4.1.6 Multi-User Navigation Results 81
4.2 Conclusion 83

Summary **84**

General Conclusion **85**

Accepted/Submitted publications **88**

References **89**

List of figures

3.1 The proposed mechanistic model architecture to characterize the COVID-19 pandemic spread, where the orange boxes are the final states and other boxes are intermediate states. 17

4.1 Plots of the investigated normalized weight functions. 31
4.2 Weighted MSE evolution during the training phase for each weight function. . . 31
4.3 Russia: Graphical validation of the LM algorithm for different weight functions. 32
4.4 Russia: Illustration of the values of the APE, RAE, and AE metrics versus the training days using the exponential weight function. 34
4.5 Italy: Graphical validation of the LM algorithm using the exponential weight function. 34
4.6 Brazil: Graphical validation of the LM algorithm using the exponential weight function. 34
4.7 Russia: Fitting results of the BFGS algorithm using the exponential weight function. 35
4.8 USA (Phase I): Graphical validation of the LM algorithm using the exponential weight function for the first period of the pandemic (January $22^{\text{nd}}, 2020$ till June $13^{\text{th}}, 2020$). 37
4.9 USA (Phase II): Graphical validation of the LM algorithm using the exponential weight function for the first period of the pandemic (June $13^{\text{th}}, 2020$ till July $7^{\text{th}}, 2020$). 37
4.10 Russia: COVID-19 spread forecasting results and expected evolution of the different states based on the estimated parameters. 39
4.11 Italy: COVID-19 spread forecasting results and expected evolution of the different states based on the estimated parameters. 40
4.12 Russia: Best and worst possible spread scenarios by varying the infection rate β. 41
4.13 Italy: Best and worst possible spread scenarios by varying the infection rate β. . 41
4.14 Russia: Forecast of the deceased cases. Comparisons between the proposed and the RW models. 42
4.15 Russia: Forecast of the infected cases. Comparisons between the proposed and the RW models. 42

4.16 Italy: Forecast of the deceased cases. Comparisons between the proposed and the RW models. 44
4.17 Italy: Forecast of the infected cases. Comparisons between the proposed and the RW models. 44

3.1 An overview of the proposed framework that recommend a safe and fast route to the users during a pandemic using SIoT concept. 57
3.2 Overview of the preprocessing phase 58
3.3 Overview of the community detection and weight computation 58
3.4 A selected area of Santander, Spain. 62
3.5 CLOR communities (The colors of each CLOR community indicates the density level of the community). 62
3.6 SFOR communities (IoT devices belonging to the same community are labeled by the same marker). 62
3.7 Graph network of roads equipped with CCTV cameras. 62
3.8 Communities were detected based on CLOR and SFOR relations using the Louvain Method. 62
3.9 Overview of the computer vision tracking system 68
3.10 PINA 72
3.11 SINA 72
3.12 Comparison between the two proposed heuristic approaches a) PINA and b) SINA. In PINA, user 2 needs to wait before entering k1 or go through k3 instead since priority is given to user 1. In b), there is no priority among the users. User 1 is arriving later, hence he needs to yield to user 2 to enter k1 safely. 72

4.1 Street network after assigning to each road a computer vision weight 76
4.2 Two examples showing the different paths recommended to the user for different values of α. 77
4.3 Example of the Pedestrians tracking in a 3 roads intersection 78
4.4 The routing updates on three time phases are based on the changes in device locations and communities in the graph. 78
4.5 A trade-off between the safety factor and destination amount for the proposed framework. 79
4.6 Special case: Time schedule of 6 users. The length of each segment (linking two diamond) indicates the time needed to navigate. The labels on each line (e.g., E1) indicate the path segments crossed by a user. W-i indicates the waiting action to yield to the user i. 80
4.7 Navigation path of 6 users using the PINA multi-user navigation algorithm . . . 81
4.8 Navigation path of 6 users using the SINA multi-user navigation algorithm . . . 82

List of Tables

1 COVID-19 statistics for some of the highly impacted countries by June 15th, 2020. 2

3.1 High-level comparison of the proposed model and previous studies on COVID-19 spread. 21

4.1 Initialization of the ODE model states in millions for each investigated country 30
4.2 Russia: Numerical validation of the LM fitting algorithm using the different weight functions 32
4.3 Numerical validation of the LM fitting algorithm using the exponential weight function for Russia, Italy, and Brazil 36
4.4 Comparison between the LM and BFGS fitting algorithms using the exponential weight function for Russia, Italy, and Brazil 37
4.5 USA: Numerical validation of the LM fitting algorithm using the exponential weight function 38
4.6 Estimated model parameters for each investigated country 39
4.7 Russia: Numerical validation of the forecast results 43
4.8 Italy: Numerical validation of the forecast results 45

4.1 Validation results of the CLOR communities 75
4.2 Validation results of the SFOR communities 75
4.3 Numerical comparison of the Navigation Time (NT) and the Cumulative Risk Score (CRS) the two multi-user algorithm 82

LIST OF ACRONYMS

Non Linear Least Square (NLLS)

Levenberg-Marquardt (LM)

Internet Of Things (IoT)

Ordinary Differential Equation (ODE)

Broyden–Fletcher–Goldfarb–Shanno algorithm (BFGS)

Computer Vision (CV)

Social IoT (SIoT)

Artificial Intelligence (AI)

Computer Vision (CV)

Graph Theory (GT)

General Introduction

0.1 Background & motivation

Since early 2020, the world has been afflicted with an unprecedented global pandemic, the SARS-CoV-19 (COVID-19). In fact, Corona viruses are a large family of viruses that trigger an infection in the contaminated's nose, sinuses, or upper throat, and even fatal acute respiratory distress syndrome (ARDS) [1,2]. In addition to humans, the coronaviruses may also infect animals including camels, cattle, cats, and bats. Even though coronaviruses originated in animals can rarely infect humans, the Middle East Respiratory Syndrome CoronaVirus (MERS-CoV) and the Severe Acute Respiratory Syndrome CoronaVirus (SARS-CoV) have broken the rule and shifted to humans. The same happened with the new coming virus, named SARS-CoV-2 by International Committee on Taxonomy of Viruses that causes a disease named Coronavirus Disease 2019 (COVID-19) by the World Health Organization (WHO). Generally, the human-to-human transmission occurs when a contaminated person's coughs, sneezes, or even talks. Indeed, released droplets may stay in the air and can infect people within a 6 feet distance [3]. The virus can also be transmitted through aerosol or surface transmission as researchers have shown that the virus may live in the air for up to three hours and on steel for two or three days [4].

Originally starting from Wuhan, Hubei a province of China, the first cases of COVID-19 were announced in the late of 2019. The outbreak quickly spread within China due to the massive transportation and large population mobility before the Chinese Spring Festival, and subsequently, it spread all over the world. The WHO declared the outbreak to be a Public Health Emergency of International Concern on January 30th, 2020 and recognized it as a pan-

Table 1: COVID-19 statistics for some of the highly impacted countries by June 15th, 2020.

Country	**Infected Cases**	**Death cases**	**Recovered cases**
USA	2,137,731	117,612	583,503
Italy	237,500	34,405	178,526
Russia	544,725	7,274	293,780
Brazil	923,189	45,241	490,005

demic on March 11th, 2020. Generally, coronaviruses are not dangerous, however, the new COVID-19 comes with a considerable mortality rate. At the time of writing the paper associated to this project, more than 12 millions cases of COVID-19 are confirmed resulting on more than 550 thousand deaths in 216 different countries, areas, and territories as indicated by WHO through the COVID-19 situation dashboard website. These numbers are expected to dramatically increase. Most of these cases are located in the United States where more than 3 million confirmed cases and more than 110 thousand confirmed deaths have been reported representing around 25% and 24% of the global results, respectively [5].

The statistics of COVID-19 spread in some of the highly impacted countries are given in Table 1.

Moreover, Italy has now confirmed more than two hundred thousand COVID-19 cases and about 33,475 related deaths, according to Worldometer, which is tracking COID-19 statistics. Spain became the fourth most coronavirus-infected nation in the world, with 286,509 cases, 27,127 deaths, and 196,958 recovered. India is suffering too due to the high population, in fact, India has 197,808 cases, 5,603 deaths, and 95,415 recovered cases.

In addition to the high fatality rates, the rapid spread of COVID-19 has engendered tremendous social and economic implications all over the world [6–10]. For example, R. Y. Kim identified the pandemic as an accelerator of the structural change in consumption and the digital transformation in the marketplace. He also explored how the pandemic accelerated the growth of e-commerce [11]. The global economy has also been impacted since the usage of human distancing, as an easiest solution to limit the spread of the disease. Thus, non-essential services

have been shut-down, and hence, hundreds and even thousands of people lost their jobs. Furthermore, since most of the world nations have closed their borders, the international trade has collapsed. Additionally, the impact of the COVID-19 pandemic has reached the telecommunication industry. Accordingly, Internet Providers (IPs) have recently reported a massive and huge traffic due to the lock-down and stay-home orders applied by several states in the USA and other regions and countries [12]. Over just few months, COVID-19 has changed the world and specifically our habits and daily lives and the situation does not seem to return to its normal soon in many countries.

Therefore, the prevention and control of the COVID-19 epidemic is a major challenge for all countries. Actually, the decision-making process during pandemics should cover three aspects: (1) to predict the possibility of the epidemic through the risk analysis of the epidemic; (2) put forward prevention and control strategies for epidemic risk; (3) conduct comprehensive assessment of the harm caused by the epidemic. Accordingly, to create an effective COVID-19 control and prevention mechanism for the current situation of the pandemic, governments should pay attention, in the first place, to solutions that assess and forecasts the spread of the virus to help the control of the pandemic, and in the second place, the solutions focusing on the prevent the spread of the virus among susceptible population.

0.2 Contributions

This project aims to propose solutions for the governments to assess, forecast, and prevent the spread of the COVID-19 pandemic. Indeed, the proposed solutions tackle two different directions as follows:

- Spread Assessing and forecasting solution.
- Mitigating the spread of the Virus.

The former solution investigate the evolution of COVID-19 in different countries of the world using an extended mechanistic model. The developed model is composed of eight states:

protected, susceptible, exposed, infectious, recovered, hospitalized, quarantined, and deaths. These states aim to characterize the possible behaviour of the crowd and their effects on the pandemic spread. The evolution of the pandemic is then modeled as an Ordinary Differential Equation (ODE) system interconnecting the different states. In order to forecast the evolution of COVID-19, we proceed with an identification process that aims to estimate the values of the coefficients of the model. To this end, a fitting optimization problem is developed having as inputs three target data-sets, i.e., observations: confirmed infected cases, death cases, and recovered cases. The objective function to be optimized is modeled as a weighted non-linear least squares function and is solved by the Levenberg-Marquardt (LM) algorithm. The weights are added to guide the focus of the fitter towards a particular period of the trend. The model is fitted and tested on multiple countries and shows accurate performances. The impact of the weights on the fitting performance has been also evaluated. The performances of the fitting algorithms are also compared to the ones of the Broyden–Fletcher–Goldfarb–Shanno (BFGS) [13] algorithm for various metrics. The estimated parameters are then used to forecast the future evolution of the pandemic given the current observation. Then, uncertainty regions are provided to visualize the expected upper and lower limits of the future evolution of the number of cases and deaths.

To the best of our knowledge, we are the the first to characterize a pandemic with a developed model presenting a loop between the states while taking into consideration all possible movements between them. Moreover, we are the first to employ a weighted fitting process that endorses the forecasting process by focusing on the last trends of the spread. We also provide a comparison of the estimated, best, and worse cases with regards to the infection rate of the disease to estimate how well the countries operated against the pandemic and finally, to validate our forecast results, we compare the performance of our proposed model to those of a Random Walk (RW) approach with drift [14] and show that our proposed model outperforms the RW in most of the investigated scenarios.

The latter solution investigate a smart navigation framework intending to give pedestrians safe routing to bypass areas where the risk of COVID-19 transmission is high, and hence pre-

vent the spread of the virus. In other words, the framework recommends a pedestrian walking route in which guarantees a social distancing and avoiding close contact. The proposed approach includes four steps: First, the framework identifies the IoT devices located in the area of interest and then establishes social graphs interconnecting these devices using different social IoT relations. Then, the Louvain community detection algorithm is applied to the SIoT graphs to determine different communities of IoT devices. In our approach, we focus primarily on two social relations: a distance-based relation that identifies crowded/high-density areas of IoT devices and a device friendship relation that allows labeling streets where the user may possess a high chance of meeting a close friend. The third step is to compute different scores representing each street's safety level or segment of a street in the area of interest according to the nearby detected social communities. Finally, in the last step, the city map is transformed into a weighted undirected graph to which we apply the Dijkstra algorithm [15] in order to determine a route characterized by a certain level of safety. A weighted bi-objective function balancing between the shortest and safest routes is developed and implemented. The framework will then deliver the trajectories to the user, e.g., via a mobile application for the best route to follow to reach a destination. The proposed routing approach takes into account the mobility of IoT devices and may update the recommended route regularly by repeating the process, as mentioned earlier.

0.3 Report outline

The remainder of this report is structured as follows:

- Part 1: This part tackles the assessment and the forecast of the COVID-19 pandemic and it contains a first Chapter that introduces the proposed solution and presents all the needed preliminaries to understand it. The second Chapter discusses the related work of all the employed approaches and techniques. Afterward, in Chapter 3, we define and describe the proposed solution, while in the last Chapter, we present and discuss the obtained results.

- Part 2: This part investigates a solution to prevent the spread among the navigating pedestrians. The first Chapter contains an introduction and presents the preliminaries needed to understand the proposed work. The second Chapter discusses the related work of all the employed approaches and techniques. Additionally, in the third Chapter, the proposed solution is investigated in a details, and in the last Chapter, we present and discuss the obtained results.

Part I

Assessing and Forecasting the Evolution of the COVID-19 Pandemic

Chapter 1

Introduction & Preliminaries

1.1 Introduction

In this part, we investigate a proposed solution that intent to assist governments to well understand the spread of the virus in the goal of endorsing the decision-making process. As a matter of fact, one of the most important steps during the control and the prevention of pandemics is to forecast the number of infected cases and deaths, and hence, optimize the decision-making process that will help to reduce the risk of contamination among the susceptible population. However, our model exploits the mechanistic modeling concept to model the spread of the virus and uses curve-fitting techniques to estimate the parameters of the model. This part is organized as follows, This chapter introduces the investigated solution in this part and explain the required preliminaries to understand the related work, cited and discussed in Chapter 2, and the proposed model, which is presented in Chapter 3. Finally, the results are discussed and presented in Chapter 4.

To fully understand this part, some basics and preliminaries should be already known. Therefore the next section will introduce and explain any concept required to follow the evolution of this part.

1.2 Preliminaries

In this section, we introduce and clarify all the exploited concepts during this first part. Accordingly, this section contains the needed knowledge to easily understand smoothly follow the proposed technical techniques. Indeed, the investigated approaches in this section belong to the mathematical pandemics modeling and Non-Linear Least Square problem solving realms.

1.2.1 Mechanistic models

Mechanistic models are a category of mathematical models that assumes in its definition that a complex system can be understood by examining the workings of its individual parts and the manner in which they are coupled. Accordingly, Mechanistic models typically have a tangible, physical aspect, in that system components are real, solid, and visible. Actually, the first mechanistic model of infectious disease transmission used in assessing intervention strategies was a mathematical model of malaria transmission developed and refined by Ronald Ross who earned a Nobel prize after this study. Mechanistic models outperform the traditional models such as regression models due to their structure that makes explicit hypotheses about the biological mechanisms of infection dynamics.

1.2.2 SEIR model

The classical SEIR model has four elements which are S (susceptible), E (exposed), I (infectious) and R (recovered). Thus, $N = S + I + E + R$ means the total number of people. The basic hypothesis of the SEIR model is that all the individuals in the model will have the four roles as time goes on. The SEIR model has some limitations for the real situations, but it provides a basic model for the research of different kinds of epidemic.

1.2.3 Non Linear Least Square (NLLS) Problem

Nonlinear Least Squares Model, As the name suggests, is any model of the basic form,

$$y = f(\vec{\boldsymbol{x}}, \vec{\boldsymbol{\beta}}) + \epsilon \tag{1.1}$$

in which

- The functional part of the model is not linear with respect to the unknown parameters, $\beta_0, \beta_1, \ldots$
- The method of least squares is used to estimate the values of the unknown parameters.

The NLLS estimator, that is usually used to solve the NLLS problem is defined as:

$$\widehat{\theta} = \underset{\theta}{\operatorname{argmin}} \sum_{n=1}^{N} \big(y_n - \widehat{y}_n(\theta)\big)^2. \tag{2}$$

1.2.4 NLLS Optimization Algorithm: Levenberg-Marquardt (LM)

In order to explain the Levenberg-Marquardt method, we need to pass through the Gauss-Newton (GN) method. Before introducing the LM algorithm, we briefly discuss the Gaussian-Newton (GN) algorithm as the former is an extension of the latter:

• Gauss-Newton (GN):

Let us define the residual $r(\boldsymbol{\theta}) = (r_1(\boldsymbol{\theta}), \ldots, r_N(\boldsymbol{\theta}))^T$, where $r_n(\boldsymbol{\theta}) = \widehat{y}_n(\boldsymbol{\theta}) - y_n$ are the so-called residuals of the NLLS problem. The GN algorithm can be derived by linearly approximating $r(\boldsymbol{\theta})$ around a given estimate of $\boldsymbol{\theta}^k$(i.e. $\boldsymbol{\theta}^k$ is the value of θ in the kth iteration and $\boldsymbol{\theta}^0$ is a user-defined parameter):

$$\boldsymbol{r}(\boldsymbol{\theta}) \approx \boldsymbol{r}(\boldsymbol{\theta}^k) + \boldsymbol{J_r}(\boldsymbol{\theta}^k)\boldsymbol{\Delta}, \tag{1.2}$$

where $Jr(\boldsymbol{\theta}^k)$ is the Jacobian matrix of r, evaluated at $\boldsymbol{\theta}^k$. The vector $\Delta = \boldsymbol{\theta} - (\boldsymbol{\theta}^k)$ is called the step vector. After substituting (1.2) into (3.3), and equating the gradient of the cost function,

we attend the normal equations:

$$\tilde{\boldsymbol{J}}_{\boldsymbol{r}}(\boldsymbol{\theta}^k)\boldsymbol{\Delta} = -\boldsymbol{J}_{\boldsymbol{r}}(\boldsymbol{\theta}^k)^T\boldsymbol{r}(\boldsymbol{\theta}^k), \tag{1.3}$$

where $\tilde{J}r(\boldsymbol{\theta}^k) = Jr(\boldsymbol{\theta})^T Jr(\boldsymbol{\theta}^k)$. Provided $\tilde{J}r(\boldsymbol{\theta}^k)$ is non-singular, a condition implicitly assumed in the Gauss-Newton algorithm, the step vector can be calculated analytically at each iteration. which, for the dimensions of the problem at hand, can be done with a minor computational effort. This algorithm can also be interpreted as an inexact full Newton algorithm where the Hessian matrix is approximated to skip the computational burden [16]. The GN algorithm works much better close to the minimum, however, it may not converge at all if it is not initialized properly [17].

• Levenberg-Marquardt (LM):

The LM algorithm [18] is an iterative technique to solve NLLS problems. This method is a combination of the Gradient Descent algorithm, that is efficient for early iterations, but performs slowly when it gets close to the best-fit values, and the GN algorithm, that is inefficient in early iterations, but performs perfectly when close to the best-fit values. The LM method uses steepest descent in early iterations and then gradually switches to the GN approach which means that in many cases it finds a solution even if it starts very far from the final minimum value and hence, it guarantees convergence. In fact, LM method can be seen as a generalization of the GN algorithm as its normal equations to estimate the fitting parameters are derived from those of the GN method with an additional damping factor that is adjusted every iteration. equations (1.3), but this time $\tilde{J}r(\boldsymbol{\theta}^k)$ is replaced by $\tilde{\boldsymbol{J}}_{\boldsymbol{r}}(\boldsymbol{\theta}^k) + \psi \times \mathrm{diag}\big(\tilde{\boldsymbol{J}}_{\boldsymbol{r}}(\boldsymbol{\theta}^k)\big)$, where ψ is a predefined parameter which may be updated at each iteration, and $diag(\tilde{J}r(\boldsymbol{\theta}^k))$ is a diagonal matrix whose entries are the elements on the diagonal of $\tilde{J}r(\boldsymbol{\theta}^k)$. When $\psi \to 0$, the calculated step vector Δ approaches the GN step obtained by solving (1.3). When $\psi \to \infty$, Δ approaches a gradient-descent step where each component is weighted according to $diag(Jr\ (\boldsymbol{\theta}^k))$.

In a normal implementation, ψ is always chosen to be different than zero at initial iterations.

For finite ψ values, it is not guaranteed that the updated $\boldsymbol{\theta}^{(k+1)}$ described in below:

$$(\boldsymbol{\theta}^{(k+1)})^T = (\boldsymbol{\theta}^k)^T + \Delta, \tag{1.4}$$

decreases the loss function. Therefore, this condition is always checked in the implementation of the LM method. If the descent-property is not fulfilled, ψ is increased, and the normal equations are again solved till the step Δ ensures a decrease in the cost function. If the descent property is fulfilled, $\boldsymbol{\theta}^{(k)}$ is updated and the value of ψ is reduced.

LM is slightly more computationally demanding than GN, but it converges for all initialization combinations that are far away from the solution, where GN often fails [19]. LM shows global convergence properties and is, therefore, the preferred method of choice in common NLLS problems and consequently for our problem [19].

Finally, to ensure an efficient fitting, especially that the optimization target function is non-convex, we consider multiple starting point optimization (i.e., multiple starting points of θ) to select the best fitting solution among the tested ones.

Chapter 2

Related Work

In literature, mathematical modelling has been widely used to model pandemics since they provide a quantitative framework that scientists can assess to build hypotheses on the potential underlying mechanisms explaining patterns in the real observed data at both spatial and temporal scales [20–24] and especially to model the COVID-19 outbreak [25–30]. Mathematical models may have high different complexity levels depending on the number of variables and parameters used to characterize the dynamic states of the system, their spatial and temporal resolutions, e.g., discrete vs. continuous time, and their design, e.g., deterministic or stochastic [31]. The two main mathematical models used to characterize pandemics are phenomenological and mechanistic models:

• **Phenomenological models** are used to describe the relationship between the patterns of the data without a specific basis on the physical laws or mechanisms using an empirical approach, i.e., phenomenological models serve to describe and explain the reasons of the observed interactions between the patterns as they are. Two useful phenomenological models to characterize pandemic growth patterns exist: the Generalized Growth Model (GGM) and the Generalized Richards Model (GRM) [31–35]. Although they are commonly used, these models have limitations and downsides because of their subjectivity. Thus, establishing the reliability and validity of these approaches can be challenging, which makes these models very subjective and requires elaborate efforts to output accurate results.

•**Mechanistic models** are used to explain patterns in the observed data while considering

key physical laws or mechanisms involved in the dynamics of the investigated problem. The Susceptible-Infected-Recovered (SIR) and Susceptible-Exposed-Infectious-Recovered (SEIR) models are the least complex mechanistic models [36], and they are frequently used to characterize pandemics [31, 37–40]. Tuen Wai Ng et al. studied the SARS pandemic and compared the SIR and SEIR models' performance in characterizing it. Then, they proposed a new SEIRP model by adding a protected state and evaluated its performance on multiple use-cases of SARS outbreak [41]. Another variant of mechanistic models are used in [42] to study the Ebola pandemic, where Tae Sug Do et al. presented a model called SLIRD to better understand the spread of the disease and tested their model on the outbreak in Nigeria. Moreover, the authors of [43] presented a calibration of a SEIR–SEI pandemic model to describe the dynamics of the 2016 Zika virus outbreak in Brazil. The authors of [26], investigated the lifting of control measurement, in Wuhan, China, especially social distancing and the effect of returning to work using an SEIR model.

Recently, the authors of [44] presented a generalized SEIR model to characterize the COVID-19 outbreak in different hot spots in China and applied inverse inference to the starting date of the outbreak. Similarly, the study in [45] examined the baseline SIR model along with the SEIR model to characterize the spread in Canada. This study modelled the social distancing through the isolation of a subset of the susceptible population. Another recent study, presented in [46], proposed a comprehensive solution to estimate the mortality due to SARS-Cov-2 infection. An SEIR mathematical model describing infection transmission and death was developed to estimate the case-fatality, the symptomatic case fatality, and the overall infection fatality ratios. The mechanistic model has been applied to different impacted regions of the world with the objective of comparing the different indicators and asses the mortality dynamics per age group. Moreover, in [47] presented an extension of the SIR model, which distinguishes between detected, undetected cases and between the different Severity of Illness (SoI) and applied it to characterize the spread in Italy. In [48], the authors presented a basic SEIR model to model the COVID-19 spread in India and used it to simulate and evaluate different scenarios of the spread based on the transmission rate and finally, they proposed potential future evolution of

the spread and possible mitigation. Along with mathematical modeling, in [49], the authors employed a machine learning algorithm namely, the decision tree algorithm, to study the indicators responsible for the evolution of COVID-19 pandemic using multi-sourced data. The study identified the population density as one of the most important determinants of infection spread. Moreover, the authors of [50] addressed the future forecasting of COVID-19 spread by identifying the threatening factors using supervised machine learning models. They focused on forecasting the number of newly infected cases, the number of deaths, and the number of recoveries in the next ten days after the training period.

To summarize, the previously mentioned studies, at some points, succeeded to model the pandemic, they only focus on specific countries or regions and their models are not generalized to fit different regions simultaneously. The fitting is also usually done using one target data-set containing the number of confirmed cases. Moreover, the presented models are usually based on simplistic models, e.g., SIR and SEIR, and do not provide an advanced overview about the different scenarios that a contaminated person may face.

Chapter 3

A Generalized Mechanistic Model for Assessing and Forecasting the Spread of COVID-19 Pandemic

3.1 Introduction

In this chapter, we introduce and explain the architecture of the proposed mechanistic model and its parameters. Also, we define the optimization problem of the model's parameters as a weighted Non-Linear-Least Square (NLLS) that intends to fit the forecasted evolution of the spread to the real data by exploiting three data-sets. Finally, we present the employed fitting algorithm and the validation methodologies that will be exploited during the results and discussion chapter to prove the efficiency of the proposed model.

3.2 Proposed Mechanistic Model

In this section, we present and discuss a generalized mechanistic model, to characterize the COVID-19 pandemic, composed of eight different states or phenotypes. These states are supposed to characterize all the life cycle of an infected case before the infection, during the infection, and after being discharged, i.e., either deceased or recovered, as shown in Fig. 3.1.

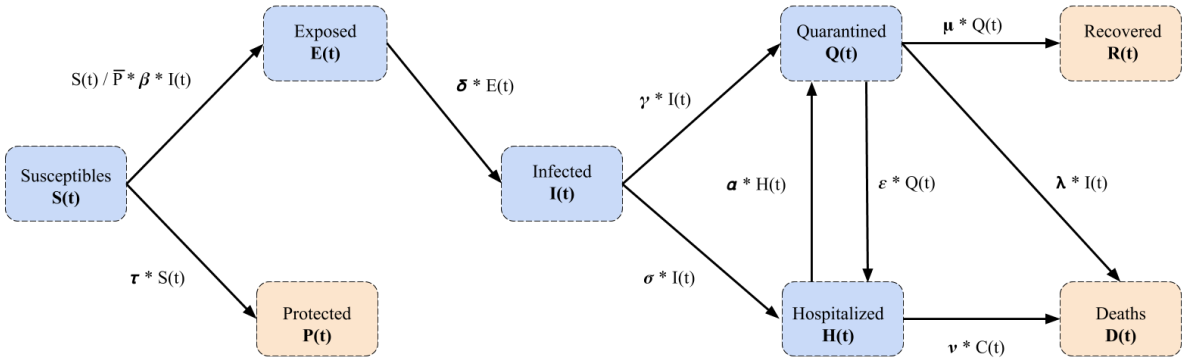


Figure 3.1: The proposed mechanistic model architecture to characterize the COVID-19 pandemic spread, where the orange boxes are the final states and other boxes are intermediate states.

Each state in our model is supposed to characterize, at a given unit of time t, in our case day, a specific population's behaviour as follows:

- $S(t)$: A state representing the total number of susceptible cases that can be infected with the COVID-19 pandemic.
- $P(t)$: A state representing the total number of protected cases that are taking the required precautions, i.e., quarantine, social distancing, etc. This state represents the category of people that are very unlikely to be infected with the COVID-19.
- $E(t)$: A state representing the total population exposed to the pandemic and are infected but not yet reported. The population within this state are supposed to be infected but not yet be infectious, i.e., in a latent period. An exposed case can be, or not, reported.
- $I(t)$: A state representing the number of confirmed infected cases with infectious capabilities that are not respecting the stay-home rules, if exist, and propagating the disease.
- $Q(t)$: A state representing the number of confirmed cases that are quarantined and are not able to infect other people. In this paper, we suppose that not all the confirmed cases are quarantined.
- $H(t)$: A state representing the number of confirmed cases that are hospitalized. This state includes the cases who require an Intensive Care Unit (ICU) bed to be treated. In this paper, we suppose that the hospitalized cases are quarantined and hence, they do not transmit the disease to other people.

- $D(t)$: A state representing the total number of deaths due to the COVID-19 pandemic.
- $R(t)$: A state representing the total number of recovered cases from the COVID-19 pandemic. We assume that the recovered cases are supposed to gain an immunity against the disease and cannot be infected again [51].

Given the definitions of the states presented earlier, we define that the total population, denoted by $\bar{P}$, as $\bar{P} = S(t) + P(t) + E(t) + I(t) + Q(t) + H(t) + R(t) + D(t)$ for each day t. We can also define the total number of confirmed infected cases, denoted by I^{rep}, and the total number of infected but not reported cases, denoted by I^{Nrep}, for each day t, as follows:

$$I^{\text{rep}}(t) = I(t) + H(t) + Q(t), \quad \forall t, \tag{3.1a}$$

$$I^{\text{Nrep}}(t) = E(t), \quad \forall t. \tag{3.1b}$$

To characterize the possible interactions between the previously presented states, we represent each possible transition from one compartment to another with an arrow as shown in Fig. 3.1. Each arrow is labeled with a rate describing the amount of time required for the transition to take place multiplied by the population of the group of individuals that the transition applies to [52]. The rates between the different compartments are defined as follows:

- δ^{-1}: The length of the incubation period, i.e., the average period from exposure to symptoms onset.
- σ^{-1}: The expected amount of time during which the health of an infected person will get worse and require hospitalization.
- γ^{-1}: The average quarantine period.
- α^{-1}: The average period for hospitalized cases to be moved from ICU bed to quarantine.
- ϵ^{-1}: The expected number of days for a quarantined case to get worse and require hospitalization.
- λ: The infected population's fatality rate.
- ν: The hospitalized population's fatality rate.

- χ: The quarantined population's fatality rate.
- μ: The quarantined population's recovery rate.

We also employ two other parameters to characterize the disease and the possible protection methods that healthy people may employ to protect themselves:

- τ: A protection rate to quantify the possible protection measurements, i.e., social distancing, mask usage, quarantine respect, etc.
- β: An infection rate reflecting the expected number of people that an infected person infects per day.

In our model, we suppose that an infected case can be healed only after passing by the quarantine state because the COVID-19 usually has symptoms, even light ones, that will oblige the contaminated persons, sooner or later, to stay home. In other terms, our assumption is to suppose that the COVID-19 symptoms with different degrees (light to severe) will appear on all the infected cases and will force them to stay home, i.e., quarantined for at least one day before being totally healed from the diseases. Since our model is based on a constant infection rate β, considering that an infected person will be placed on quarantine before its final recovery will allow our model to learn that an infected case in state I will not affect people daily, i.e., this assumption will make our model closer to reality by allowing it to automatically learn that an infected person may or may not infect people daily and will help, when needed, determine accurate basic reproduction number values regardless the constant parameters.

Moreover, we assume that an infected case may require intensive care as soon as its contamination is confirmed and hence, can be immediately hospitalized in state H. The infected cases may also choose to be quarantined in state Q and hence, will not be able to spread the disease or they may also keep contaminating other people and spreading the disease in state I. We also assume that hospitalized cases are quarantined by default so they do not spread the virus but they may pass away while in intensive care, otherwise they may be moved to the quarantine state Q before their final discharge, i.e., either to recovered state R or the death state D. Moreover, in the model, we suppose that an infected case may move in loop between the quarantined and hospitalized states multiple times. Also, we assume that infected cases may

die while being on quarantine or hospitalized at any instant of time otherwise they will recover. Finally, our analysis exclusively encloses the COVID-19 cases and hence, we only consider the death cases that are only due to the COVID-19.

The proposed model, shown in Fig. 3.1, works as a closed system and is designed to track the local spread inside multiple countries to help officials take convenient protective policies against this pandemic such as border opening dates and stay-home orders. It can be translated into an ODE system that resumes the variation speed between the different states. The ODE system is expressed as follows:

$$\frac{\partial S(t)}{\partial t} = -\beta \cdot \frac{S(t)}{\bar{P}} \cdot I(t) - \tau \cdot S(t), \tag{3.2a}$$

$$\frac{\partial P(t)}{\partial t} = \tau \cdot S(t), \tag{3.2b}$$

$$\frac{\partial E(t)}{\partial t} = \beta \cdot \frac{S(t)}{\bar{P}} \cdot I(t) - \delta \cdot E(t), \tag{3.2c}$$

$$\frac{\partial I(t)}{\partial t} = \delta \cdot E - \gamma \cdot I - \lambda \cdot I - \sigma \cdot I, \tag{3.2d}$$

$$\frac{\partial Q(t)}{\partial t} = \gamma \cdot I(t) + \alpha \cdot H(t) - \epsilon \cdot Q(t) - \chi \cdot Q(t) - \mu \cdot Q(t), \tag{3.2e}$$

$$\frac{\partial H(t)}{\partial t} = \sigma \cdot I(t) + \epsilon \cdot Q(t) - \alpha \cdot H(t) - \nu \cdot H(t), \tag{3.2f}$$

$$\frac{\partial R(t)}{\partial t} = \mu \cdot Q(t), \tag{3.2g}$$

$$\frac{\partial D(t)}{\partial t} = \lambda \cdot I(t) + \chi \cdot Q(t) + \mu \cdot H(t). \tag{3.2h}$$

Table 3.1: High-level comparison of the proposed model and previous studies on COVID-19 spread.

Feature	**Proposed work**	**[13]**	**[53]**	**[54]**
Number of states	8	4	4	6
Protected state	✓	✗	✗	✓
Quarantine state	✓	✗	✗	✓
Hospitalized state	✓	✗	✗	✗
Weighted model	✓	✗	✗	✗
Last trends favored	✓	✗	✗	✗

An initialization of the system is needed in order to solve the ODE system (3.2). The initial conditions will be denoted as follows by S_0, P_0, E_0, I_0, Q_0, H_0, D_0, and R_0 for $S(t)$, $P(t)$, $E(t)$, $I(t)$, $Q(t)$, $H(t)$, $D(t)$, and $R(t)$, respectively. This model will be used to characterize the spread of the COVID-19 pandemic. To this end, curve fitting will be applied using three target data sets to estimate the parameters of the model. Once estimated, the ODE will be solved by integrating each equation in (3.2) over the desired period to determine the population of each state with respect to the initial conditions provided.

Lastly, our proposed model has many novel contributions compared to previous studies on COVID-19 spread. In Table 3.1, we introduce a high-level comparison between our model and those used in these studies.

3.3 Curve Fitting Model

The goal of this section is to present the curve fitting model and discuss the algorithm employed to solve the corresponding optimization problem, i.e., the Levenberg-Marquard method. Finally, we discuss the employed techniques to validate the efficiency of our model.

The optimization phase is based on three target real-world data set (real data or observations), collected from official data sources. Each data set represents a vector of length N where N is the number of training days, i.e., period during which real data is observed and collected.

The vectors are defined as follows:

- Deaths (D_r): the elements of this vector indicate the total number of confirmed deaths due to COVID-19 at each day officially reported.
- Recovered (R_r): the elements of this vector indicate the total number of recovered cases at each day officially reported.
- Infected (I_r): each element of I_r contains the total number of confirmed infected cases officially reported. This accounts for previous infected cases plus the new reported cases that day.

In this paper, we investigate a multiple data set optimization problem, in which we aim to fit the curve of the model to the actual data by minimizing an objective function that takes into consideration the errors between every real and estimated data at each training day. In the sequel, we denote by n the nth training day where $n = 1, \ldots, N$. Hence and for notation purposes only, we use this discrete notation n instead of the continuous one t used in (3.2). Finally, we propose to employ a weighted Non-Linear Least Squares (NLLS) method where the objective function that calculates the Mean Squared Error (MSE) between the predicted and the real data values given the estimated values of the fitting variables is expressed as follows:

$$\widehat{\boldsymbol{\theta}} = \arg\min_{\boldsymbol{\theta}} \sum_{n=1}^{N} w_n \big((\widehat{I}_r(n, \boldsymbol{\theta}) - I_r(n))^2 + (\widehat{R}(n, \boldsymbol{\theta}) - R_r(n))^2 + (\widehat{D}(n, \boldsymbol{\theta}) - D_r(n))^2 \big), \tag{3.3}$$

where $I_r(n)$, $R_r(n)$, and $D_r(n)$ are the nth element of the real data of the infected, recovered, and deaths vectors, respectively. The estimated vectors $\widehat{I}_r(n, \boldsymbol{\theta}) = \widehat{I}(n, \boldsymbol{\theta}) + \widehat{Q}(n, \boldsymbol{\theta}) + \widehat{H}(n, \boldsymbol{\theta}) + \widehat{R}(n, \boldsymbol{\theta}) + \widehat{D}(n, \boldsymbol{\theta})$, $\widehat{R}(n, \boldsymbol{\theta})$, and $\widehat{D}(n, \boldsymbol{\theta})$ are the predicted data of the total number of $I_r(n)$, $R_r(n)$, and $D_r(n)$ given a value of a predict vector $\boldsymbol{\theta}$, respectively. The optimized vector $\widehat{\boldsymbol{\theta}}$ includes the list of all target parameters as follows:

$$\widehat{\boldsymbol{\theta}} = [\widehat{\beta} \;\; \widehat{\delta} \;\; \widehat{\rho} \;\; \widehat{\mu} \;\; \widehat{\lambda} \;\; \widehat{\chi} \;\; \widehat{\alpha} \;\; \widehat{\sigma} \;\; \widehat{\epsilon} \;\; \widehat{\nu} \;\; \widehat{\tau}]^T, \tag{3.4}$$

and finally, w_n are the weights of the data where $w_n \in [0, 1]$ and $\sum_{n=1}^{N} w_n = 1$. Notice that the $\widehat{I}_r(n, \boldsymbol{\theta})$ is expressed as the sum of all the states that involve infected, recovered, and deceased persons as per the definition of $I_r(n)$. The idea of employing weights is to endorse the fitting process and force the model to prioritize the last trends of the spread for each country and prevent the initial periods from miss-leading the model, and hence, ensure that the forecast follows the same latest trends to provide accurate forecast results that are on the verge of reality. While uniform weights maintain the same importance level for all the observed results for each day, the piecewise constant, linear, and exponential weights will increase the importance of the latest observed results and neglect the earliest ones uniformly, linearly, and exponentially, respectively. A convenient choice of the weights, that does not completely neglect the first observations, will not necessarily lead to an overfitting since the predictions for a given day t are highly correlated with the data of the last recent period. In other words, the number of active infected cases reported in the recent few past days are those who are going to spread the disease and contaminate a part of the susceptible population S. This correlation relationship between day t and the previous days is perfectly modeled using the weights that will give a higher importance with respect to time so that our model will learn two major properties to help the forecast: i) the smooth evolution of the weights will help optimize the parameters of the model and so learn the relationship between the states and ii) the last trends will help achieve accurate forecasting results.

3.3.1 Model Weights

The objective function of the fitting problem is weighted according to the training days. Hence, the values of the weights will allow the possibility to orient the focus of the fitting model towards some specific days more than the others. This will allow, for example, to relax the focus on the fitting model on the first periods of the training where the number of infected cases is very low. Indeed, the ultimate objective is to predict the evolution of the COVID-19 spread right after the training phase. In this paper, we test the following three configurations of

weights:

• Uniform function: this is the typical model, which is investigated in most of the previous studies. In this model, a uniform weight is assigned to each training day, i.e., $w_n = \frac{1}{N}, \forall n$. Hence, all the days are treated similarly by the fitting model.

• Piecewise constant function: in this case, some consecutive days are given additional priority more than the others, e.g., $w_1 = \ldots, = w_{n^0} = p_1$ and $w_{n^0+1} = \ldots, = w_N = p_2$ where $n^0 p_1 + (N - n^0)p_2 = 1$, $p_1 < p_2$, and $n^0 \in [1, N]$. In this example, the fitting model will focus less on the n^0 first days than the remaining days. Hence, less fitting errors will be tolerated for the first days.

• Non-decreasing function: in this model, the first day will be assigned the least weight value while the last day will be assigned the highest weight value. The weights are assigned to the remaining days following a non-decreasing function, e.g., linear or exponential function. Hence, the fitting model will progressively give more importance to last days. This is a very important choice since the first days of the pandemic are usually characterized with a low number of deaths, infected cases, and recovered persons. Moreover, as we aim to forecast the future evolution of the pandemic, it is worthwhile to have accurate fitting at the end of the fitting period so as to obtain exact trends at the last portion of each data.

3.3.2 Optimization Problem

The ODE system presented in (3.2) could be expressed in the following non-linear matrix form:

$$\dot{\boldsymbol{X}}(t) = \boldsymbol{A} \times \boldsymbol{X}(t) + \boldsymbol{B} \times (S(t) \cdot I(t)), \quad \forall t, \tag{3.5}$$

where

$$\boldsymbol{X}(t) = [S(t)\, P(t)\, E(t)\, I(t)\, Q(t)\, H(t)\, R(t)\, D(t)]^T, \tag{3.6a}$$

$$\dot{\boldsymbol{X}}(t) = [\dot{S}(t)\, \dot{P}(t)\, \dot{E}(t)\, \dot{I}(t)\, \dot{Q}(t)\, \dot{H}(t)\, \dot{R}(t)\, \dot{D}(t)]^T, \tag{3.6b}$$

$$\boldsymbol{B} = \frac{\beta}{\bar{P}}[-1\,0\,1\,0\,0\,0\,0\,0]^T, \tag{3.6c}$$

Note that $\dot{\boldsymbol{X}}(t)$ is the derivative with respect to time of $\boldsymbol{X}(t)$. Finally,

$$\boldsymbol{A} = \begin{pmatrix} -\tau & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ \tau & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & -\delta & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & \delta & -(\gamma+\lambda+\sigma) & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & \gamma & -(\mu+\epsilon+\chi) & \alpha & 0 & 0 \\ 0 & 0 & 0 & \sigma & \epsilon & -(\alpha+\nu) & 0 & 0 \\ 0 & 0 & 0 & 0 & \mu & 0 & 0 & 0 \\ 0 & 0 & 0 & \lambda & \chi & \mu & 0 & 0 \end{pmatrix}. \tag{3.7}$$

Hence, the Non-Linear Least squares optimization problem for the proposed mechanistic model for COVID-19 characterization:

$$\begin{aligned} (P) \qquad \widehat{\boldsymbol{\theta}} &= \arg\min_{\boldsymbol{\theta}} \sum_{n=1}^{N} w_n \big((\widehat{I}_r(n,\boldsymbol{\theta}) - I_r(n))^2 \\ &\quad + (\widehat{R}(n,\boldsymbol{\theta}) - R_r(n))^2 + (\widehat{D}(n,\boldsymbol{\theta}) - D_r(n))^2 \big), \end{aligned}$$

subject to:

$$\begin{aligned} \text{ODE model: } \dot{\boldsymbol{X}}(n,\boldsymbol{\theta}) &= \boldsymbol{A} \times \boldsymbol{X}(n,\boldsymbol{\theta}) \\ &\quad + \boldsymbol{B} \times \left(S(n,\boldsymbol{\theta}) \cdot I(n,\boldsymbol{\theta}) \right), \quad \forall n, \\ \text{Initialization: } P(0) &= P_0,\ S(0) = S_0,\ E(0) = E_0, \\ I(0) &= I_0,\ Q(0) = Q_0,\ H(0) = H_0, \\ D(0) &= D_0, \text{ and } R(0) = R_0. \end{aligned} \tag{3.8}$$

3.3.3 Fitting Algorithm

Due to the non-convexity of the problem, the NLLS optimization problem given in (3.8) cannot be analytically and optimally solved and therefore, we propose to employ numerical algorithms. The most used technique is to employ optimization algorithms that attempt to locate the local minima with gradient-based techniques. The main representative of this class of algorithms is the Newton algorithm. A downside of this algorithm is that it is considerably time-consuming due to the need for line searches and the computation of the Hessian matrix. To solve NLLS estimation problems, it is often better to exploit the quadratic structure of the cost function, as it is done by the Gauss-Newton (GN) [55] and Levenberg-Marquard (LM) [18]

algorithms, which are, by far, the most popular NLLS optimization algorithms. In this study, we exploit the fitting algorithm to estimate the parameters presented in (3.2) that minimize the squared error between the real data $I_r(n)$, $R_r(n)$, and $D_r(n)$ and their corresponding predicted values.

3.3.4 Fitting Algorithm Validation

The model validation is the most important step in the model building process. Our model validation process is composed of two levels:

• Graphical validation: In this level, we validate the model by comparing the original data and the fitted data in the same graph. This method is important to visualize the overall variation of the model through time.

• Numerical validation: In this level, we calculate numerical metrics to evaluate the error between the estimation and the real data and to verify the wellness of the fit. The employed metrics are classified into three types [56]:

- Scale-dependent metrics: These metrics provide insights about the difference between the predicted and measured values. The most commonly used are: Mean Absolute Error (MAE) and Normalized Root Mean Squared Error (NRMSE) normalized by the difference between maximum and minimum actual data. Note that our observed data, in practice, is not a constant data. Their expressions are given as follows:

$$\text{AE}_n = |y_n - \hat{y}_n|, \quad \forall n, \tag{3.9a}$$

$$\text{MAE} = \frac{1}{N}\sum_{n=1}^{N} \text{AE}_n, \tag{3.9b}$$

$$\text{RMSE} = \sqrt{\frac{1}{N}\sum_{n=1}^{N} (y_n - \hat{y}_n)^2}, \tag{3.9c}$$

$$\text{NRMSE} = \frac{\text{RMSE}}{\max(y_n) - \min(y_n)}, \tag{3.9d}$$

- Percentage-error metrics: These metrics provide insights about the percentage of the difference between the approximated and observed values. The most commonly used is the Symmetric Absolute Percent Error (SAPE). Its expression and its mean are given as follows:

$$\text{SAPE}_n = \frac{|y_n - \hat{y}_n|}{\left(\frac{y_n + \hat{y}_n}{2}\right)}, \quad \forall n, \tag{3.10}$$

$$\text{SMAPE} = \frac{1}{N}\sum_{n=1}^{N} \text{SAPE}_n. \tag{3.11}$$

- Relative-error metrics: These metrics provide insights about the error with regards to its real observed values. The most commonly used are the Mean Relative Absolute Error (MRAE) and the coefficient of determination R^2, which are expressed as follows:

$$\text{MRAE} = \frac{1}{N}\sum_{n=1}^{N} \frac{|y_n - \hat{y}_n|}{|y_n - \bar{y}|}, \tag{3.12a}$$

$$R^2 = 1 - \frac{\sum_{n=1}^{N} (y_n - \hat{y}_n)^2}{\sum_{n=1}^{N} (y_n - \bar{y})^2}, \tag{3.12b}$$

  where $\bar{y}$ is the mean value of the observed data.

Similarly to the percentage-error metrics, the relative-error metrics takes into account the percentage change between the estimated and the observed values of the data. However, it provides insights on the quality of the training in general by considering the order of amplitude of the data that may significantly vary between different parts of the training period. The values

of these metrics are independent of the population of the studied countries. A perfect fitting should achieve an MRAE and R^2 close to 0 and 1, respectively.

Our model is tested and validated using those two levels of validation in the following section.

3.4 Conclusion

In this chapter, we presented the proposed mechanistic model, and we explained the used weighing technique. Additionally, we defined the investigated optimization problem and the employed fitting algorithm. In the next chapter, we present and investigate the fitting and forecasting results of the proposed model using the previously mentioned methodologies to show the model's efficiency.

Chapter 4

Results & Discussion

4.1 Introduction

In this chapter, we present the simulation results, then we investigate the proposed model performance, and finally, we validate the use of the proposed fitting technique. To this end, we start by scrutinizing the COVID-19 spread in Russia to highlight the effectiveness of the LM algorithm training phase and the exploited weighing technique in the objective function. Afterward, we present the fitting results for all the other investigated countries, such as Brazil, Italy, and the USA using the proposed approach and the best weight combination. Also, we compare the performances of the employed LM algorithm to those of another fitting approach, i.e., BFGS [13].

4.2 Simulation Parameters

The proposed mechanistic model is investigated using a real world data set containing the number of reported infected, deceased, and recovered cases for the different countries. The Novel Coronavirus (COVID-19) Cases Data data set used in this paper is obtained from The Humanitarian Data Exchange website[1]. The initialization values of each country are given in Table 4.1. In this study, the training period starts on January 22nd, 2020 and ends on June 4th, 2020 unless otherwise stated. For some countries, we suppose that $E_0 = 1$ even though no

[1] https://data.humdata.org/dataset/novel-coronavirus-2019-ncov-cases

Table 4.1: Initialization of the ODE model states in millions for each investigated country

Country	S_0	P_0	E_0	I_0	Q_0	H_0	R_0	D_0
Russia	145	0	10^{-6}	10^{-6}	0	0	0	0
Italy	60	0	10^{-6}	10^{-6}	0	0	0	0
Brazil	212	0	10^{-6}	10^{-6}	0	0	0	0
USA (Phase I)	331	0	10^{-6}	10^{-6}	0	0	0	0
USA (Phase II)	212	118	1	2.06	0.58	0.14	0.55	0.11

confirmed cases are reported, namely for the case of Russia and Brazil, to take into consideration the delay of an exposed case to be reported, i.e., the time from being exposed to the virus until the appearance of the symptoms. Finally, the estimated parameters obtained from the fitting are used to forecast the future pandemic spread for each country by integrating the ODE equations over the desired period. The selected countries are chosen due to their high number of confirmed cases reported to WHO. They are classified among the highest infected countries and witnessing varying trends, which might be challenging for our model. The forecast of the spread for each country is over a period $\Delta t = 365$ days, starting from January 22nd, 2020. This work is implemented using python 3.7. As for the ODE integrator, we use the SciPy library where we employ the 'odeint' function that solves systems of first-order ordinary differential equations using 'lsoda' implementation of the FORTRAN library 'odepack'. For the fitting, we employ the python library 'Lmfit' that solves the formulated optimization problem (P) using a predefined algorithm (e.g., LM and BFGS algorithms) Taking advantage of the 'odeint' function, we use a multi-start optimization to run the 'Lmfit' and then, randomly select initialization values achieved the best fitting performance (i.e., minimizing the objective function of the optimization problem (P)). Note that, we selected the same initialization values for both fitting algorithms (LM and BFGS).

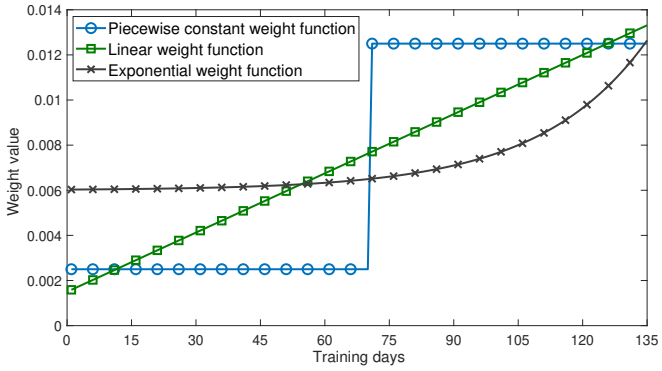


Figure 4.1: Plots of the investigated normalized weight functions.

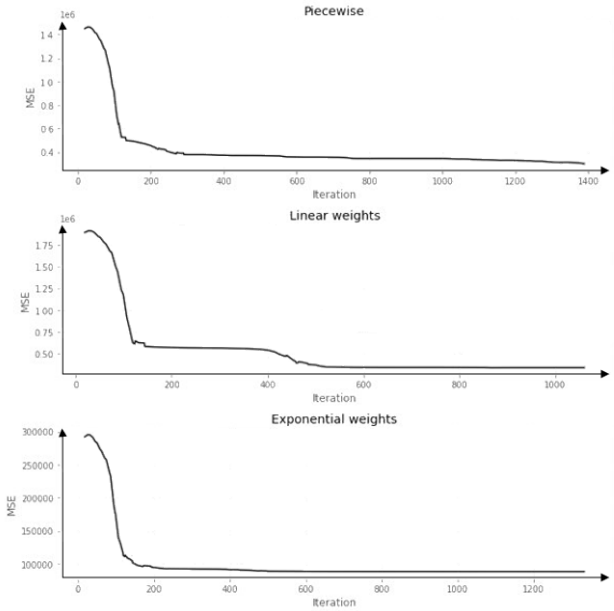


Figure 4.2: Weighted MSE evolution during the training phase for each weight function.

4.3 Training Phase and Weights Impact: The Case of Russia

During this phase, we employ the LM algorithm to solve the fitting optimization problem and estimate the ODE system parameters. To this end, we investigate the effect of weights employment in the fitting process. In this paper, three forms of weights are considered: piecewise constant, linear, and exponential weights. The expressions of the employed normalized weight functions are given, respectively, as follows:

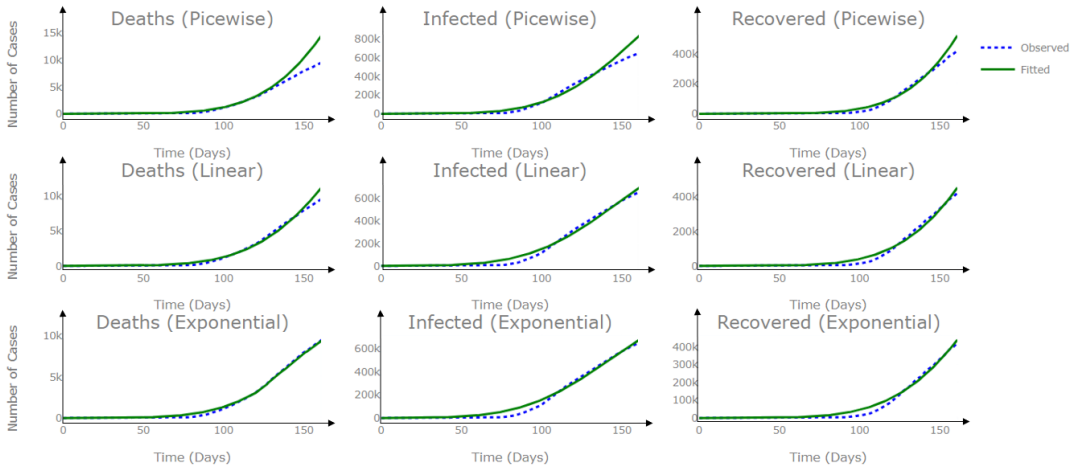


Figure 4.3: Russia: Graphical validation of the LM algorithm for different weight functions.

Table 4.2: Russia: Numerical validation of the LM fitting algorithm using the different weight functions

Weight Function	**MAE**	**NRMSE**	**SMAPE**	**MRAE**	R^2
Piecewise	89476.19	0.78	3.13	1.12	0.862
Linear	68215.24	0.43	2.98	0.86	0.903
Exponential	**44215.39**	**0.21**	**1.22**	**0.58**	**0.947**

$$\forall n = 1, \ldots, N,$$

$$w_n^{\text{pw}} = \begin{cases} 25 \cdot 10^{-4} & \text{if } n < 70 \\ 125 \cdot 10^{-4} & \text{if } n \geq 70 \end{cases} \tag{4.1a}$$

$$w_n^{\text{lnr}} = A_l n + B_l, \text{ where } A_l = 8.75 \cdot 10^{-5}, B_l = 1.5 \cdot 10^{-3}, \tag{4.1b}$$

$$w_n^{\text{exp}} = A_e \exp^{(B_e n)} + C_e, \text{ where } A_e = 3 \cdot 10^{-5},$$

$$B_e = 4 \cdot 10^{-2}, C_e = 6 \cdot 10^{-3}. \tag{4.1c}$$

Note that the weight functions are obtained after normalization over the number of training days ($N = 135$ days) and that the sum of their values is equal to 1. Fig. 4.1 plots the curves corresponding to the considered normalized weight functions.

In Fig. 4.2, we represent the variation of the MSE error with respect to the training iterations for the case of Russia employing the three different weight functions and the LM algorithm.

It is shown that, in all cases, the LM reaches stagnation at different minimum values, which confirms the convergence of the model. However, these values are very close to each other for all weight functions. Note that the plotted MSE in this figure considers the different data sets all together, i.e., the infected, deceased, and recovered data. Indeed, it plots the achieved weighted objective function expressed in (3.3) during the training process.

Graphically, we visualize the trained fitting of the different observed states separately using the LM algorithm for the different functions of the weights in Fig. 4.3. By observation, these graphs confirm that the exponential and linear weights outperform the piecewise constant function of the weights. Indeed, although the fitted data do not exactly fit the real data at the beginning of the pandemic, it successfully follows it at the latest training days, which makes the models ready to forecast with respect to these trends.

The fitting results for each weight function using the LM algorithm are numerically corroborated in Table 4.2 where we provide the achieved average values for the different validation metrics. The table shows that the use of exponential weights allowed to obtain less error than the other investigated weights in terms of all the evaluation metrics: state-dependent, percentage-error, and relative-error metrics. For instance, the MAE with the exponential weight provides is around 44215 which is 51% less than the MAE obtained with the piecewise constant weight function. In terms of R^2, that reflects the goodness of fit, we can notice that the exponential function of the weights provides the highest value 0.94, which is very close to 1, compared to the R^2 achieved by the piecewise constant weight function (i.e., $R^2 = 0.86$). This proves the efficiency of the fitting of the exponential weights. Indeed, as mentioned earlier, the non-decreasing functions of the weights allow the fitting algorithm to focus more on the latest training days in progressive way and due to the nature of the pandemic which starts with very small number of cases, the fitting errors are extremely reduced for relatively high number of infected, deceased, and recovered cases.

Finally, in Fig. 4.4, we plot the daily values for each training day of the APE, RAE, and AE metrics using the exponential function of the weights. The figure clearly illustrates the focus of the fitting on the last days of the training. Indeed, the SMAPE is significantly reduced for

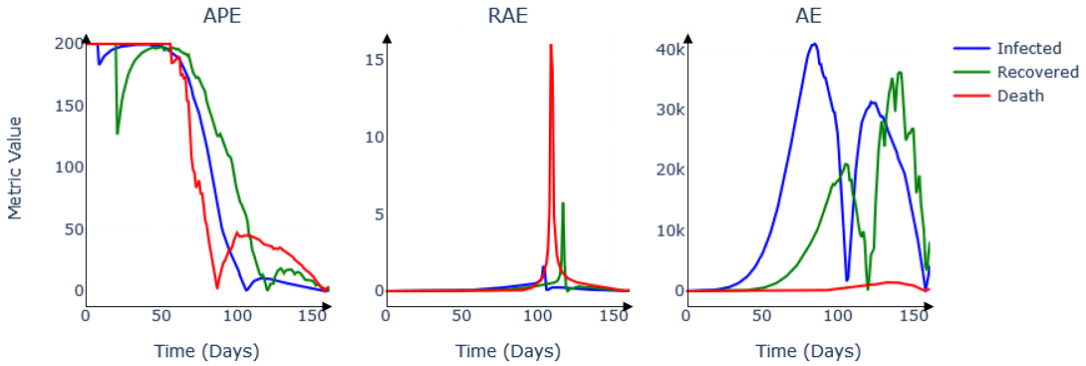


Figure 4.4: Russia: Illustration of the values of the APE, RAE, and AE metrics versus the training days using the exponential weight function.

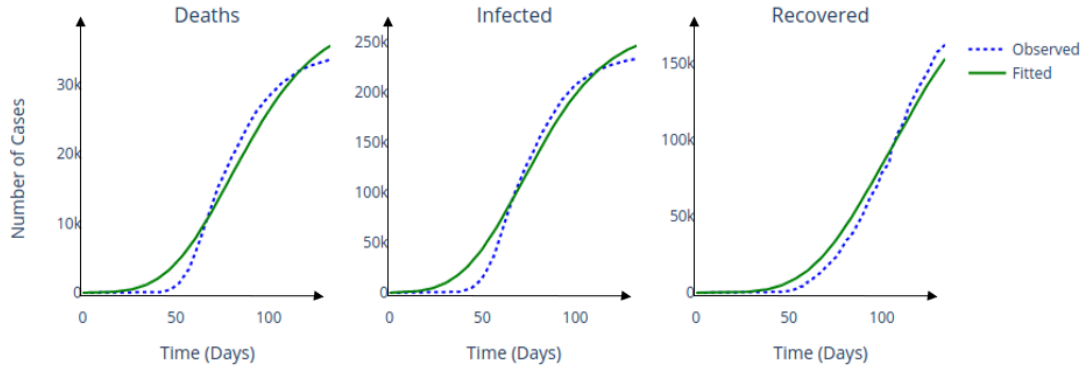


Figure 4.5: Italy: Graphical validation of the LM algorithm using the exponential weight function.

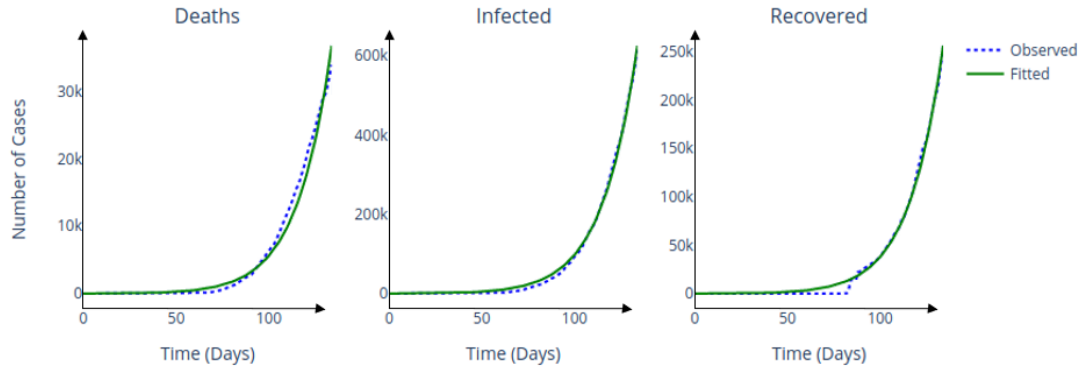


Figure 4.6: Brazil: Graphical validation of the LM algorithm using the exponential weight function.

the different states during that period. The RAE is again confirming the efficiency of the fitting where the relative-error is less than 2% for the whole training period except some particular spots as the fitting is not exactly perfect. Finally, the AE highlights the absolute error for each day and state. Although it may reach the order of thousands, this error remains small compared to the real total number of infected and recovered cases.

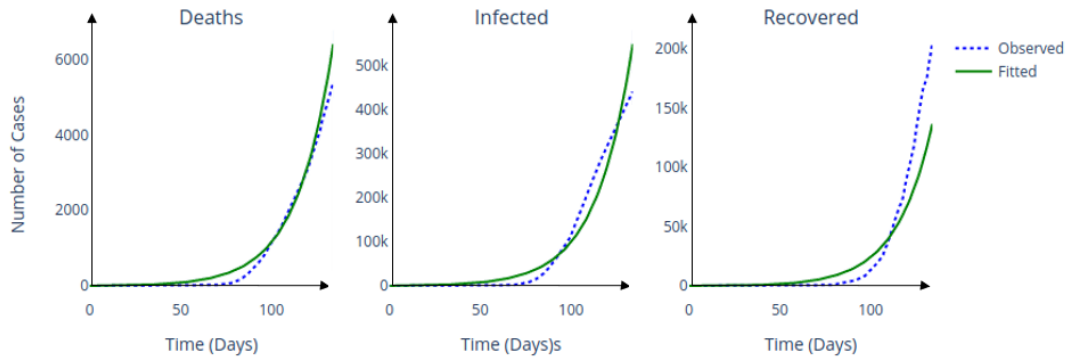


Figure 4.7: Russia: Fitting results of the BFGS algorithm using the exponential weight function.

Throughout the rest of this section, we proceed with the exponential function of the weights to perform the fitting process.

4.4 Fitting Performance and Validation

In this section, we present the results of the proposed weighted fitting technique using the LM algorithm for different countries. Afterwards, we compare the efficiency of the proposed fitting model to another fitting algorithm, i.e, the BFGS.

4.4.0.0.1 The Cases of Russia, Italy, and Brazil

We propose to provide the fitting results for several countries to prove that our model is capable of providing accurate fitting in many situations. Indeed, we chose Italy as one of the first affected countries witnessing one of the highest death rates during the first four months of the pandemic, and which is showing now stable and much safer situation. Russia is also a very interesting scenario to study due to its unique spread profile (i.e., Russia witnessed a tremendous spike in COVID-19 virus infections after a period characterized with a low infection rate). Finally, we investigate the case of Brazil, which is currently witnessing its spike.

The results presented in Fig. 4.3, Fig. 4.5, and Fig. 4.6 show that the model can adapt the last trend of the data for all previously mentioned countries with some minor errors. Indeed, due to the existence of three different data sets having different orders of magnitude and trends, the fitting algorithm may not fit exactly all the curves but globally the fitting is effective as

Table 4.3: Numerical validation of the LM fitting algorithm using the exponential weight function for Russia, Italy, and Brazil

Country	**MAE**	**NRMSE**	**SMAPE**	**MRAE**	$\boldsymbol{R}^2$
Russia	44215.39	0.21	1.22	0.58	0.947
Brazil	62419.17	0.277	1.12	0.34	0.995
Italy	9769.24	1.81	1.3	0.94	0.951

confirmed by the achieved metrics presented in Table 4.3. Italy is showing the weakest results among the three investigated countries. Although it achieves similar R^2 as Russia, the fitting of Italy reaches around twice the MRAE value obtained with Russia. It is also confirmed graphically in Fig. 4.5 for the infected and due its small population. Notice also the MAE of Italy is low compared to Russia and Brazil because of the low number of infected cases reported as well as its small population. The fitting of Brazil is the most accurate as confirmed graphically and with the achieved evaluation metrics ($R^2 = 0.99$ and MRAE $= 0.34$). Minor errors are noticed mainly during the first period of the pandemic as tolerated by the utilized weight function.

Furthermore, in this study, we compare the performance of the proposed fitting algorithm to the one of another fitting algorithm employed in [13] where the BFGS algorithm is used to resolve the fitting problem. In Fig. 4.7, we plot an example of the fitting for the case of Russia using the exponential weight function and BFGS algorithm. It is shown that the BFGS partially fails in fitting all the three data sets, especially the latest days of the infected and recovered cases. In Table 4.4, we provide a performance comparison between the LM and BFGS algorithms for all the considered metrics and the investigated countries. We notice that LM considerably outperforms the BFGS especially in Italy and then, Russia. However, both algorithms achieve similar results for the case of Brazil with a certain advantage for the BFGS in terms of MAE. In general, the LM algorithm provides much more accurate results than the BFGS.

4.4.0.0.2 The Case of USA

In this study, we consider the USA as a special case. Indeed, unlike other countries which

Table 4.4: Comparison between the LM and BFGS fitting algorithms using the exponential weight function for Russia, Italy, and Brazil

Country	**Algorithm**	**MAE**	**NRMSE**	**SMAPE**	**MRAE**	R^2
Russia	BFGS	488155.78	1.24	1.5	1.72	0.857
	LM	**44215.39**	**0.21**	**1.22**	**0.58**	**0.947**
Brazil	BFGS	45801.68	0.122	1.04	0.284	0.994
	LM	**62419.17**	**0.277**	**1.12**	**0.34**	**0.995**
Italy	BFGS	122355.74	2.45	2.7	1.24	0.116
	LM	**9769.24**	**1.81**	**1.3**	**0.94**	**0.951**

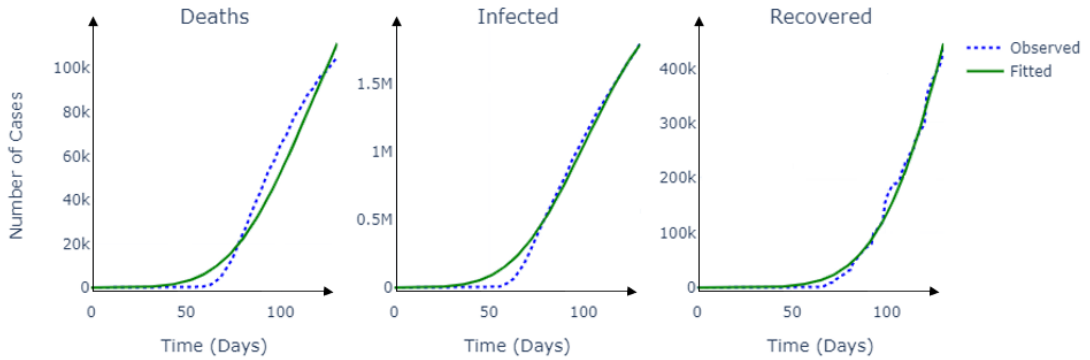


Figure 4.8: USA (Phase I): Graphical validation of the LM algorithm using the exponential weight function for the first period of the pandemic (January 22nd, 2020 till June 13th, 2020).

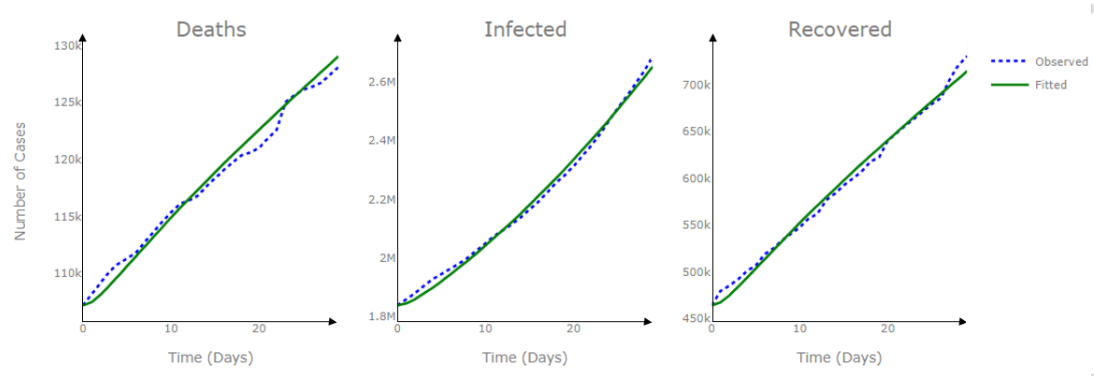


Figure 4.9: USA (Phase II): Graphical validation of the LM algorithm using the exponential weight function for the first period of the pandemic (June 13th, 2020 till July 7th, 2020).

are controlled by a single authority, the USA is composed of fifty different states. Each one of them is governed independently of the other and in a different manner. For example, some states are imposing the lockdown, e.g., NY and NJ, and the stay home order at the beginning of the pandemic (early March) while others do not or impose it tardily, e.g., FL. Moreover, the Black lives matter protests on different US states were unpredictable and their impact on the spread can not be measured and/or characterized and hence, this can have a huge impact on the

Table 4.5: USA: Numerical validation of the LM fitting algorithm using the exponential weight function

Country	MAE	NRMSE	SMAPE	MRAE	R^2
USA (Phase I)	84589.82	0.425	1.09	0.59	0.992
USA (Phase II)	22141.24	0.312	0.94	0.46	0.993
USA (Phase I & II)	103257.24	2.877	1.79	1.83	0.363

spread of the COVID-19. Hence, treating the US as a single entity is not appropriate and may lead to inaccurate results due to the heterogeneity of the states. Therefore, one possible option is to investigate each state separately as a single homogeneous entity. However, due to the non-availability of the data in our database that we used with other countries, we do not include these results in this study. However, we propose to deal with the USA case by dividing the fitting period into two periods or phases so that the model will be aware that two waves exist, one dominated by the northeastern states, NY, NJ, and CT, during the first four months of the pandemic and the other is dominated by southern states such as TX, FL, and CA starting from June 13th. In this scenario, we can obtain relatively accurate fitting results as shown in Fig. 4.8 and 4.9 with $R^2 = 0.992$ and $R^2 = 0.993$, respectively, as given in Table 4.5. In the latter table, we also provide the fitting results for the whole period (Phase I and Phase II combined) where two COVID-19 waves are registered. As explained earlier, the fitting was not successful with a poor $R^2 = 0.36$. Nevertheless, along the two periods, the model successfully fits the real data and shows acceptable results despite the previously mentioned characteristics of the studied country. In other words, for each period, different parameters of the mechanistic model are estimated unlike the other studied countries in the previous sections. Hence, for the USA, we only prove that our model is able to fit the reported cases representing the spread for these two periods. Otherwise, combining the two periods will require the adoption of fitful model parameters that vary with time, e.g., $\beta(n)$ instead of constant β. This study is more elaborate and will be the focus of our future extension of this work.

The estimated parameters of the model of each country are provided in Table 4.6. These values are used to forecast the future evolution of the pandemic in the next section.

Table 4.6: Estimated model parameters for each investigated country

Parameters	Russia	Italy	Brazil	USA (Phase I)	USA (Phase II)
β	1.279	1.248	1.039	1.436	1.482
δ	0.041	0.050	0.017	0.043	0.0513
ρ	0.002	0.014	$1.5 \cdot 10^{-7}$	$1.7 \cdot 10^{-5}$	$1.8 \cdot 10^{-6}$
μ	$1.5 \cdot 10^{-4}$	0.015	$2.1 \cdot 10^{-4}$	0.004	0.001
λ	0.011	0.149	0.099	0.048	0.015
χ	0.002	$4 \cdot 10^{-12}$	$2.1 \cdot 10^{-6}$	0.001	0.003
α	0.349	0.077	0.835	0.347	1.968
σ	0.135	0.135	0.144	0.126	0.125
ϵ	1.237	0.568	$7.05 \cdot 10^{-5}$	1.527	0.909
ν	0.007	$5 \cdot 10^{-7}$	0.076	$7 \cdot 10^{-4}$	$2.5 \cdot 10^{-6}$
τ	0.016	0.023	0.002	0.020	0.025

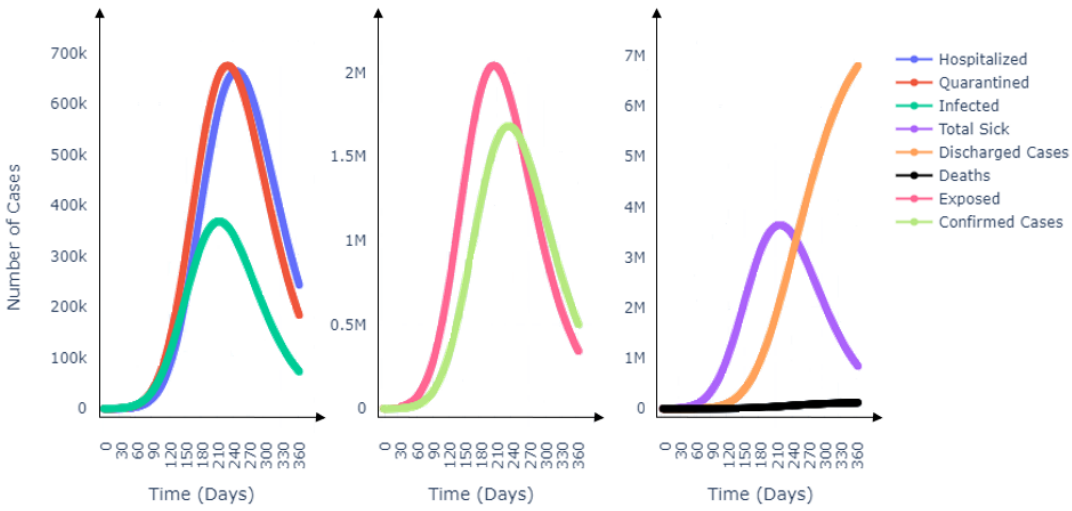


Figure 4.10: Russia: COVID-19 spread forecasting results and expected evolution of the different states based on the estimated parameters.

4.5 Expected Spread and Forecasting Validation

In this section, we extrapolate the COVID-19 spread in the studied countries using our fitted models to forecast its evolution during the next period. We also discuss in detail the evolution of each state. In addition, we provide the upper and lower bounds of the expected spread representing the worst and best case scenarios for these countries based on our model. We particularly select to investigate the forecasting results of Russia and Italy as they have already reached their peaks, unlike Brazil. The case of the USA is also omitted due to the inaccurate fitting results that we discussed the reasons earlier. Finally, we compare the forecasting

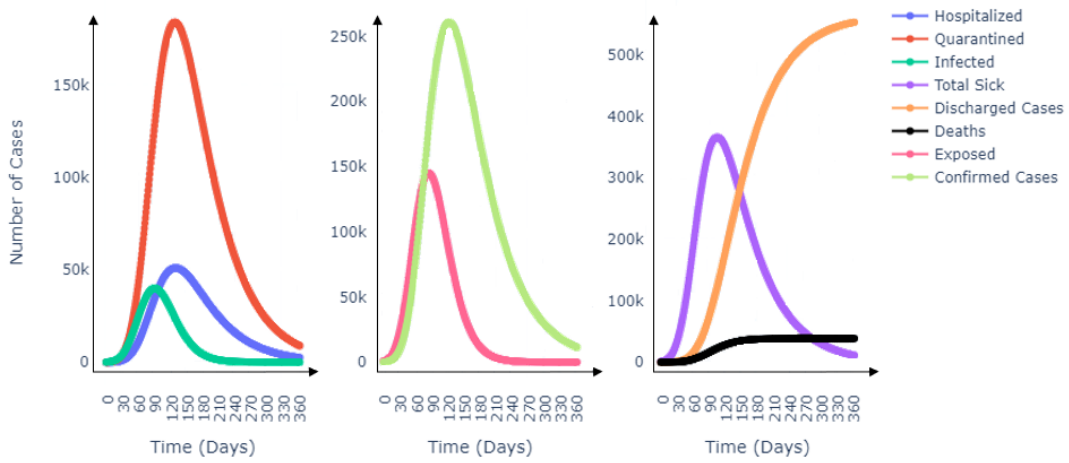


Figure 4.11: Italy: COVID-19 spread forecasting results and expected evolution of the different states based on the estimated parameters.

performance of our model to the one of the RW model.

The forecasting per state, for Russia and Italy, are shown in Figs. 4.10 and 4.11, respectively. In these figures, the total sick represents the sum of the population of the states E, I, Q, and H, that are already contaminated with the virus. As for the confirmed cases, it stands for all the reported cases represented by the states I, H, and Q. The fitting model based on the current spread parameters expects that the cumulative number of infected cases will reach about seven million cases and 250 thousands in Russia and Italy, respectively. The forecast results for Italy show that the infected cases are being reported as soon as the exposed cases start to appear as it shown by evolution of the pink (exposed cases) and light green curves (confirmed cases) of Fig. 4.11. Moreover, the big gap between the confirmed cases and the exposed cases (confirmed cases is always higher than exposed cases) reflects that the country is efficiently identifying the infected cases, which proves the effectiveness of the prevention approach employed by this country to avoid higher propagation of the virus. However, for Russia, Fig. 4.10 shows that the number of exposed cases is more important than the number of confirmed cases which explains the huge number of cumulative cases that are expected to be reported by the end of the forecast period. Nevertheless, these results may change in practice. The local authorities, in the case of Russia, may impose stricter policies which may limit the propagation of the spread. Recall that our forecast are based on the real data observed in each country and hence, the forecast will

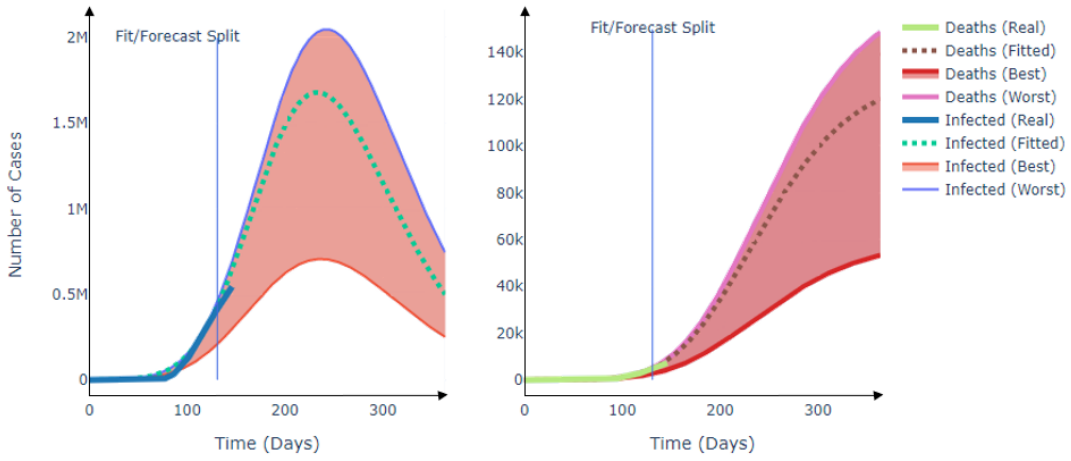


Figure 4.12: Russia: Best and worst possible spread scenarios by varying the infection rate β.

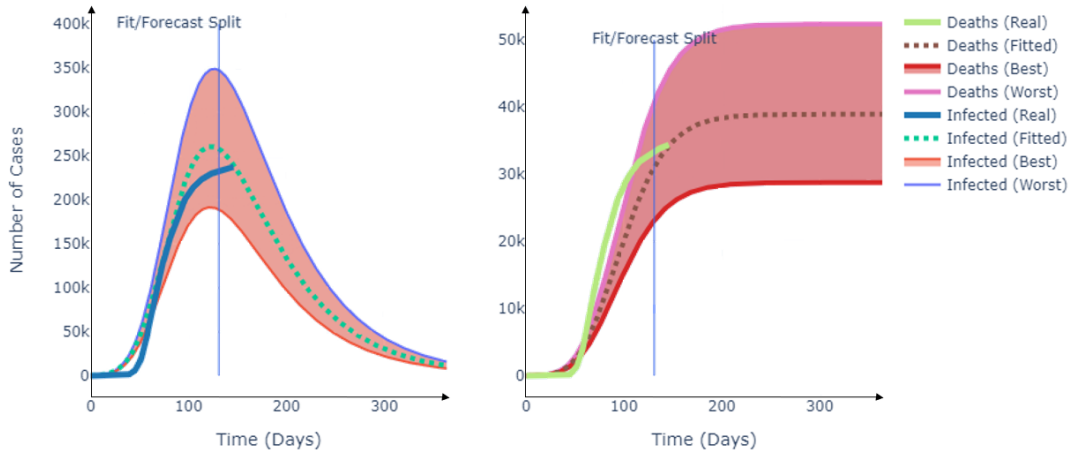


Figure 4.13: Italy: Best and worst possible spread scenarios by varying the infection rate β.

extrapolate the evolution of the model states based on the spread data. Hence, if a country is not adopting an effective strategy at the beginning of the pandemic, the estimated results might be relatively critical as it was expected by our model for Russia.

Also, in Figs. 4.10 and 4.11, we observe that the majority of confirmed cases are at the quarantine or hospitalized state, i.e., either being quarantined at home (red curve) or being quarantined at hospitals (black curve), which proves the effectiveness of the stay home orders taken in these countries and their policies to limit the spread. However, we can notice that, in Italy, the quarantining process starts around 45 days since the beginning of the training period, which is around March 10th the day when the stay home order was announced). In Russia, the lockdown starts around 75 days (end of March) and the number of cases is expected to be very

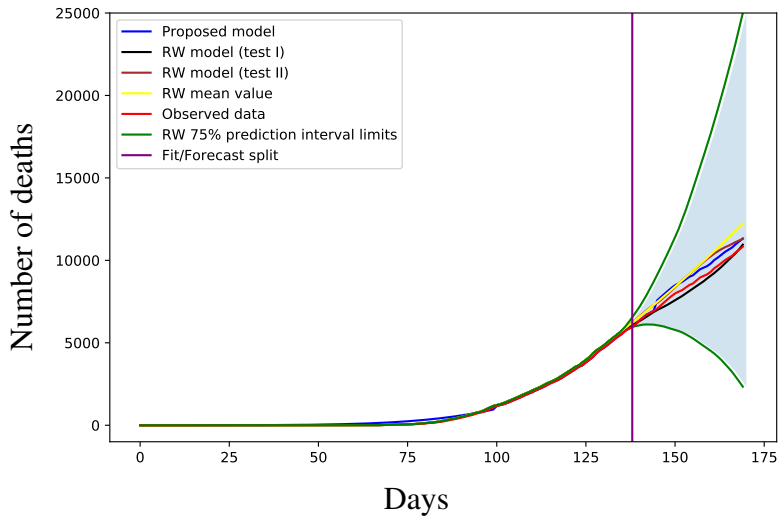


Figure 4.14: Russia: Forecast of the deceased cases. Comparisons between the proposed and the RW models.

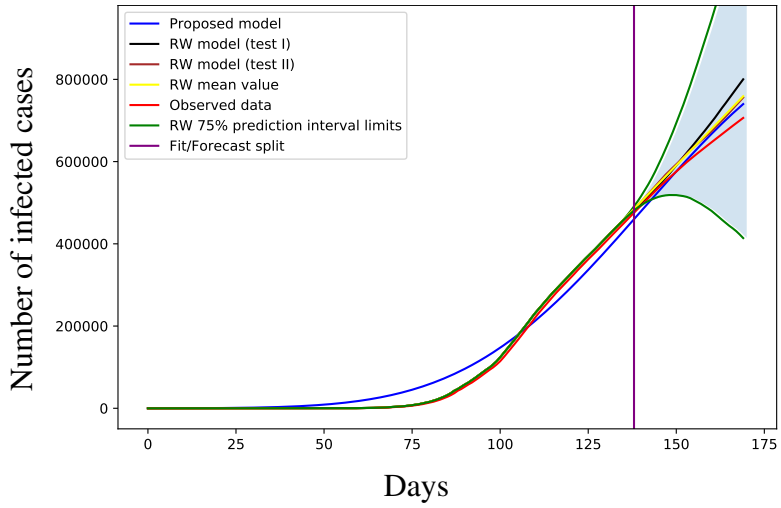


Figure 4.15: Russia: Forecast of the infected cases. Comparisons between the proposed and the RW models.

high if no actions are taken to stop the spread. By the start of 2021 we expect that few infected cases will be reported and the spread will be significantly reduced for both countries.

Since our fitting model is trained during the first four months of the pandemic spread, the forecast results may not be accurate in practice. Indeed, several policies can be further ordered and social distancing will be much better applied and hence, these actions may slow down the spread. In other scenarios, people will not conveniently apply the COVID-19 prevention guidelines anymore, which may speed up the propagation of the virus. Therefore, in Figs. 4.12 and 4.13, we provide the uncertainty regions that may characterize the COVID-19 spread in

Table 4.7: Russia: Numerical validation of the forecast results

Investigated states	**Infected**		**Deaths**	
Metrics	**NRMSE**	**SMAPE**	**NRMSE**	**SMAPE**
RW model (Test I)	0.42	0.18	0.134	0.044
RW model (Test II)	0.27	0.122	0.411	0.143
Proposed model (Exp.)	**0.099**	**0.052**	**0.066**	**0.0304**
Proposed model (Uni.)	0.19	0.075	0.084	0.051
RW mean	0.226	0.121	0.175	0.068
RW upper bound	2.188	0.680	0.744	0.350
RW lower bound	1.935	1.312	1.716	0.447

terms of number of active cases and number of deaths. The objective is to use the parameters estimated by our model and vary the infection rate, β, that reflects the daily number of contaminated person that an infected person can infect. By varying this parameter, we can evaluate the best and worst case scenarios that may occur for these countries. In Figs. 4.12 and 4.13, we also provide the observed results during the seventeen days right after the fitting period (fit/forecast split in the figure). The objective is to show that the fitting and the resulting forecast are accurate and have the same trends as it is observed in reality in terms of reported infected and deceased cases. Regarding the expected evolution, we notice that for Italy the number of active cases will significantly diminish and the number of deceased persons will reach its maximum and slightly increases. However, for Russia, if no actions are taken by the authorities and officials, our model expects that the number of infected cases (new cases plus active cases) will keep increasing until reaching a peak in August and then, it will start decreasing significantly. Moreover, the number of deaths is expected to continue growing and slowing down starting from September.

In the sequel, we provide two comparisons of our proposed model with a reference model, namely the RW approach with drift algorithm. The comparison consists of forecasting the data for the next 31 days right after the fitting period. We compare the results graphically and numerically to evaluate the efficiency of the proposed model. This comparison targets only the infected and deaths states due to the lack of data for the other states, which prevents us to build their respective RW models.

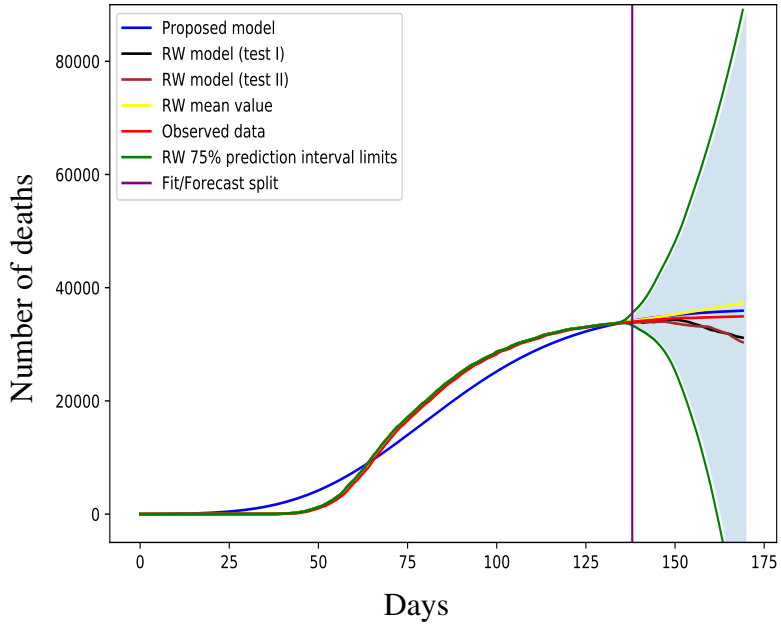


Figure 4.16: Italy: Forecast of the deceased cases. Comparisons between the proposed and the RW models.

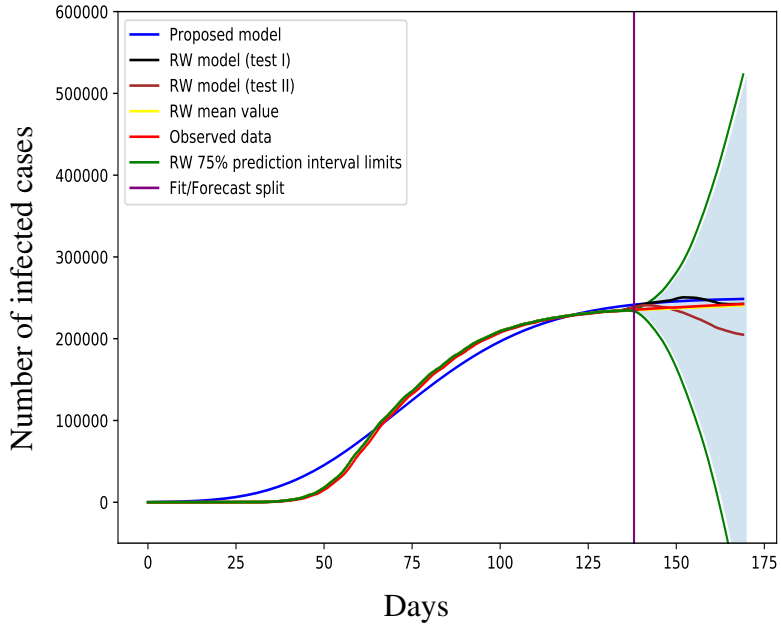


Figure 4.17: Italy: Forecast of the infected cases. Comparisons between the proposed and the RW models.

Initially, the RW model is built using the observed data during the fitting period. Then, it is used to forecast the evolution of the number of active and deceased cases. We have generated two random forecasting attempts that we denoted by RW model (Test I) and RW model (Test II). In addition to that, we have provided the forecast based on the estimated mean (RW mean)

Table 4.8: Italy: Numerical validation of the forecast results

Investigated states	**Infected**		**Deaths**	
Metrics	**NRMSE**	**SMAPE**	**NRMSE**	**SMAPE**
RW model (Test I)	19.05	0.36	3.515	0.145
RW model (Test II)	1.566	0.045	6.28	0.363
Proposed model (Exp.)	**0.80**	**0.024**	0.457	0.022
Proposed model (Uni.)	0.91	0.038	0.62	0.035
RW mean	2.47	0.063	**0.121**	**0.006**
RW upper bound	66.75	0.742	38.68	1.421
RW lower bound	84.197	1.545	36.279	0.714

and the forecasts based on a 75% prediction interval, named RW upper bound and RW lower bound where there is 75% chance that the predicted observations will fall within their ranges.

Figs. 4.14, 4.15, 4.16, and 4.17 plot the forecast number of deceased and infected cases for Russia and Italy, respectively using our proposed model (with exponential weights) and the RW model. In each figure, six curves are presented with different colors: the observed data during the whole period (red), our proposed model during the fitting and forecasting periods (black), the RW mean (yellow), the RW model (Test I) and (Test II) (black and brown, respectively), and finally, the 75% prediction interval identified by the two green limits. The curves related to the RW model are plotted during the fitting period. The split between the fitting and forecasting periods are indicated by the purple vertical line. For fairness reasons, we train the models (proposed and RW) over the same fitting period (135 days) and investigating their prediction performance for a forecasting period of 31 days.

The numerical results related to Figs. 4.14, 4.15, 4.16 and 4.17 are given in Tables 4.7 and 4.8, respectively, where the NRMSE and SMAPE metrics for each forecasting attempt are provided. The results show that the RW model has a wide range of possible manifestations due to the huge variation of the observed data during the fitting phase. A random attempt to forecast the observed data may not necessarily provide accurate results as shown in the Tables. For instance, in terms of NRMSE for the case of Russia, our model, which provides a deterministic forecast, achieves NRMSE values equal to 0.99 and 0.066 for the infected and deceased, while the RW attempts are quite far from these results as indicated in Table 4.7. Through these tables,

we also investigate the forecasting results using two different weight functions: the uniform (Uni.), no weights, and the exponential (Exp.) weights. It is shown that NRMSE and SMAPE values using the exponential weights are slightly lower than the ones obtained with the uniform weight functions. This result is due to the more accurate fitting obtained with the exponential weights that guarantees a certain correlation between the forecast observations and the fitted ones.

However, if we consider the mean value, which is very representative of the RW model, we can notice that our proposed model outperforms in most of the cases (e.g., infected cases for Italy and infected and deceased cases for Russia). In many cases, the RW model fails in accurately predicting the observed data due to its wide prediction interval even if the RW mean is showing accurate results. On the contrary, the deterministic prediction of our model has efficiently predicted the future pandemic spread.

The achieved metrics given in Tables 4.7 and 4.8 clarify and corroborate the graphical results and proves that our approach effectively outperforms the RW model in most of the cases.

4.6 Limitation of the Study

In this study, we have proposed a mechanistic model for assessing and forecasting the spread of the COVID-19 pandemic. The pandemic of interest is very novel and even though it has already spread all over the world, scientists still do not have enough knowledge to model it properly, especially due to the lack of data and evidence about its contamination process. In this study, we have made some assumptions to conceive our model, which may impact its performance. First, we have considered a closed system, which does not distinguish between local and non-local cases. Hence, the spread is assumed to be originated from a first local case or a first imported case. At the time of this study, there is no available data that provide accurate and precise statistics about non-local cases and whether they are responsible for the virus spread or not. Therefore, the model only considers the local spread of the virus and discards any cases

imported from outside of the country of interest. The study also assumes long-term immune memory. In other words, cases that are recovered are assumed to be immunized against the virus and hence, can not be contaminated again. Indeed, the available data set does not provide detailed information about these scenarios and there is no strong evidence for short-term immune memory.

Another future improvement of this study could be the concrete consideration of asymptomatic cases. In this study, we have assumed that every case is symptomatic at least for a short period of time, e.g., 1 day, and hence, each infected case must pass by a quarantine state even for a short period. Statistically, around 30% of patients who tested positive were asymptomatic and may spread the virus. In this study, we did not distinguish between these the asymptomatic and symptomatic cases due to the absence of related data for each of the investigated countries. Therefore, we have supposed that the COVID-19 symptoms with different degrees (light to severe) will appear on all the infected cases and will force them to stay home, i.e., quarantined for at least one day before being totally healed from the diseases.

This study is based on an ODE system with non-variant parameters and especially a fixed infection rate β. By considering that an infected person will be placed on quarantine before its final recovery will allow our model to learn that an infected case in state I will not affect people daily unlike typical SIR and SEIR models. This assumption will make our model closer to reality by allowing it to automatically learn that an infected person may or may not infect people daily and will help, when needed, determine accurate basic reproduction number values regardless of the constant parameters. Other works are considering a similar scenario where recovery is only possible through quarantine can be found in [44]. However, a more elaborate and accurate model would consider a time-varying infection rate. In that case, more precise results could be obtained.

Finally, our model assumes that the investigated countries have centralized governmental systems. Hence, the human decisions, such as border openings, lock-down, indoor services, and mask mandates, and their impacts on the disease spread have uniform consequences As a result, the prediction of the future evolution of the pandemic within federal countries, such

as the USA, can not be accurately performed with our model due to the fact that every state has its own regulations and policies and hence, it is more convenient to investigate each state separately and devise a generalized model for the whole country.

All these assumptions have made our forecasting results valid for a short-period of time, e.g., 30 days as shown by our forecasting results. Our study does not claim long-term prediction of the disease spread unless similar human factors and management practices are maintained.

4.7 Conclusion

This chapter presented the results of the fitting and forecasting of the proposed model. As a first part, we presented the simulation parameters and data-set exploited during the algorithm's training. Afterward, we proved the efficiency of employing the weighing technique by studying the case of Russia as a reference example. Additionally, we studied the performance of the fitting technique for the case of Russia, Italy, Brazil, and the USA. The forecasting results showed promising results in following the last trend of the real-data. Finally, the model outperforms a Random walk based model for the case of Russia and Italy.

Summary

In this study, we developed a mechanistic model composed of eight states to characterize the COVID-19 spread in different countries. An ODE system is formulated to mathematically model the interactions between the different states. A curve fitting using real-world observed data sets is developed to determine accurate fitting solutions and estimate the parameters of the ODE model. We use the LM algorithm to solve and estimate the ODE system parameters. We have shown that the employed algorithms outperform a reference model, i.e., BFGS. The fitting technique is represented along with the model that aims to endorse its forecasting results by focusing on the last trends of the data, and hence, giving accurate results that are on the verge of these trends. Multiple countries with different spread trends are investigated with the proposed work. We reveal that our model is able to fit the real-world data of different countries showing different spread trends, namely, Russia, Brazil, Italy, and the USA. Despite the challenges and the various human factors, i.e., social distancing, quarantine, wearing masks that are not directly characterized by our model and that may affect the spread and its evolution. It is shown that accurate forecasting results are obtained, e.g., Italy and Russia.

The proposed model is not only one of the rarest extended mechanistic models that include all the possible eight states that an individual may go through during the illness cycle but also it is the only model that employs a weighted fitting technique to follow the latest trends in addition that the proposed model is trained on three target data sets. However, this work can be endorsed by the employment of variable parameters that change over time to better fit the variations of the data, and hence, the challenges related to the case of the USA can be addressed.

Part II

Mitigating the Spread of the COVID-19 Pandemic

Chapter 1

Introduction & Preliminaries

1.1 Introduction

In this chapter, we investigate a smart navigation framework intending to give pedestrians, and even cyclists, safe routing to bypass areas where the risk of COVID-19 transmission is high. In other words, the framework recommends a pedestrian walking route that guarantees social distancing and avoiding close contact. The proposed approach includes four main phases.
To do so, we propose to exploit IoT devices located in the area of interest and establishes social graphs interconnecting these devices using different social IoT relations. Accordingly, we apply the Louvain community detection algorithm to the SIoT graphs to determine different communities of IoT devices. In addition, to the community detection part, the proposed framework exploits Closed-Circuit Tele-Vision (CCTV) video cameras registered in the IoT database to detect the pedestrian flow in a real-time manner and extract safety-related information about the crowds in the street, including the offline pedestrians in the decision-making during the navigation. The main goal of this phase is to compute the computer vision and the weights coming from the CLOR and SFOR communities. According to the previously calculated weights, the framework assigns scores to each street or segment of a street in the area of interest. Finally, the city map is transformed into a weighted undirected graph and a routing algorithm is applied to determine a route characterized by a certain level of safety.

1.2 Preliminaries

This section presents the proposed framework related concepts that are required to understand it clearly. The explained approaches are linked to the Internet Of Things and computer vision concepts.

1.2.1 Internet of Things (IoT)

The Internet of Things (IoT) or otherwise called the Internet of Everything refers to ”a network of networks that allows, via standardized electronic identification systems and unified, and wireless mobile devices, to identify directly and unambiguously digital entities and physical objects and thus to be able to retrieve, store, transfer, and process, without discontinuity between the physical and virtual world’s related data. ”

1.2.2 Social IoT (SIoT)

The Social infrastructure, defined as a subset of the infrastructure sector that includes assets which accommodate social services, is converging with the IoT creating a new approach known as Social IoT. As the matter of fact, Social IoT exploits distributed sensors and other connected devices to improve the efficiency of social solutions in several fields such as transportation, Energy , and utility services.

1.2.3 Object Detection algorithm: YOLOv3

Object detection algorithms involves identifying the presence, location, and type of one or more objects in a given image. The “You Only Look Once (YOLO)” is a family of end-to-end deep-learning models designed to detect objects in a fast manner [57]. Specifically, YOLOv3 is a real-time object detection algorithm that builds upon YOLOv2 with several enhancements. Indeed, it uses a new backbone network, Darknet-53 that utilizes residual connections.

Chapter 2

Related work

In early 2020, the world was hit by an unprecedented pandemic that severely affected most countries and created global health care and economic pressures. To prevent its spread, limiting the exposure to the virus is the main priority of local authorities. Precautionary practices such as hand cleaning, mask-wearing, social distancing, and close contact avoidance are highly recommended and even imposed in many countries. Besides, technological solutions have been tested and implemented to help mitigate the spread of COVID-19 [58] in the world. One of the most promising approaches is to exploit heterogeneous and omnipresent communication systems such as the Internet of Things (IoT) to enable e-monitoring techniques such as spread tracking, contact tracing, and crowded area monitoring. IoT can provide cost-efficient and practical solutions to help practicing social distancing and hence, limit the spread of the infection. The wide range of IoT applications in healthcare and monitoring can produce tools to enhance social distancing without austerity measures such as strict lock-downs and travel restrictions. The ability to control the spread of the virus can be reinforced using IoT services by collecting and analyzing the data to monitor for quarantine and reduce the spread of the virus [59, 60]. The current smartphones and wearable devices' and the existing infrastructure can boost the development of IoT-based solutions in a quick and large-scale manner to combat pandemics. Many examples of IoT-based solutions to contend the pandemic effects have been proposed in literature [61]. For instance, the smart disease surveillance systems had demonstrated an efficient degree of control for the pandemic's spread within the city of Wuhan and other major

cities in China [62]. Despite the significant issues of privacy, South Korea's exemplary accomplishments for containing the COVID-19 until today is due, in part, to the commissioning a coherent information system that tracks visitors and confirmed patients with an alerting system of potential infections [63]. The system provides the community with essential information to assess the spread. Taiwan used various IoT technologies, such as tracking the citizens and travelers through their mobile phone locations. Thus, if citizens are exposed to an area with a high risk of getting infected, they will be altered. Also, if a traveler comes from a high-risk area and violates self-quarantine procedure during the incubation period, in that case, the residents in that area will be notified through a text message to alert them [64].

On the other hand, Artificial Intelligence (AI) has shown its promises in revolutionizing a variety of fields touching different aspects of our life [65]. Within an omnipresent IoT network, AI provides opportunities of better exploiting the widespread data generated by thousands and millions of connected devices. As the data from IoT devices is aggregated in a database, it can be readily used to mitigate the outbreak effects and prevent its spread [65]. For instance, computer vision and deep learning can provide real-time social distancing monitoring by tracking people and their social distancing practice. The information is continuously sent to the authorizing body that can take accordingly in real-time. This helps in minimizing the spread of coronavirus and reduce the spread of the pandemic.

Another example is the AI-based emergency traffic control which opens the way to ambulances and other emergency service providers during rush hours. Red Ninja is a Liverpool based company who developed an algorithm called Life First Emergency Traffic Control (LiFE) that enables paramedics to use real-time data about road traffic to optimize their navigation in congested roads. The automatic human body temperature sensing machines are installed in many countries where camera is integrated with the sensor and employ the temperature camera scanners to send real-time information to the server where AI techniques are developed to recognize faces, detect symptomatic cases, and monitor their mobility and social distancing practices.

A fundamental solution to reduce the transmission of infectious diseases. in general and particularly the COVID-19, is maintaining a safe distance between people and avoid close con-

tact. AI and IoT can perform a vital function in helping with social distancing practices. Thus, the built-in capabilities of connected devices such as GPS, thermometer, and other sensors in the IoT system can be exploited to better practice social distancing. For instance, in constructions or industrial zones, wearables can be employed to maintain a safe distance between workers by generating alerts if social distancing is violated. It also allows to track the spread of the virus when an infected person is present in the working area and hence, avoid the complete shutdown of the institution [63].

Several previous work investigated the problem of the navigation while avoiding some areas to ensure a specific goal. For example, the work in [66] developed a web-based route planning tool for cyclists in Montreal, Canada, in order to reduce cyclists' exposures to traffic-related air pollution. The idea behind this work is to suggest a lower exposure alternatives route to the shortest path for cycling trips. Additionally, the authors of [67] proposes smart urban routing system tailored for pedestrians and cyclists. The system allows to retrieve the best route between a source location and a destination according to user-defined objective function (e.g., selecting the route with the best air quality or with the lowest average temperature).

The emergence of social IoT (SIoT) can be a valuable tool to leverage the traditional IoT systems and enable a better understanding of the ubiquitous IoT network [68]. SIoT models the devices and users in the system with different social relations interconnecting the different network's components (e.g., smartphones, computers, sensors). These relationships can be established between machine-to-machine, human-to-machine, and human-to-human connections [69] and transform the IoT network into a socially connected network of devices that can be effectively analyzed using graph analytics tools such as community detection [70] and machine learning [71]. By assessing the SIoT, providing new applications to battle the virus spread can emerge and contribute to minimizing the pandemic's negative impacts.

Chapter 3

A Social IoT-driven Pedestrian Routing Framework to Mitigate the Spread of the COVID-19

3.1 Introduction

In this chapter, we study the proposed smart navigation approach that intends to give pedestrians safe routing to bypass areas where the risk of COVID-19 transmission is high. As a first part, we present the needed input for the framework, then we introduce the proposed framework general architecture, by explaining its 4 phases. Accordingly, the real-time navigation feature is presented to show that our approach is valid for real-life uses. Finally, two multi-user routing algorithms are proposed to enable the framework to consider other users during the navigation decisions making.

3.2 PROPOSED FRAMEWORK

In this section, we present in details the main phases of the proposed SIoT-driven routing framework for safe navigation.

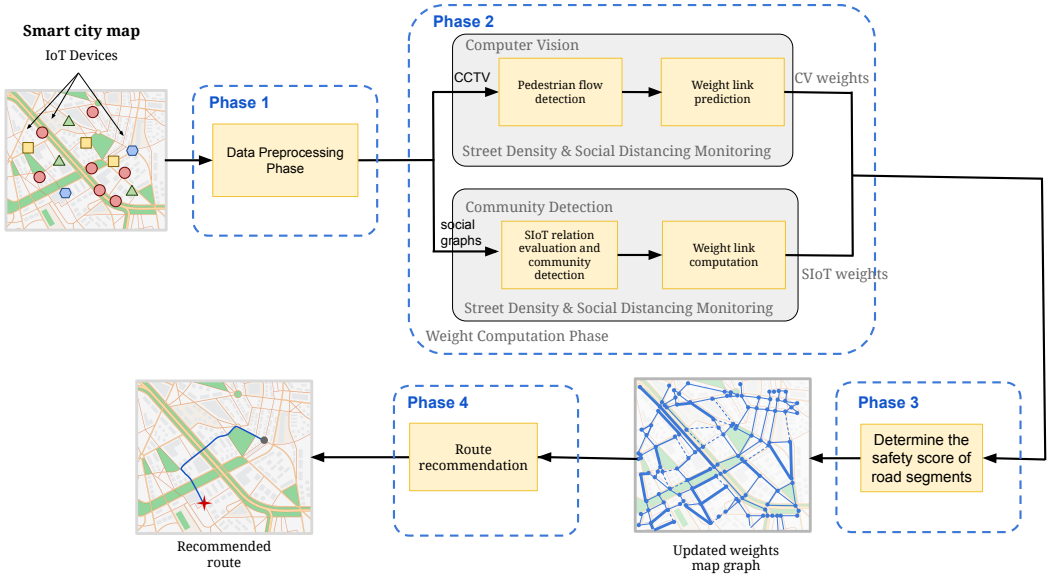


Figure 3.1: An overview of the proposed framework that recommend a safe and fast route to the users during a pandemic using SIoT concept.

3.2.1 Data-set

The proposed framework requires 2 main information as input:

- IoT devices information: The targeted devices are the environment sensors and the highly computational devices such as smartphones, smartwatches, and personal computers. The devices' information includes two types of ownerships, a public-owned, usually the city authority, and a private-owned device owned by the citizens and organizationa in the city. Each IoT device is labeled as static or mobile to indicate mobility. Other information are also used, for example the geographical position, owner, type etc.

- CCTV video cameras: The CCTV cameras in the monitored roads provide high-quality visual information, in addition to other information such as the geographical location of the cameras', the exact time, and the date.

3.3 Framework Architecture

In this section, we discuss in details the architecture of the proposed framework. Indeed, in Fig. 3.1, we present the different components and phases of the proposed navigation framework. Our goal is to assist a user requesting a route from a starting point to a destination to bypass

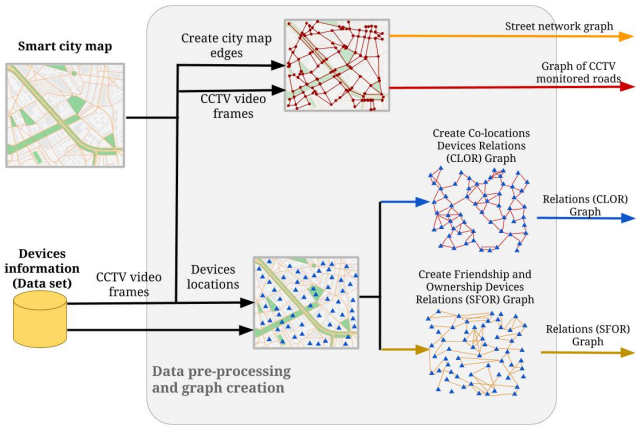


Figure 3.2: Overview of the preprocessing phase

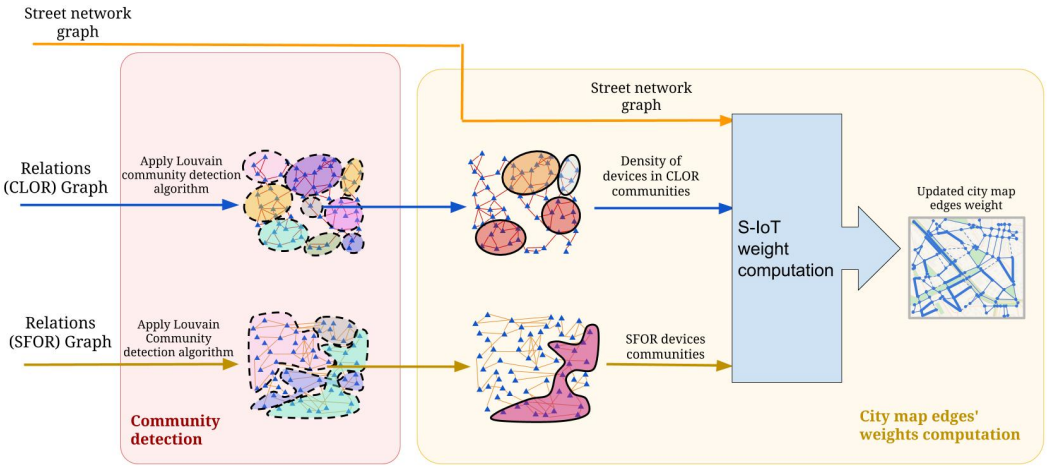


Figure 3.3: Overview of the community detection and weight computation

the areas in which the risk of contamination is relatively high. The risk level of the determined route is based on the data provided by IoT devices connected to the platform, e.g, smartphones, personal computers, in the area of interest. Moreover, the navigation decision is subject to to the SIoT social relations connecting the user of interest and other IoT devices and devices' owners. Finally, the video frames generated by the CCTV cameras of the IoT database are used to predict risky routes.

As the matter of fact, the first phase or the preprocessing phase, as shown in Fig. 3.2, identifies the IoT devices located in the area of interest and establishes social graphs interconnecting these devices using different social IoT relations. Accordingly, this phase is responsible for preparing the street network graph.

In the second phase, described in Fig. 3.3, the Louvain community detection algorithm is ap-

plied to the SIoT graphs to determine different communities of IoT devices. Our approach focuses primarily on two social relations: 1) a distance-based relation that identifies crowded and highly dense areas of IoT devices and 2) a device friendship relation that allows labeling streets where the user may possess a high chance of meeting a close friend. In addition to the community detection part, the proposed framework exploits CCTV video cameras registered in the IoT database to detect the pedestrian flow in a real-time manner and extract safety-related information about the crowds in the street, including the offline pedestrians in the decision-making during the navigation. The main goal of this phase is to compute the computer vision weights and the weights coming from the CLOR and SFOR communities.
According to the previously calculated weights, the third phase intends to assign scores to each street or segment of a street in the area of interest.

Finally, in the last phase, the city map is transformed into a weighted undirected graph. We apply a routing algorithm, specifically, the Dijkstra algorithm [15], To to determine a route characterized by a certain level of safety. A weighted objective function balancing between the shortest and safest routes is developed and implemented. The framework will then deliver the trajectories to the user, e.g., via a mobile application for the best route to follow to reach a destination. The proposed routing approach takes into account the mobility of IoT devices and may update the recommended route regularly by repeating the process, as mentioned earlier.

The framework also considers the traveled distance to ensure a balance between the safety and the rapidity of the navigation. Three inputs are required to determine the safest/fastest route::

- The pedestrian street network: this input consists of a multi-dimensional graph representing the street network of the area of interest.
- User-related information data-set: It contains real-time information of the IoT devices for all users in a smart city. The information includes the devices' current locations, device owners, and their types.
- CCTV camera's videos: the framework has access to CCTV cameras connected to the

platform in some roads that provide real-time videos.

As shown in Fig.3.1, the framework starts with a preprocessing phase in which four graphs are generated: i) a graph representing the city street network map, ii) a a graph updated in real-time that presents the data coming from the CCTV cameras and aims to include information that reflects the risk caused by the offline pedestrians, iii) a graph illustrating the social relations reflecting the geographical relation of the connected IoT devices called the Co-LOcation based Relation (CLOR), and a graph identifying the friendship levels among the IoT devices, termed as the Social Friendship and Ownership-based Relation (SFOR).

The second phase intends to analyze the IoT devices' social relations and accordingly to determine communities for each social relation as follows:

- CLOR communities: The devices that belongs to the same CLOR community are co-located. Indeed, the size of a CLOR community indicates the density level of the corresponding geographical region.

- SFOR communities: Devices belonging to the same SFOR community indicate that their owners share mutual and high friendship levels. Hence, there is a risk of close contact if these devices get close to each others.

This phase helps reduce the complexity of the problem by avoiding the processing of a vast and complex IoT network, and process only the communities of the socially connected IoT devices.

In the third phase, we intend to involve the offline pedestrians that were not included during the last two phases. To this end, the framework exploits the CCTV cameras in some roads to extract risk-related information concerning the crowd in the monitored roads. In fact, in many cities and industrial zones CCTV are present in most of the roads to provide reliable and high-quality visual information in a real-time manner.

In this phase, we utilize computer vision methods to extract the risk-relation information from the visually detected pedestrians. Indeed, for each road we estimate the number of pedestrians at every instance, the entrance frequency of pedestrians to the investigated road, and any

social-distancing rule disobedience among the crowd. Consequently, we can estimate the congestion level in order to assign a weight to the monitored road segment. However, some roads are monitored using CCTV cameras. Therefore we estimate their weight using an intelligent graph theory algorithm, namely Graph Convolutional Weight Completion (GCWC) algorithm. This phase will output a graph with different risk levels of virus exposure for each road. In the next phase, we assign to every edge of the map graph a weight measuring the traveled distance and the risk level of the corresponding street or segment of the street from the previous phases. Finally, the last phase of the framework applies a multi-user graph routing algorithm that search a route with a minimum sum of weights and hence, it determines the most suitable path to recommend to the user of interest.

3.3.1 Street Network Graph Structure

The street network is transformed into a graph as a first step. Accordingly, long roads/streets with lengths higher than a threshold length L_{th} are divided into multiple segments where each segment of the road is labeled solely and can be later assigned a different weight. The number of segments for a road of length L_{road} can be expressed as $\lceil \frac{L_{road}}{L_{th}} \rceil$. The crated graph is defined as $G = (V, E)$, where vertices V represent road intersections and edges E represent streets. We denote by A a weighted adjacency matrix associated to graph G where $A_{u,v} = \omega_{u,v}$, and $\omega_{u,v}$ is the weight of edge between two intersections u, v. If the two intersections u, v are not directly linked then $A_{u,v} = 0$.

3.3.2 SIoT Relation and Community Detection

During this phase, we exploit the SIoT paradigm by clustering the IoT devices, in the investigated area, into two types of communities.

AS the matter of fact, we cluster the devices' into communities based on two main relations, the Co-location/co-work based relation (CLOR) and Social friendship and ownership relation (SFOR), that are used in this study to determine the social interconnections among the IoT



Figure 3.4: A selected area of Santander, Spain.



Figure 3.5: CLOR communities (The colors of each CLOR community indicates the density level of the community).

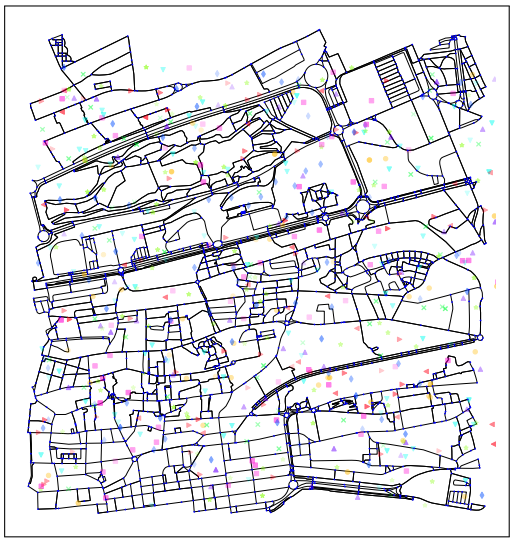


Figure 3.6: SFOR communities (IoT devices belonging to the same community are labeled by the same marker).

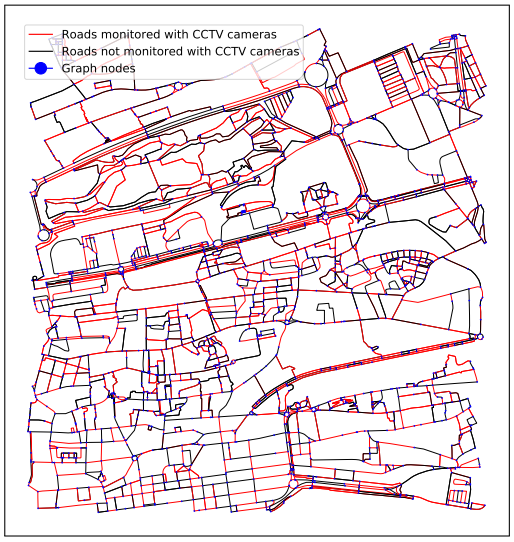


Figure 3.7: Graph network of roads equipped with CCTV cameras.

Figure 3.8: Communities were detected based on CLOR and SFOR relations using the Louvain Method.

devices:

• *Co-location/co-work based relation (CLOR):* The geographical locations of the IoT devices can be used to define a specific relation reflecting the fact that two devices are co-located in a given area at a particular instant of time. By setting a defined threshold for the distance between the devices, we can specify whether devices belong to a distinct cluster. Hence, they establish relations between them based on the distance between the devices. This relation helps

in identifying crowded areas where there is a risk of limited social distancing practice among these devices. Therefore, it is vital to avoid passing the overcrowded areas that are labeled as high risks areas.

• *Social friendship and ownership relation (SFOR):* In this relation, we identify the devices that might be owned by the pedestrian of interest of his/her social friends. We assume that it is more likely that the pedestrian will meet or be in close contact with people using devices belonging to the same SFOR community of his/her devices, and hence, there is a risk of contamination with people that he/she knows. To create SFOR relations, we first consider that the same person's two devices are strongly connected. We establish their SFOR relations using social media networks or other friendship indicators regarding devices owned by different owners. For example, if two owners are friends in the social network, then the SFOR relation between their devices can be modeled by an edge with a weight reflecting the number of people to reach each owner. Thus, the SFOR relation can be used to avoid users' friends or friends of a friend.

The CLOR and SFOR topologies are undirected and weighted networks. The nodes in these graphs are heterogeneous IoT devices. The edges between these devices representing the SIoT relations stated previously. These graphs do not include self-loop edges because the relations definitions do not require such a feature.

By creating social IoT relations, we aim to reduce the problem's complexity by employing a community detection algorithm, namely Louvain, that can be used to calculate the effects of the different IoT communities on the routing recommendation method. A community detection algorithm, in general, converts the complex graph into distinguished clusters of nodes that share strong relations. To this end, we apply the Louvain method [72] which is a greedy optimization method that attempts to maximize the "modularity" [72] of a partition of the network. The main advantage of using the Louvain is the running time of $O(n \log n)$, which is considerably faster comparing to a similar methods [68, 73, 74]. The community detection outcomes, in our framework, will be used in *phase 3*.

The Louvain method applied to the CLOR graph will extract close devices or the co-located device into a set of communities. In SFOR communities, the devices are not necessarily geo-

graphically co-located. On the contrary to CLOR, they are sparsely distributed in a geographical area. However, the owners of these devices may know each other and can meet each other. Therefore, it is recommended that a given pedestrian does not pass by devices belonging to the same SFOR community that his/her device is assigned to for safety reasons and better social distancing practice. The performance of community detection is investigated using three metrics:

- Modularity score: it is a metric designed to measure the strength of division of a network into modules.
- Coverage score: the coverage of a clustering is given as the fraction of the weight of all intra-cluster edges with respect to the total weight of all edges in the whole graph [75].
- Performance score: the performance of a partition is the ratio of the number of intra-community edges added to inter-community non-edges with the total number of potential edges.

3.3.3 Computer Vision Phase

Since the SIoT phase considers the connected pedestrians only and neglects all offline pedestrians, another phase is required to include the offline pedestrians in the optimal path selection process.

Additionally, most city roads contain CCTV cameras and they are considered as a reliable source of information. Hence, we exploit this source of data to extract information related to the flow of pedestrians in each road. However, the data-set, discussed previously, contains the roads that are monitored with CCTV cameras and it also includes the roads that are not monitored. For example, Fig.3.7 presents the roads of a street network map that are monitored with CCTV cameras (in Red). We employ computer vision algorithms to extract the information from CCTV video frames. Indeed, the collected frames will be fed into a computer vision algorithm to predict several features, such as the disobedience of the social distancing rule, the

average number of people in that road at an exact timestamp, and the frequency of people entering and leaving the road. The employed computer vision framework is presented in Fig. 3.9. As we notice, the collected frames using CCTV cameras are fed into YOu Look Once version 3 (YOLOv3) [57] which is Convolutional Neural Network (CNN) based object detector. the last layers of the YOLOv3 were fine-tuned to detect only pedestrians and neglect other environmental objects. In fact, Yolov3 is considered as one of the most efficient algorithms for object detection due to its rapidity, accuracy, and ability to detect small objects in very crowded places.

The detection of objects is not enough to ensure accurate results, therefore we need to track every object to avoid counting a pedestrian more than once. Precisely, the tracking process is the association of two detected objects in two adjacent frames. Indeed, after detection the pedestrians, the estimated bounding box of each one is fed to the Kalman filter to update the target state(i.e. Kalman filter is an optimal recursive Bayesian stochastic method that propagates conditional probability density of the system state based on the actual state). The next step is the matching process which consists in assigning every new detection to existing targets [76], given an example: two pedestrians are detected in the first frame and they has been assigned, respectively, ID1 and ID2. When the video evolves, the object detector, YOLOv3, detects two pedestrians and during the matching step the framework recognizes the ID of each pedestrian in relation to the previous frame.

To process the matching, the framework employs the Hungarian algorithm. The latter algorithm exploits two metrics to ensure the matching:

- The Mahalanobis distance: calculated between predicted Kalman states and newly arrived measurements and aims to incorporate motion information in the matching process.
- Deep appearance descriptors: those are the semantic descriptors of an image, extracted by a specific convolutional neural network that intent to extract the low-level image features.

Finally, the bounding box associated with target IDs are used to estimate the number of people going in each road, the frequency of accessing a specific road, and detecting any disobedience

of the social distancing rule.

3.3.4 Weight Link Computation

In this phase, we propose to compute road weights given their surrounding communities' statuses. Hence, the route selection algorithm in this framework will minimize the sum of weights along the path. To do so, we assign to each road segment e a weight that is balanced by a coefficient $\alpha\,(\in [0,1])$ representing the level of safety set by the pedestrian of interest. Setting a value of $\alpha \to 0$, the pedestrian intends to determine the shortest path to reach the destination with low consideration of risks. However, if $\alpha \to 1$, the pedestrian is looking to follow the safest trajectory independently of the expected traveled distance. Values of $\alpha \in]0,1[$ achieves a trade-off between both routing strategies. The edge's weight of the road network, denoted by ω_e can be expressed as follows:

$$\omega_e = (1-\alpha)\,\omega_e^{dist} + \alpha\,\omega_e^{sft}, \tag{3.1}$$

Where ω_e^{dist} is the weight reflecting the expected traveled distance when crossing edge e, while $\omega_e^{sft} = \omega^{CV} + \omega_e^{SIoT}$ is the weight reflecting the safety level of the edge which measured using the weights coming from SIoT community detection part, ω_e^{SIoT}, and the computer vision part ω_e^{CV}. The weight ω_e^{dist} is a normalized value of its length.

3.3.4.1 SIoT Weights Computation

This part explain the computation process of the weights coming from the CLOR and SFOR communities. In fact, the value of Social IoT weights ω_e^{SIoT} is measured by combining the impact of the surrounding CLOR communities and devices belonging to the same SFOR community of the pedestrian's device as follows:

$$\omega_e^{SIoT} = \omega_e^{CLOR} + \omega_e^{SFOR}. \tag{3.2}$$

The CLOR weights ω_e^{CLOR} are calculated using the following expression:

$$\omega_e^{CLOR} = \sum_{c \in \mathcal{C}_e^{CLOR}} \frac{\gamma_c}{|\mathcal{C}_e^{CLOR}|}, \tag{3.3}$$

where $\mathcal{C}_e^{CLOR}$ is the set of CLOR communities that intersect with the edge e. The cardinality of this set is denoted by $|\mathcal{C}_e^{CLOR}|$. The CLOR communities are modeled as polygons that circumscribe devices belonging to each community with an outer offset ρ. We denote these polygons by $P_c, \forall c \in \mathcal{C}^{CLOR}$ where $\mathcal{C}^{CLOR}$ denotes the set of all CLOR communities obtained using the Louvain method. Thus, the set $\mathcal{C}_e^{CLOR}$ can be defined as: $\mathcal{C}_e^{CLOR} = \{c \in \mathcal{C}^{CLOR} \mid P_c \cap \{e\} \neq \emptyset\}$ where $\emptyset$ is the empty set. The offset parameter ρ is added to all polygons associated with the CLOR communities to ensure a safe social distancing for the navigating user. Finally, γ_c denotes the density of community c and is calculated as follows:

$$\gamma_c = \frac{N_c}{A_c}, \tag{3.4}$$

where N_c is the number of devices in community c and A_c is the surface of the area of P_c.

Similarly the SFOR weights can be computed as follows:

$$\omega_e^{SFOR} = \sum_{u \in \mathcal{C}_{u^*,e}} \frac{\Omega_{u,u^*}^{SFOR}}{|\mathcal{C}_{u^*,e}|}, \tag{3.5}$$

the $\mathcal{C}_{u^*,e}$ is a set of devices that are in SFOR relation with the devices u^* of the user pedestrian and their $d(u,e) \leq d_{th}$. It corresponds to the SFOR community to which the user device u^* belongs. Besides, the user device distance is denoted as d_{th} that measures the edge e distance. The coefficient Ω_{u,u^*}^{SFOR} is obtained from the SFOR graph, and it measures the SFOR relation between device u and u^*. The parameter d_{th} denotes the allowed distance in which the user owns the IoT devices will try to avoid any device within the same SFOR community.

In general, the SIoT weight of an edge e significantly increases if it is surrounded by high-density CLOR communities and/or many devices belonging to the same SFOR community of

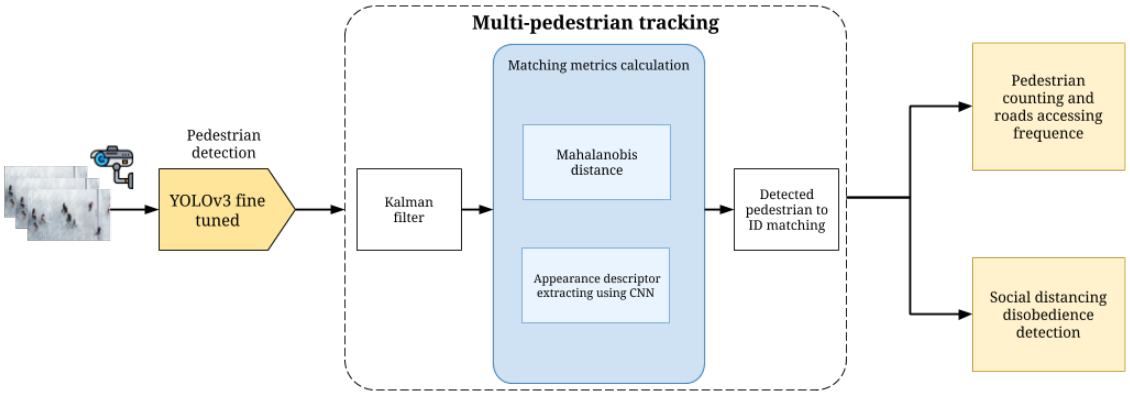


Figure 3.9: Overview of the computer vision tracking system

the user of interest. Therefore, in the navigation phase, we aim to select the edges with lower risk, i.e., the trajectory that minimizes the sum of ω_e^{SIoT} for a user looking for a safe walk.

3.3.4.2 Computer Vision Weights Computation

For each road, we assign a weight based on the previously discussed features. Those weights are calculated as follows:

$$\omega_e^{CV} = \omega_e^{PF} + \omega_e^{SD} \tag{3.6}$$

where ω_e^{PF} is the number of pedestrians accessing the investigated road per second (i.e. the frequency of pedestrian accessing the road), and ω_e^{SD} is the ratio of people who disrespect the social distancing rule in that road.

The latter weight will be assigned to each CCTV monitored edge (i.e., the edges colored in red in Fig. 3.7). Those weights are calculated at each timestamp as the average of the weights calculated for each frame during the timestamp.

So far, CV weights are available only in the roads or intersections that are monitored using CCTV cameras registered in the IoT database. Other non-monitored roads lack this data and are not assigned any CV weights. Consequently, propose an AI-based solution to impute the missing features that were extracted in the computer vision part and hence, predict weights for non-monitored roads based on the features of the monitored roads surrounding them.

To solve the problem of the missing weights for some edges, we propose to apply an arti-

ficial intelligence approach applied for graph theory that exploits the neural network approach. Specifically, the Graph Convolutional Neural Network (GCNN) approach.

This graph based artificial intelligence exploits the topology of the road network graph and the correlations of weights among adjacent edges to estimate stochastic weights for all edges.

This process adopts the intuition of the auto-encoder. Indeed, the street roads graph is provided as input to be fed into a GCNN which is followed by a pooling layer that will embed the graph into a lower-dimensional space, which can be regarded as the encoding process. Finally, we decode the output of the encoder using a fully connected layer and thus obtain an estimated fully weighted graph. During the embedding phase, a new totally weighted embedded graph is created. The edges of the embedded graph are weighted and the problem of the missing weights is not raised. The decoding phase will restore the initial graph but with different weights for all edges with no exception.

3.3.5 Real-Time Navigation

Our framework is capable of dynamically recommending new paths to the user based on his/her current location while considering the mobility of other IoT devices. Hence, it needs to update the CLOR and SFOR communities after a specific period and the computer vision features are updated dynamically to provide real-time factors. Consequently, the selected path is updated from a time slot to another. In other words, the phases 2 to 4 will be repeated for each time slot until the user reaches his/her destination.

3.3.6 Multi-User Navigation

A multi-user functioning is required to obtain an efficient navigation framework. Indeed, the concept of the multi-agent is omnipresent in the navigation realm such as in organizing the cities traffic flow and vehicles' shortest path selection. Our framework supports the multi-user functioning as well as the case of single-user. However, for a real-life case, navigation frameworks must be able to involve other framework users in their navigation decisions. This

Algorithm 1 Step-by-step Iterative Navigation Algorithm (SINA)

Inputs: $\{St_j, De_j\}, j \in \{1, \cdots, N\}$ where N total number of the users.
Initialization Phase:
Set $SP_j = St_j$ and $DP_j = De_j$.
Find initial path for each user j using Dijkstra(SP_j,DP_j).
for $T = 1, \ldots, \bar{T}$ **do**
    Determine the list $\mathcal{L}$ of intersections at which the condition (3.7) is not verified.
    for l in $\mathcal{L}$ **do**
        Determine the list $\mathcal{U}$ of users heading to the intersection l.
        Calculate the arrival time AT_u of each user $u \in \mathcal{U}$ to the intersection l.
        Find $\hat{u} = argmin_{u \in \mathcal{U}}(AT_u)$.
        Freeze the navigation of users u in $\mathcal{U} \setminus \{\hat{u}\}$ for a period of Δ_t and update their starting point $SP_j = l$.
        Update path for user u using Dijkstra(SP_u,DP_u).
    end for
end for

concept will make our model useful for many cases, for example, it can be used in industrial areas or campuses to assist workers/students in navigating and hence enhance the protection of the pedestrians and decrease the probability of getting infected in the investigated areas. Next, we present two Dijkstra-based heuristic algorithms that adopt different strategies for choosing the optimal path, the algorithms are identified as PINA and SINA. The former algorithm determines a full trajectory for a prioritized user then, focuses on the next user, while the second algorithm, SINA, finds the simultaneous steps taken by each pedestrian at each iteration.

3.3.6.1 Step-by-step Iterative Navigation Algorithm (SINA)

The framework is characterized by a centralized architecture, every user is identified independently from other users. Indeed, each user is defined with a unique ID and has his/her own attributes such as the navigation speed, estimated from his/her navigation history, and the exact instant of his/her last connection. In the multi-user navigation, the framework synchronizes the trajectories of users, in a real-time manner, to avoid the presence of many users at the same place simultaneously and hence, push towards a better social distancing practice within the area of interest. To do so, at each timestamp $T \in \bar{T}$, the framework calculate the arrival time of each user to his/her next intersection, and hence detect any possible conflict between the users by

Algorithm 2 Priority Based Iterative Navigation Algorithm (PINA)

Inputs: $\{St_j, De_j\}, j \in \{1, \cdots, N\}$ where N total number of the users.
Initialization Phase:
Initial path for each user j PA_j = Dijkstra(St_j,De_j).
Set $SP_j = St_j$ and $DP_j = De_j$.
for u in $\mathcal{U}$ **do**
    Calculate the arrival time AT_u for higher priority users $\in \{1, \ldots, u-1\}$.
    Determine the list of segments $\mathcal{L}$ where conflicts with higher priority users are detected.
    Block those segments during the arrival instants plus/minus gap time Δ_t to prevent collision for the rest of users.
    Update path for user u using Dijkstra(SP_u,DP_u).
    Navigate the user u using the updated path.
end for

verifying the following constraint:

$$|AT_u - AT_v| < \Delta_t \tag{3.7}$$

where AT_i is the arrival time of user i to intersection u and Δ_t is the necessary period of time to avoid conflicts between users.

This constraint intends to determine the user that will attend the intersection first. This user will keep navigating while others wait a period of time until the constraint 3.7 is no more valid.

3.3.6.2 Priority Based Iterative Navigation Algorithm (PINA)

The PINA algorithm avoids any conflict by redirecting the user to the second selected path. To this end, a priority between users is mandatory for this algorithm to avoid the problem of conflicting with other users while avoiding another one. Actually, the users will be ordered according to the first come first serve concept, after determining the priority levels of all users, the safest trajectory is determined for the user with the highest priority in the investigated area. To do so, we exploit the Dijkstra's algorithm [15] applied to the street network graph defined in 3.3.1. Then, the trajectory of the lower priority user will be calculated based on the trajectories of the users with higher priority to avoid conflicts. Hence, arriving at an intersection of the map must respect the (3.7) so delays are counted in these intersections. In case of conflict, all roads that lead to that investigated node are closed and a second trajectory is calculated. The

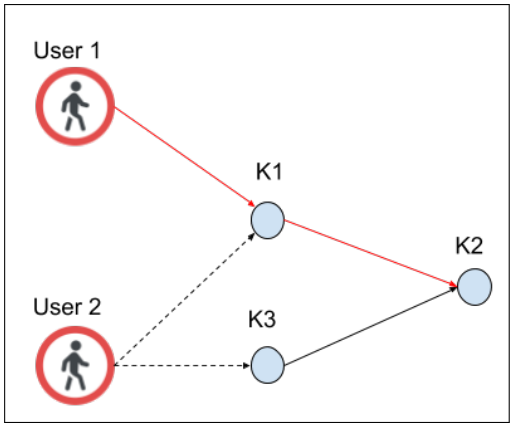


Figure 3.10: PINA

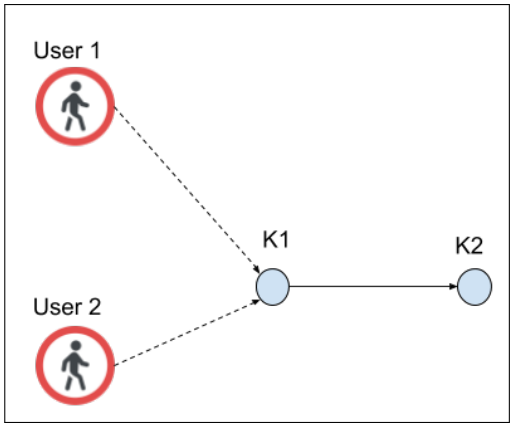


Figure 3.11: SINA

Figure 3.12: Comparison between the two proposed heuristic approaches a) PINA and b) SINA. In PINA, user 2 needs to wait before entering k1 or go through k3 instead since priority is given to user 1. In b), there is no priority among the users. User 1 is arriving later, hence he needs to yield to user 2 to enter k1 safely.

intersections are not permanently closed but they are inaccessible just for the instants where the previous user entered them plus/minus the time gap Δ_t. The algorithm describing this process is given in Algorithm 2. The selected path for each user needs to consider the potential risk of collision with all previous users. In other words, some roads should be blocked for the current user during a certain period, therefore the user will select a second optimal path that avoid the collision with the other users with higher priority. Then, the user moves one phase ahead and updates its current location till reaching its destination.

The idea behind the SINA is to determine the next phase to be chosen by each member of the users simultaneously. In other words, at each iteration, each user will take an action and move towards its destination. To reduce the probability of getting infected, the framework will assist the user to attend his destination, in the safest way, through choosing between two possible actions, wait or navigate. The SINA aims to satisfy all the users and provide them with the safest trajectory without any specific pre-ordering. Indeed, When simultaneously reaching the same intersection, the first-in-first-out (FIFO) principle is applied between the users. Consequently, the user arriving first to the intersection enters the road segment-first, while the second user waits if necessary. The algorithm starts with an initialization phase where it determines the fastest paths of each user using the Dijkstra's algorithm assuming solo trips.

The iterative phase starts by taking an action for each user based on the path determined in the initialization phase. After each phase (action), the arrival instants of each user at each intersection are recorded. In case of collision risk, the FIFO principle is applied at that intersection. The algorithm converges when all the users reach their destinations. The pseudo-code of the SINA is given in Algorithm 1.

In Fig. 3.12, we highlight the differences between the PINA and SINA algorithms. In PINA, the trajectory of user 1 is already determined and the user 2 will need to find its trajectory accordingly. Hence, two options are possible at intersection $k1$ either user 2 will wait so it enters $k1$ after a time gap Δ_t or it will choose a second path. In SINA, the user 2 arriving first to the intersection $k1$ will have the priority to enter the segment $(k1, k2)$. User 1 will then be forced to wait.

3.4 Conclusion

In this chapter, we studied the proposed smart navigation approach that intends to give pedestrians safe routing in order to bypass areas where the risk of COVID-19 transmission is high. As a first part, we present the framework's input, then we introduced the proposed framework's general architecture, by explaining each phase. To study the trade-off between the risk and the length of the selected roads we performed a Monte -Carlo simulation in which we determined the best value of α that will balance between the safety and distance. Accordingly, we showed that our framework works in a real-time manner and hence it is applicable for use in real-life. Finally, two multi-user routing algorithms are proposed and compared to ensure an efficient involvement of other framework users during the navigation decision process.

Chapter 4

Results & Discussion

4.1 Introduction

This chapter describes, as a first part, the exploited Data-set and the investigated area map. additionally, we present the obtained graph after the preprocessing phase, the CLOR and SFOR social community graphs, the results of the pedestrians tracking during the computer phase, the multi-user and the real-time functionalities of the framework, and finally we show the effect of the social distancing on the navigation decision through a Monte-Carlo simulation. The main goal of this chapter is to prove the efficiency of all parts and functionalities in the proposed framework.

4.1.1 Data-set and Map Description

In our simulations, we select a 6×6 km^2 area in Santander, Spain. We extract the map using OpenStreetMap project[1] and convert it to a road graph using the OSMnx method [77]. Moreover, we project the devices in the selected area from a real-world IoT data-set provided in [78]. The data-set includes 16216 devices covering the whole city. The devices vary from simple sensors such as street lights, environment sensors, and highly computational devices such as smartphones and personal computers. The devices are owned by private and public entities. The local authorities usually own public devices. For the private-owned devices,

[1] https://www.openstreetmap.org

Table 4.1: Validation results of the CLOR communities

	Santander city Map	Santander's port Map
Modularity score	0.934	0.892
Coverage score	0.977	0.926
Performance score	0.961	0.918

Table 4.2: Validation results of the SFOR communities

	Santander city Map	Santander's port Map
Modularity score	0.901	0.814
Coverage score	0.953	0.902
Performance score	0.923	0.884

there are static and mobile devices. In our simulations, we select mobile devices owned by private entities that are most likely owned by human beings such as smartphones, smartwatches, tablets, personal computers, etc. The remaining devices from the previous selection process result in 1312 personal IoT devices.

The selected devices will have SFOR and CLOR SIoT relations. For the CLOR relations, we create a mesh network and drop the edges connecting two devices separated by a distance higher than 1 km. The community detection algorithm applied to the CLOR will return a set of relatively high-density communities to determine high-risk infection areas with the limited practice of social distancing. For the SFOR relation, we employ the social network of the owners of the IoT devices. Since we lack access to the owners' social network, we use Watts–Strogatz generator [79] that portrays a social network between the owners. The relations of devices with the same owner are assigned an edge of 1, while the direct friends-owned devices will have an edge of 0.5. Other devices are given weights computed while considering the minimum number of hops needed for one of the vertices (owners) to reach the other vertices. We restricted the relations to three friends of friends since they are socially far away from each other.

4.1.2 Clustering and Community Detection

Figs. 3.5 and 3.6 shows the communities obtained from the SIoT graphs by applying the Louvain method. We obtain 56 CLOR communities represented by colored polygons and hav-

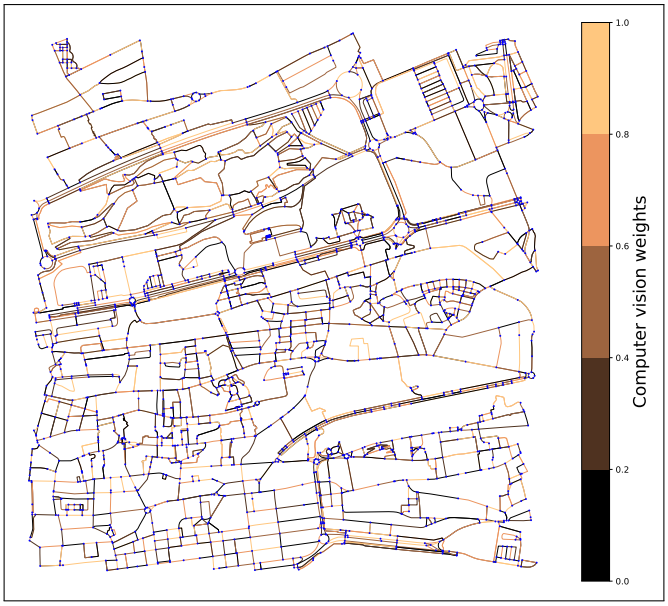


Figure 4.1: Street network after assigning to each road a computer vision weight

ing different density levels, as illustrated in Fig. 3.5. The CLOR communities are classified, based on their densities, into five classes. There is one very high-density community located almost at the center of the map and other blue CLOR communities with high-density that the user needs to avoid for safety. The medium and low-risk areas might be avoided, but they can be recommended. For the SFOR, it results in 10 communities with diverse types of devices located all over the map. Each community is denoted with different shapes and colors in Fig. 3.6. The user of interest will belong to one of these communities and needs to avoid close contact with them. To investigate the efficiency of the Louvain algorithm we calculated the modularity, coverage, and the performance score, defined in section 3.3.2. Indeed, the calculated validation scores of communities in the two investigated areas are presented in the Tables 4.1 and 4.2. The former table confirms that the CLOR community detection phase is providing efficient results and harmonic communities for both maps, and the latter table strength the results provided by the SFOR communities.

4.1.3 Computer Vision & Weight Link Prediction

Fig. 3.7 presents the networks of roads that are monitored with CCTV cameras, colored in red, and contains information about the behavior of pedestrians on those roads. However, the

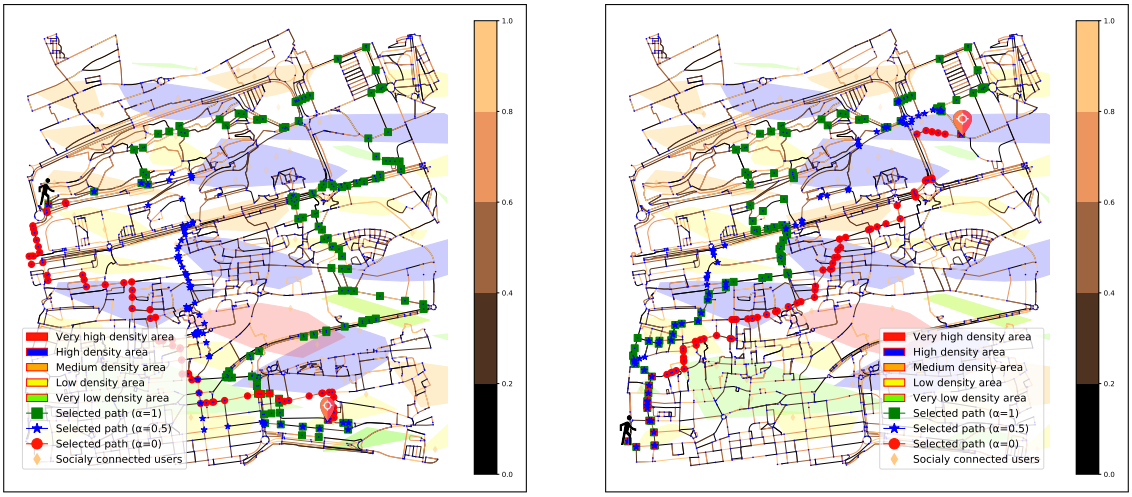


Figure 4.2: Two examples showing the different paths recommended to the user for different values of α.

roads unequipped with CCTV cameras are colored in black. Hence, we proceed by calculation the computer vision weights, as detailed in 3.3.4.2, and we update the graph edge weights by following the mathematical formulation as explained in 3.3.4 for the CCTV monitored roads. The navigation algorithm requires all edges to be weighed, therefore, we estimate the weight of the edges that represents the roads with no cameras using the weight link prediction on the graph as described in the 3.3.4.2. The obtained graph network after the imputation process is presented in the Fig. 4.1 with a color-bar that classify the roads into 5 classes based on the computer vision weights.

At this level, the graph network is ready for the navigation algorithm described previously. The proposed algorithm exploits the Dijkstra algorithm locally but it doesn't converge to the same path as Dijkstra. In fact, our algorithm aims to avoid any simultaneous presence of users in the same intersection or any situation where the inter-user distance is less than 2 meters from other users of the framework (i.e. this situation is possible in small roundabouts).

4.1.4 Navigation Results

Fig. 4.2 illustrates two examples of the recommended routes for the user given different starting points and destinations for three values of α after applying the Dijkstra's algorithm using the computed weights. If $\alpha = 0$, then the framework will recommend the shortest path

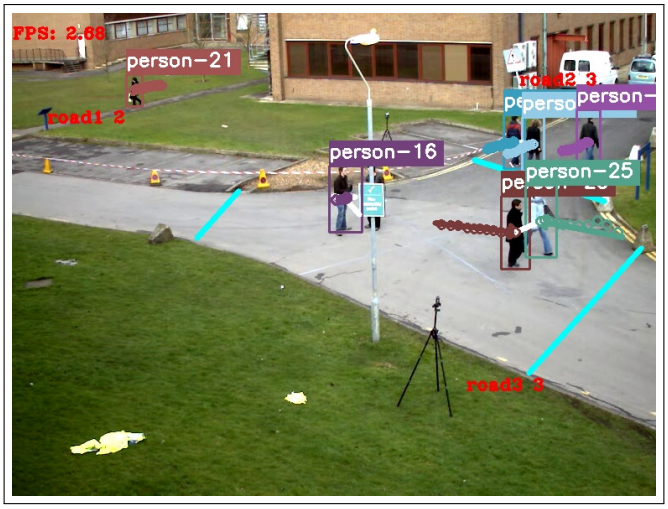


Figure 4.3: Example of the Pedestrians tracking in a 3 roads intersection

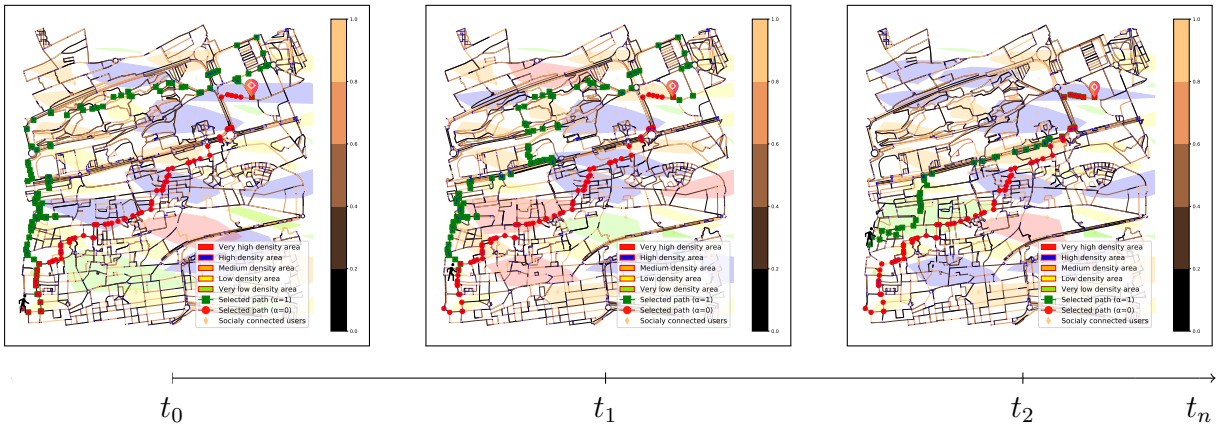


Figure 4.4: The routing updates on three time phases are based on the changes in device locations and communities in the graph.

(the red route), otherwise if $\alpha = 1$, the safest path with minimum exposure to the virus is recommended (the green). However, for $\alpha = 0.5$, a trade-off between both metrics is provided (the blue route). In Fig. 4.2, it is clear that along the green route, the user is avoiding most of the high dense areas by surrounding them. It just crosses some of the low-density areas. It also avoids getting closer to other SFOR-related devices unless it is forced to do it. This leads to a long route of 2.77 km. With the red route, the user is unaware of the risk of contamination and crosses all the high-density areas. The corresponding traveled distance is equal to 2.1 km. Finally, for the blue route, the algorithm avoids the red zone and tolerates passing by some blue areas. This results in a traveled distance of 2.4 km.

Our framework can be adapted to a dynamic scenario by examining real-time mobility. In

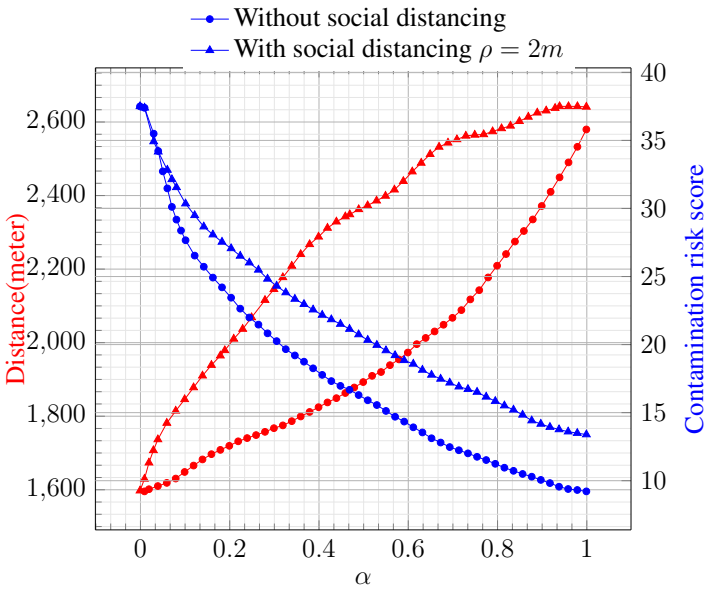


Figure 4.5: A trade-off between the safety factor and destination amount for the proposed framework.

Fig. 4.4, we plot the routes for three consecutive time slots. Each time slot, the IoT devices change their locations and, consequently, influence the CLOR relations and their communities and the position of devices in SFOR. In the dynamic scenario, the shortest path will remain intact. The algorithm is only aware of the distance to be crossed, while in the proposed framework that considers the safety weights, the trajectory is regularly updated given the location of the devices at each time slot. Accordingly, the user starting at the left bottom corner of the map will notice that its trajectory is partially updated at a time slot (t_1) since several SFOR devices left the area, and the user can cross in the middle of the map to reach its destination. In the next time slot (t_2), the navigation algorithm is executed again. Notice that the user is forced to go around it to reach its destination. As long as he/she is moving, the user is getting closer to the destination, especially if a correct value of α is chosen. Choosing α close to 1 may lead to confusing results. Therefore, to balance between safety and travel time, an optimized choice of α should be made.

4.1.5 Monte Carlo Simulation

In this section, we proceed with the Monte Carlo simulation to understand the impact of the social distancing factor ρ on the selected path. In fact, the equation 3.1 define the weight of

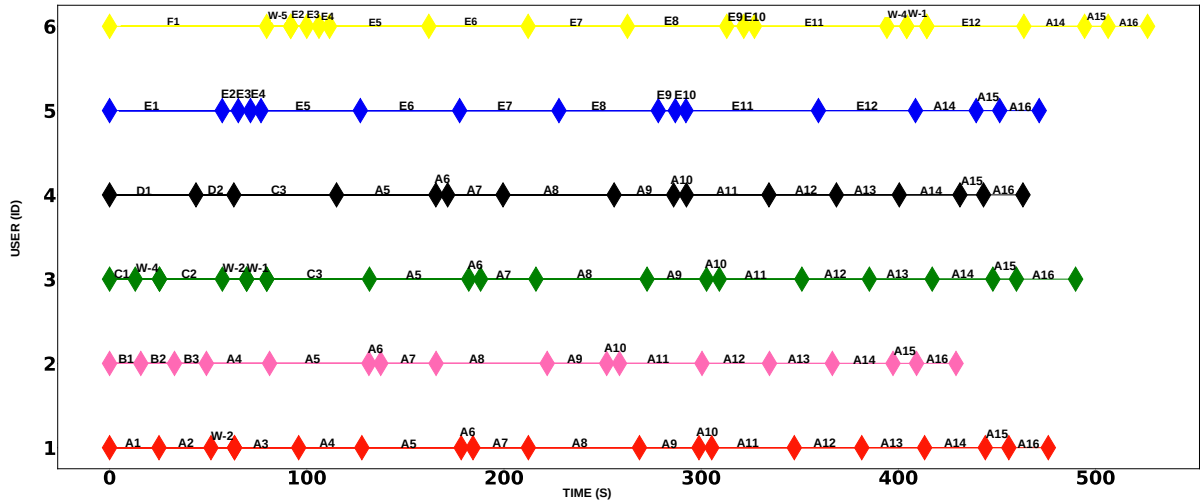


Figure 4.6: Special case: Time schedule of 6 users. The length of each segment (linking two diamond) indicates the time needed to navigate. The labels on each line (e.g., E1) indicate the path segments crossed by a user. W-i indicates the waiting action to yield to the user i.

every edge as a trade-off between the navigated distance and the risk score. The α parameter control this trade-off. We employ the Monte Carlo Simulation technique to investigate the trade-off between the two previously mentioned weights for the case with and without the social distancing factor ρ.

Actually, in Fig. 4.5, we plot the final travel distance as the average of the traveled distance for 100 trajectory and the risk score measuring the average of the cumulative risk score along the same 100 trajectory versus different values of α for two choices of ρ (the social distancing outer offset). A higher value of ρ indicates an increasing preventive navigation strategy aiming to find trajectories a little bit far away from CLOR communities. The figure shows that by increasing α, the travel distance increases, moving from 1.6 km to around 2.6 km while the cumulative safety score is almost linearly decreasing. A compromise between safety and speed can be achieved for α around 0.44. By increasing the outer offset ρ, a more strict social distancing is applied, and hence, the traveled distance increases even for the same value of α. For instance, for $\alpha = 0.44$, the distance changes from 1.9 km to 2.3 km, with a slight improvement of the safety score.



Figure 4.7: Navigation path of 6 users using the PINA multi-user navigation algorithm

4.1.6 Multi-User Navigation Results

In this part, we investigate and compare the performance of the two proposed multi-user navigation algorithms, PINA and SINA. To this end, the smallest area, with 1×1 km^2 area, is investigated. The new map is a port in the Santander city in which 6 workers navigate from different start points to the same destination. The trajectories of all workers are shown in the Figs. 4.7 and 4.8, respectively, for the PINA and SINA algorithm. The navigation of the 6 users presents 4 conflict location, those locations are represented with warning signs. We note that the high priority worker for the PINA is the one with the smallest ID.

The PINA results in Fig. 4.7 shows that the framework change the path of the user at each conflict and keep minimizing the risk by avoiding high risky area and socially connected user. However, the SFOR results presented in Fig. 4.8 shows the trajectories of the user and the conflict locations. In order to investigate the navigation of SFOR we presented the cumulative navigation time of each user in Fig. 4.6. Indeed, the figures show the nodes included in the path of every user in the temporal axis, in other words, the rhombus in the picture limit a navigation



Figure 4.8: Navigation path of 6 users using the SINA multi-user navigation algorithm

Table 4.3: Numerical comparison of the Navigation Time (NT) and the Cumulative Risk Score (CRS) the two multi-user algorithm

	SINA		PINA	
	CRS	NT (s)	CRS	NT (s)
User 1	12.35	484	12.35	474
User 2	10.13	436	13.96	436
User 3	9.45	498	9.45	468
User 4	9.85	461	9.91	461
User 5	11.76	474	11.76	474
User 6	11.91	528	12.09	498

or waiting process, and the line between them is the navigation period. actually, we can notice that there are edges with successive alphabetic letters (i.e. A, B, C, D, E, and F) which represent edges in the street network (i.e. presented in Fig. 4.8. Additionally, the letter W-i represents a waiting time due to conflict with the user i. If we take a general look at the figure 4.6, we can see some users with no waiting time and those users are prioritized users using the First In First Out (FIFO) principle. Other users have waiting time and this to avoid the conflicts. Table 4.3 compare risk score and the cumulative navigation time of both algorithms for each user. Actually, we can notice that the PINA over perform the SINA in term of navigation time due

to the absence of the waiting process (e.g. the users 1, 3, 6 reached their destination faster with PINA than the SINA and other users are unchanged). However, the SINA outperform in term of risk score. Indeed, the waiting process in the SINA is useful if the user aims to navigate 100% safely but if the user prefers to go faster and avoid the waiting process (e.g. running pedestrian), PINA is the suitable choice. The trade-off between the rapidity and the safety will be ensured by the choice of the user.

4.2 Conclusion

In this chapter, we described the exploited Data-set and the investigated area map. additionally, we have presented the street network graph of the Santander city after the preprocessing phase and we discussed the obtained CLOR and SFOR communities using several metrics to prove the efficiency of the community detection algorithm. Afterward, we showed pedestrians' tracking results and extracted information during the computer vision phase. A Monte-Carlo simulation was performed to study the effect trade-off between the distance and the safety of the selected routes. Finally, the multi-user and the real-time functionalities are investigated through real-life scenarios to show their performance and their ability to involve other framework users' during the navigation decision process.

Summary

Ubiquitous IoT can provide solutions to combat pandemics such as COVID-19. Also, we provide a practical framework that could assist in such circumstances. The framework recommends a trajectory for pedestrians to reach their destination while bypassing high risky areas. It employs technical and social advantages of smart IoT devices, graph theory, and computer vision techniques to determine risky areas and help people better practice social distancing. The framework works in a real-time manner based on the user's needs and mobility of other devices. Additionally, the framework involves the offline pedestrians by exploiting CCTV cameras to track and extract risk-related information, to be used during the computation phase of the road's weight. Accordingly, this framework is adapted easily to several cases. Besides, the framework is used to synchronize the navigation of pedestrians in indoor areas such as campuses and industrial workplaces. The framework, also, considers other navigating users during the navigation decision making by employing two multi-user Dijkstra-based navigation algorithms.

General Conclusion

During the outbreak of the global challenge of the COVID-19 pandemic, the reliance on technologies such as IoT, AI, Blockchain, Big Data Analytics, and Cloud computing has increased. IoT plays a major role in reducing the risks of coronavirus spread by providing platforms that help in following the protocols defined by WHO. Pandemic evolution forecasting solution assists governments in providing timely response to contain the spread of the virus. Additionally, IoT-based solution helps to mitigate the spread of the virus.

Our contribution to help in containing this pandemic tackles two main directions:

- For the first study, we developed a mechanistic model composed of eight states that aims to characterize the COVID-19 spread in different countries. An ODE system is formulated to mathematically model the interactions between 8 different states. Afterward, a curve fitting using real-world observed data sets is developed to determine accurate fitting solutions and estimate the parameters of the ODE model. We exploit the LM algorithm to solve and estimate the ODE system parameters. We have shown that the employed algorithms outperform a reference model, i.e., BFGS. The fitting technique is represented along with the model that aims to endorse its forecasting results by focusing on the last trends of the data, and hence, giving accurate results that are on the verge of these trends. Multiple countries with different spread trends are investigated with the proposed work. We reveal that our model can fit the real-world data of different countries showing different spread trends, namely, Russia, Brazil, Italy, and the USA. Despite the challenges and the various human factors, i.e., social distancing, quarantine, wearing masks that are not

directly characterized by our model and that may affect the spread and its evolution. It is shown that accurate forecasting results are obtained, e.g., Italy and Russia.

- As a second study, we proposed a practical framework that recommends a trajectory for pedestrians to reach their destination while bypassing areas with a high risk of exposure to the virus. It employs technical and social advantages of smart IoT devices, graph theory, and computer vision techniques to determine risky areas and help people better practice social distancing. The framework works in a real-time manner based on the user's needs and mobility of other devices. Additionally, the framework involves offline pedestrians by exploiting CCTV cameras to track and extract risk-related information to calculate roads' weights. Accordingly, this framework is easily adaptable to several cases. Also, the framework synchronizes the navigation of pedestrians in indoor areas such as campuses and industrial workplaces. The framework considers other navigating users during the navigation decision making by employing two multi-user Dijkstra-based navigation algorithms. In this work, the model showed very promising results in reducing the level of exposure to the virus while balancing between the safety and rapidity of the navigation.

Our work showed promising results in forecasting, assessing, and mitigating the spread of the pandemic. Consequently, governments could exploit the two proposed frameworks to contain the spread of the virus. In future work, we plan to include, in the pandemic's evolution model, more parameters to test the effect of governmental decisions on the spread of the virus to avoid any possible mistakes that could lead to an increase in the virus spread. Additionally, artificial intelligence techniques will be included to help in forecasting the evolution of the COVID-19. Indeed, some AI's fields are capable of solving the lack of data problem and hence, increase the efficiency of the current model by eliminating some previously mentioned assumptions.

For the routing framework, we plan to enhance the computer vision phase by extracting more risk-related information, such as detect pedestrians that disobey the face-masking rule. Accordingly, we intend to reduce the computational time and the complexity of the framework

by enhancing each phase of the framework and especially, the multi-user navigation. Finally, we aim to exploit artificial intelligence algorithms, during the community detection, to extract more social relations and enhance the performance of the framework.

Accepted/Submitted publications

[1] **H. Friji**, H. Ghazzai, H. Besbes, and Y. Massoud, "A DQN-Based autonomous Car-Following framework using RGB-D frames," **accepted** in IEEE Global Conference on Artificial Intelligence and Internet of Things (GCAIoT) (2020 IEEE GCAIoT), Dec.2020.

[2] A. Khanfor, **H. Friji**, H. Ghazzai, and Y. Massoud, *"A Social IoT-driven Pedestrian Routing Approach during Epidemic Time"* **accepted** *in IEEE Global Conference on Artificial Intelligence and Internet of Things (GCAIoT) (2020 IEEE GCAIoT), Dec.2020.*

[3] **H. Friji**, R. Hamadi, H. Ghazzai, H. Besbes, and Y. Massoud, *"A Generalized Mechanistic Model for Assessing and Forecasting the Spread of the COVID-19 Pandemic"* **submitted** *to IEEE Access*.

[4] M.Massmoudi, **H. Friji**, H. Ghazzai, H. Besbes, and Y. Massoud, *"A Reinforcement Learning Framework for Video Frame-based Autonomous Car-following"* **submitted** *to IEEE Open Journal of Intelligent Transportation Systems (OJ-ITS).*

[5] **H. Friji**, A. Khanfor, H. Ghazzai, and Y. Massoud, *"An End-to-End Framework for Pedestrians' Routing to Mitigate the spread of Pandemics"* **to be submitted** .

References

[1] Z. Xu, L. Shi, Y. Wang, J. Zhang, L. Huang, C. Zhang, S. Liu, P. Zhao, H. Liu, L. Zhu *et al.*, "Pathological findings of COVID-19 associated with acute respiratory distress syndrome," *The Lancet respiratory medicine*, vol. 8, no. 4, pp. 420–422, Feb. 2020.

[2] A. Nasir, K. Shaukat, I. A. Hameed, S. Luo, T. M. Alam, and F. Iqbal, "A bibliometric analysis of corona pandemic in social sciences: A review of influential aspects and conceptual structure," *IEEE Access*, vol. 8, pp. 133 377–133 402, July 2020.

[3] Q. Li et al., "Early transmission dynamics in Wuhan, China, of novel coronavirus–infected pneumonia," *New England Journal of Medicine*, vol. 382, no. 13, pp. 1199–1207, Mar. 2020.

[4] N. Van Doremalen, T. Bushmaker, D. H. Morris, M. G. Holbrook, A. Gamble, B. N. Williamson, A. Tamin, J. L. Harcourt, N. J. Thornburg, S. I. Gerber *et al.*, "Aerosol and surface stability of SARS-CoV-2 as compared with SARS-CoV-1," *New England Journal of Medicine*, vol. 382, no. 16, pp. 1564–1567, Apr. 2020.

[5] P. Staszkiewicz, I. Chomiak-Orsa, and I. Staszkiewicz, "Dynamics of the COVID-19 contagion and mortality: Country factors, social media, and market response evidence from a global panel analysis," *IEEE Access*, vol. 8, pp. 106 009–106 022, June 2020.

[6] J. T. Wu, K. Leung, and G. M. Leung, "Nowcasting and forecasting the potential domestic and international spread of the 2019-nCoV outbreak originating in Wuhan, China: a modelling study," *The Lancet*, vol. 395, no. 10225, pp. 689–697, Jan. 2020.

[7] F. Song, N. Shi, F. Shan, Z. Zhang, J. Shen, H. Lu, Y. Ling, Y. Jiang, and Y. Shi, "Emerging 2019 novel coronavirus (2019-nCoV) pneumonia," *Radiology*, vol. 295, no. 1, pp. 210–217, Feb. 2020.

[8] H. Shi, X. Han, N. Jiang, Y. Cao, O. Alwalid, J. Gu, Y. Fan, and C. Zheng, "Radiological findings from 81 patients with covid-19 pneumonia in wuhan, china: a descriptive study," *The Lancet Infectious Diseases*, vol. 20, no. 4, pp. 425 – 434, Apr. 2020. [Online]. Available: http://www.sciencedirect.com/science/article/pii/S1473309920300864

[9] "Coronavirus disease 2019 (COVID-19): situation report of the 1st july, 163," *World Health Organization, tech. rep.*, 2020.

[10] V. Chamola, V. Hassija, V. Gupta, and M. Guizani, "A comprehensive review of the COVID-19 pandemic and the role of IoT, Drones, AI, Blockchain, and 5G in managing its impact," *IEEE Access*, vol. 8, pp. 90 225–90 265, May 2020.

[11] R. Y. Kim, "The impact of COVID-19 on consumers: Preparing for digital sales," *IEEE Engineering Management Review*, vol. 48, no. 3, pp. 212–218, Apr. 2020.

[12] C. Mark and W. Craig, "Understanding COVID-19's impact on the telecom sector," *Deloitte, tech. rep.*, Apr. 2020.

[13] A. L. Kapetanović and D. Poljak, "Modeling the epidemic outbreak and dynamics of COVID-19 in Croatia," *arXiv preprint arXiv:2005.01434*, 2020.

[14] K. Linka, M. Peirlinck, and E. Kuhl, "The reproduction number of COVID-19 and its correlation with public health interventions," *medRxiv*, July 2020. [Online]. Available: https://www.medrxiv.org/content/early/2020/07/07/2020.05.01.20088047

[15] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, Dec 1959. [Online]. Available: https://doi.org/10.1007/BF01386390

[16] J. Nocedal and S. J. Wright, *Numerical Optimization*, 2nd ed. New York, NY, USA: Springer, 2006.

[17] D. W. Marquardt, "An algorithm for least-squares estimation of nonlinear parameters," *SIAM Journal on Applied Mathematics*, vol. 11, no. 2, pp. 431–441, 1963. [Online]. Available: http://dx.doi.org/10.1137/0111030

[18] G. Ramos-Llordén, G. Vegas-Sánchez-Ferrero, M. Björk, F. Vanhevel, P. M. Parizel, R. San José Estépar, A. J. den Dekker, and J. Sijbers, "NOVIFAST: A Fast Algorithm for Accurate and Precise VFA MRI T_1 Mapping," *IEEE Transactions on Medical Imaging*, vol. 37, no. 11, pp. 2414–2427, June 2018.

[19] J. Chen and W. Li, "Convergence of Gauss-Newton's method and uniqueness of the solution," *Appl. Math. Comput.*, vol. 170, no. 1, pp. 686–705, Nov. 2005. [Online]. Available: http://dblp.uni-trier.de/db/journals/amc/amc170.html#ChenL05a

[20] G. Sertsou, N. Wilson, M. Baker, P. Nelson, and M. G. Roberts, "Key transmission parameters of an institutional outbreak during the 1918 influenza pandemic estimated by mathematical modelling," *Theoretical Biology and Medical Modelling*, vol. 3, no. 1, p. 38, Nov. 2006. [Online]. Available: https://doi.org/10.1186/1742-4682-3-38

[21] A. Flahault, E. Vergu, L. Coudeville, and R. F. Grais, "Strategies for containing a global influenza pandemic," *Vaccine*, vol. 24, no. 44, pp. 6751 – 6755, Nov. 2006. [Online]. Available: http://www.sciencedirect.com/science/article/pii/S0264410X06006311

[22] A. K. Lugnér, M. van Boven, R. de Vries, M. J. Postma, and J. Wallinga, "Cost effectiveness of vaccination against pandemic influenza in European countries: mathematical modelling analysis," *BMJ*, vol. 345, p. 4445, July 2012.

[23] Pandemic Influenza Outbreak Research Modelling Team (Pan-InfORM and others, "Modelling an influenza pandemic: A guide for the perplexed," *CMAJ*, vol. 181, no. 3-4, pp. 171–173, July 2009.

[24] D. Menon, B. Taylor, S. Ridley, and U. Intensive Care Society, "Modelling the impact of an influenza pandemic on critical care services in England," *Anaesthesia*, vol. 60, no. 10, pp. 952–954, Sept. 2005.

[25] A. J. Kucharski, T. W. Russell, C. Diamond, Y. Liu, J. Edmunds, S. Funk, R. M. Eggo, F. Sun, M. Jit, J. D. Munday, N. Davies, A. Gimma, K. van Zandvoort, H. Gibbs, J. Hellewell, C. I. Jarvis, S. Clifford, B. J. Quilty, N. I. Bosse, S. Abbott, P. Klepac, and S. Flasche, "Early dynamics of transmission and control of COVID-19: a mathematical modelling study," *The Lancet Infectious Diseases*, vol. 20, no. 5, pp. 553 – 558, May 2020. [Online]. Available: http://www.sciencedirect.com/science/article/pii/S1473309920301444

[26] K. Prem, Y. Liu, T. W. Russell, A. J. Kucharski, R. M. Eggo, N. Davies, Centre for the Mathematical Modelling of Infectious Diseases COVID-19 Working Group, M. Jit, and P. Klepac, "The effect of control strategies to reduce social mixing on outcomes of the covid-19 epidemic in wuhan, china: a modelling study," *The Lancet. Public health*, vol. 5, no. 5, p. e261—e270, May 2020. [Online]. Available: https://europepmc.org/articles/PMC7158905

[27] Q. Lin, S. Zhao, D. Gao, Y. Lou, S. Yang, S. S. Musa, M. H. Wang, Y. Cai, W. Wang, L. Yang, and D. He, "A conceptual model for the coronavirus disease 2019 (COVID-19) outbreak in Wuhan, China with individual reaction and governmental action," *International Journal of Infectious Diseases*, vol. 93, pp. 211 – 216, Mar. 2020.

[28] G. Giordano, F. Blanchini, R. Bruno, P. Colaneri, A. Di Filippo, A. Di Matteo, and M. Colaneri, "Modelling the COVID-19 epidemic and implementation of population-wide interventions in italy," *Nature Medicine*, vol. 26, no. 6, pp. 855–860, June 2020. [Online]. Available: https://doi.org/10.1038/s41591-020-0883-7

[29] C. Anastassopoulou, L. Russo, A. Tsakris, and C. Siettos, "Data-based analysis, modelling and forecasting of the COVID-19 outbreak," *PLOS ONE*, vol. 15, no. 3, pp. 1–21, Mar. 2020.

[30] S. Chang, N. Harding, C. Zachreson, O. M. Cliff, and M. Prokopenko, "Modelling transmission and control of the COVID-19 pandemic in Australia," *ArXiv*, vol. abs/2003.10218, May 2020.

[31] G. Chowell, "Fitting dynamic models to epidemic outbreaks with quantified uncertainty: A primer for parameter uncertainty, identifiability, and forecasts," *Infectious Disease Modelling*, vol. 2, no. 3, pp. 379 – 398, Aug. 2017. [Online]. Available: http://www.sciencedirect.com/science/article/pii/S2468042717300234

[32] G. Chowell, L. Sattenspiel, S. Bansal, and C. Viboud, "Mathematical models to characterize early epidemic growth: A review," *Physics of life reviews*, vol. 18, pp. 66–97, Sept. 2016.

[33] B. Pell, Y. Kuang, C. Viboud, and G. Chowell, "Using phenomenological models for forecasting the 2015 Ebola challenge," *Epidemics*, vol. 22, pp. 62 – 70, Mar. 2018. [Online]. Available: http://www.sciencedirect.com/science/article/pii/S1755436516300433

[34] K. Wu, D. Darcet, Q. Wang, and D. Sornette, "Generalized logistic growth modeling of the COVID-19 outbreak: comparing the dynamics in the 29 provinces in China and in the rest of the world," *Nonlinear Dynamics*, 10.1007/s11071-020-05862-6 Aug. 2020. [Online]. Available: https://doi.org/10.1007/s11071-020-05862-6

[35] C. Viboud, L. Simonsen, and G. Chowell, "A generalized-growth model to characterize the early ascending phase of infectious disease outbreaks," *Epidemics*, vol. 15, pp. 27 – 37, June 2016. [Online]. Available: http://www.sciencedirect.com/science/article/pii/S1755436516000037

[36] F. Yang, H. Liu, H. Qi, and X. Liu, "SEIR evolutionary simulation model of the infectious disease emergency," in *International Conference on Industrial Informatics - Comput-*

ing Technology, Intelligent Technology, Industrial Information Integration (ICIICII'16), Wuhan, China, Dec. 2016, pp. 315–318.

[37] B. Xu, J. Cai, D. He, G. Chowell, and B. Xu, "Mechanistic modelling of multiple waves in an influenza epidemic or pandemic," *Journal of Theoretical Biology*, vol. 486, p. 110070, Feb. 2020. [Online]. Available: http://www.sciencedirect.com/science/article/pii/S0022519319304394

[38] J. Lessler and D. A. T. Cummings, "Mechanistic models of infectious disease and their impact on public health," *American journal of epidemiology*, vol. 183, no. 5, pp. 415–422, Mar. 2016. [Online]. Available: https://pubmed.ncbi.nlm.nih.gov/26893297

[39] W. Chen, "A mathematical model of Ebola virus based on SIR model," in *International Conference on Industrial Informatics - Computing Technology, Intelligent Technology, Industrial Information Integration*, Wuhan, China, Dec. 2015, pp. 213–216.

[40] Y. Maki and H. Hirose, "Infectious disease spread analysis using stochastic differential equations for SIR model," in *4th International Conference on Intelligent Systems, Modelling and Simulation*, Bangkok, Thailand, Jan. 2013, pp. 152–156.

[41] T. W. Ng, G. Turinici, and A. Danchin, "A double epidemic model for the sars propagation," *BMC infectious diseases*, vol. 3, pp. 19–19, Sept. 2003. [Online]. Available: https://pubmed.ncbi.nlm.nih.gov/12964944

[42] T. S. Do and Y. S. Lee, "Modeling the spread of ebola," *Osong public health and research perspectives*, vol. 7, no. 1, pp. 43–48, Feb. 2016. [Online]. Available: https://pubmed.ncbi.nlm.nih.gov/26981342

[43] E. Dantas, M. Tosin, and A. Cunha Jr, "Calibration of a SEIR-SEI epidemic model to describe the Zika virus outbreak in Brazil," *Applied Mathematics and Computation*, vol. 338, pp. 249–259, Dec. 2018.

[44] L. Peng, W. Yang, D. Zhang, C. Zhuge, and L. Hong, "Epidemic analysis of COVID-19 in China by dynamical modeling," *arXiv preprint arXiv:2002.06563*, Feb. 2020.

[45] J. Yawney and S. A. Gadsden, "A study of the COVID-19 impacts on the canadian population," *IEEE Access*, vol. 8, pp. 128 240–128 249, July 2020.

[46] A. Hauser, M. J. Counotte, C. C. Margossian, G. Konstantinoudis, N. Low, C. L. Althaus, and J. Riou, "Estimation of SARS-CoV-2 mortality during the early stages of an epidemic:

A modeling study in Hubei, China, and six regions in Europe," *PLOS Medicine*, vol. 17, pp. 1–17, July 2020.

[47] G. Giordano, F. Blanchini, R. Bruno, P. Colaneri, A. Di Filippo, A. Di Matteo, and M. Colaneri, "Modelling the COVID-19 epidemic and implementation of population-wide interventions in Italy," *Nature Medicine*, vol. 26, no. 6, pp. 855–860, June 2020. [Online]. Available: https://doi.org/10.1038/s41591-020-0883-7

[48] R. Gopal, V. Chandrasekar, and M. Lakshmanan, "Dynamical modelling and analysis of COVID-19 in India," *arXiv preprint arXiv:2005.08255*, May 2020.

[49] Y. Zhang, Y. Li, B. Yang, X. Zheng, and M. Chen, "Risk assessment of COVID-19 based on multisource data from a geographical viewpoint," *IEEE Access*, vol. 8, pp. 125 702–125 713, June 2020.

[50] F. Rustam, A. A. Reshi, A. Mehmood, S. Ullah, B. On, W. Aslam, and G. S. Choi, "COVID-19 future forecasting using supervised machine learning models," *IEEE Access*, vol. 8, pp. 101 489–101 499, May 2020.

[51] R. D. Kirkcaldy, B. A. King, and J. T. Brooks, "COVID-19 and Postinfection Immunity: Limited Evidence, Many Remaining Questions," *JAMA*, vol. 323, no. 22, pp. 2245–2246, June 2020.

[52] M. Small and D. Cavanagh, "Modelling strong control measures for epidemic propagation with networks—a COVID-19 case study," *IEEE Access*, vol. 8, pp. 109 719–109 731, June 2020.

[53] R. Gupta, G. Pandey, P. Chaudhary, and S. K. Pal, "SEIR and regression model based COVID-19 outbreak predictions in India," *medRxiv*, Apr. 2020. [Online]. Available: https://www.medrxiv.org/content/early/2020/04/03/2020.04.01.20049825

[54] S. X. D. Tan and L. Chen, "Real-time differential epidemic analysis and prediction for COVID-19 pandemic," Apr. 2020.

[55] M. Kommenda, M. Affenzeller, G. Kronberger, and S. M. Winkler, "Nonlinear least squares optimization of constants in symbolic regression," in *Computer Aided Systems Theory - EUROCAST 2013*, R. Moreno-Díaz, F. Pichler, and A. Quesada-Arencibia, Eds.

[56] A. Botchkarev, "A new typology design of performance metrics to measure errors in machine learning regression algorithms," *Interdisciplinary Journal of Information, Knowledge, and Management*, vol. 14, pp. 45–76, Dec. 2019.

[57] J. Redmon and A. Farhadi, "Yolov3: An incremental improvement," 2018, arXiv:1804.02767.

[58] Q.-V. Pham, D. C. Nguyen, W.-J. Hwang, P. N. Pathirana *et al.*, "Artificial intelligence (ai) and big data for coronavirus (covid-19) pandemic: A survey on the state-of-the-arts," 2020.

[59] A. Mathew, F. A. SA, H. Pooja, and A. Verma, "Smart disease surveillance based on internet of things (iot)," *International Journal of Advanced Research in Computer and Communication Engineering*, vol. 4, no. 5, pp. 180–183, 2015.

[60] R. P. Singh, M. Javaid, A. Haleem, and R. Suman, "Internet of things (iot) applications to fight against covid-19 pandemic," *Diabetes Metabolic Syndrome: Clinical Research Reviews*, vol. 14, no. 4, pp. 521 – 524, July-Aug. 2020. [Online]. Available: http://www.sciencedirect.com/science/article/pii/S1871402120301065

[61] V. Chamola, V. Hassija, V. Gupta, and M. Guizani, "A comprehensive review of the covid-19 pandemic and the role of iot, drones, ai, blockchain, and 5g in managing its impact," *IEEE Access*, vol. 8, pp. 90 225–90 265, May 2020.

[62] N. C. Peeri, N. Shrestha, M. S. Rahman, R. Zaki, Z. Tan, S. Bibi, M. Baghbanzadeh, N. Aghamohammadi, W. Zhang, and U. Haque, "The SARS, MERS and novel coronavirus (COVID-19) epidemics, the newest and biggest global health threats: what lessons have we learned?" *International Journal of Epidemiology*, vol. 49, no. 3, pp. 717–726, Feb, 2020. [Online]. Available: https://doi.org/10.1093/ije/dyaa033

[63] D. S. W. Ting, L. Carin, V. Dzau, and T. Y. Wong, "Digital technology and covid-19," *Nature medicine*, vol. 26, no. 4, pp. 459–461, Mar. 2020.

[64] C. J. Wang, C. Y. Ng, and R. H. Brook, "Response to covid-19 in taiwan: big data analytics, new technology, and proactive testing," *Jama*, vol. 323, no. 14, pp. 1341–1342, Apr. 2020.

[65] M. Kamal, A. Aljohani, and E. Alanazi, "Iot meets covid-19: Status, challenges, and opportunities," *arXiv preprint arXiv:2007.12268*, 2020.

[66] M. Hatzopoulou, S. Weichenthal, G. Barreau, M. Goldberg, W. Farrell, D. Crouse, and N. Ross, "A web-based route planning tool to reduce cyclists' exposures to traffic pollution: A case study in montreal, canada," *Environmental Research*, vol. 123, pp. 58–61, May 2013.

[67] A. Pimpinella, A. E. Redondi, and M. Cesana, "Walk this way! an iot-based urban routing system for smart cities," *Computer Networks*, vol. 162, p. 106857, Oct. 2019.

[68] A. Khanfor et al., "Application of community detection algorithms on social internet-of-things networks," in *IEEE International Conference on Microelectronics (ICM'19)*, Dec. 2019, pp. 94–97, Cairo, Egypt.

[69] L. Atzori, A. Iera, G. Morabito, and M. Nitti, "The social internet of things (SIoT)–when social networks meet the internet of things: Concept, architecture and network characterization," *Computer networks*, vol. 56, no. 16, pp. 3594–3608, Nov. 2012.

[70] J. Shuja, K. Bilal, E. Alanazi, W. Alasmary, and A. Alashaikh, "Applying machine learning techniques for caching in edge networks: A comprehensive survey," *arXiv preprint arXiv:2006.16864*, 2020.

[71] A. Khanfor, R. Hamadi, H. Ghazzai, Y. Yang, M. R. Haider, and Y. Massoud, "Computational resource allocation for edge computing in social internet-of-things," in *2020 IEEE 63rd International Midwest Symposium on Circuits and Systems (MWSCAS)*, Aug. 2020, pp. 233–236.

[72] V. D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre, "Fast unfolding of communities in large networks," *Journal of statistical mechanics: theory and experiment*, vol. 2008, no. 10, p. P10008, Oct. 2008.

[73] A. Khanfor, H. Ghazzai, Y. Yang, M. R. Haider, and Y. Massoud, "Automated service discovery for social internet-of-things systems," *IEEE International Symposium on Circuits and Systems (ISCAS'20), Seville, Spain.*, Oct. 2020.

[74] S. Wang and R. Koopman, "Clustering articles based on semantic similarity," *Scientometrics*, vol. 111, no. 2, pp. 1017–1031, May 2017. [Online]. Available: https://doi.org/10.1007/s11192-017-2298-x

[75] H. Almeida, D. Guedes, W. Meira, and M. J. Zaki, "Is there a best quality metric for graph clusters?" in *Machine Learning and Knowledge Discovery in Databases*, D. Gunopulos, T. Hofmann, D. Malerba, and M. Vazirgiannis, Eds., Sept. 2011.

[76] N. Wojke and A. Bewley, "Deep cosine metric learning for person re-identification," in *IEEE Winter Conference on Applications of Computer Vision (WACV)*. IEEE, 2018, pp. 748–756.

[77] G. Boeing, "Osmnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks," *Computers, Environment and Urban Systems*, vol. 65, pp. 126–139, May 2017.

[78] C. Marche et al., "A dataset for performance analysis of the social internet of things," in *IEEE Annual International Symposium on Personal, Indoor and Mobile Radio Communications (PIMRC'18)*, Bologna, Italy, Sept. 2018.

[79] D. J. Watts and S. H. Strogatz, "Collective dynamics of 'small-world'networks," *nature*, vol. 393, no. 6684, p. 440, June 1998.

APPENDIX 1

A DQN-based Autonomous Car-following Framework Using RGB-D Frames

Hamdi Friji[1], Hakim Ghazzai[1], Hichem Besbes[2], and Yehia Massoud[1]

[1]School of Systems and Enterprises, Stevens Institute of Technology, Hoboken, NJ, USA
[2]Higher School of Communications of Tunis, University of Carthage, Tunis, Tunisia

***Abstract*—Modeling car-following behavior has recently garnered much attention due to the wide variety of applications it may be utilized in, such as accident analysis, driver assessment, and support systems. Some of the latest approaches investigate scenario-based autonomous driving algorithms. In this paper, we propose an end-to-end car-following framework that, based on high dimensional RGB-D features only, it ensures autonomous driving by following the actions of a leader car while taking into account other environmental factors (e.g. pedestrians, sidewalk crashing, etc.) To this end, a reinforcement learning (RL) algorithm, precisely an improved Deep Q-Network algorithm, is designed to avoid crashes with the leader car and its detection loss while effectively driving on road. The model is trained and tested using the CARLA simulator in different environments. Our preliminary tests show promising results for enhancing the driving capabilities of autonomous vehicles in many situations such as highways, one-way roads, and no-overtaking roads.**

***Index Terms*—Artificial intelligence, autonomous vehicle, intelligent transportation systems, reinforcement learning, car-following.**

I. Introduction

Autonomous driving [1] [2], by definition, is the ability to sense the environment and navigate without any human involvement. Ideally, a self-driving vehicle does not require a human passenger to take its control at any time. Autonomous vehicles are expected to be empowered with capabilities allowing them to navigate anywhere as experienced human drivers do. However, enabling a self-driving vehicle to safely navigate through complex environments is not an easy task. At any instant, an autonomous vehicle must make several decisions based on the current road situation. One of the latest emerging challenges for autonomous vehicles is the design of effective car-following approaches, which consists of making an autonomous vehicle, called a follower, to autonomously follow another vehicle, called the leader [3], [4]. Every decision of the follower will be correlated with the behavior of the leader and constrained by the navigation environment. Such an approach is expected to enhance the autonomous driving policy in many situations, such as driving on highways, one-way roads, and no-overtaking roads where vehicles need to follow each other safely.

Artificial Intelligence (AI) [5] is playing a key role in designing and automating the autonomous vehicle operation. Thus, over the last few years, the autonomous driving research area has been invaded by all kinds of AI algorithms, especially the Reinforcement Learning (RL) algorithms. In fact, Deep RL (DRL) is a type of deep learning technique that enables an agent to learn in an interactive environment from trial and error using feedback from its own actions and experiences. Generally, the goals of RL differ from those of Machine Learning (ML) [6]. While the goal in ML [7] is to find similarities and differences between data points, in RL, the objective is to find a suitable action model that would maximize the total cumulative reward of the agent. Therefore, RL is much more efficient for autonomous driving problems.

In previous car-following studies, researchers used the car-following approach to mimic a human-like driving by exploiting features extracted from different sensory inputs. To address this purpose, several car-following approaches are proposed. In [4], the authors suggested employing the Q-learning algorithm, for the car-following problem, based on video-frames processing only with a limited state-space and action-space, to build a car-following agent. In [8], the authors proposed a driving style recognition model based on vehicle trajectory data to avoid rear-end collision crash and hence, design driver assistance and vehicle control system. In [9], Zhu et al suggested a framework for human-like autonomous car-following. This method employed DRL to predict the appropriate acceleration from speed, the relative speed between vehicles, and inter-vehicle spacing. The results prove that the proposed model can contribute to the development of advanced traffic-flow models. In [10], Gao et al. established the reward function of every driver data, based on Inverse RL (IRL) algorithm and analyzed the driving characteristics and the following strategies. Also, they discussed an anthropomorphic following model and presented a learning car-following algorithm by using IRL combined with car-following data of a driving simulator. Finally, the authors of [11] proposed to combine the machine learning models with the kinematics-based car-following models that can overcome the shortcomings of machine learning models, using an optimal combination of prediction methods to create a novel car-following model able to avoid crashing. Unlike previous works, which have mainly focused on typical car-following problems for human-driving vehicles or using simplified data inputs such as vehicles positions and speed, this study proposes an end-to-end car-following framework that exploits the Red Green Blue and Depth (RGB-D) [12] frames to make autonomous navigation decisions.

In this paper, we develop a car-following framework based on an improved version of the Deep Q-Networks (DQN) algorithm. The objective of the improvements of the DQN model is to extract the most significant information from image sensory inputs and combine them with other environmental features to enhance the driving capabilities of the autonomous vehicle.

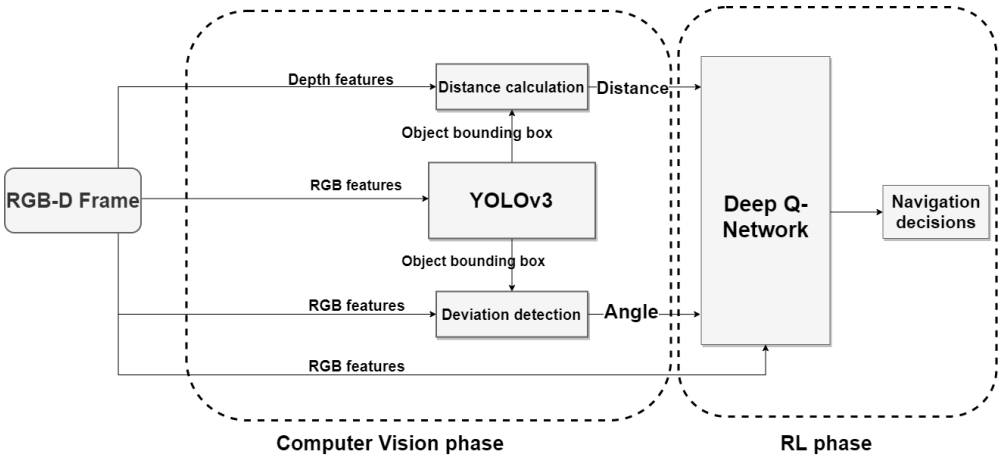


Fig. 1: The architecture of the proposed autonomous car-following model.

Our intention is to investigate the ability of the framework to ensure the following of a leader-vehicle using only image-based features collected from the front camera of the follower. This is performed while avoiding collision with the leader vehicle and any other actor in the environment without losing the detection of the leader. The proposed approach, presented in Fig. 1, is essentially based on the two following phases:

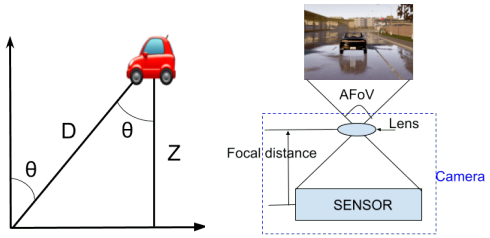
• Computer vision phase: the agent automatically detects objects in the environment then uses the characteristics of depth images to compute the distance and the deviation angle between the leader and the follower [13] by exploiting the information extracted with the YOu Look Once version 3 (YOLOv3) algorithm [14].

• RL phase: a modified version of the DQN algorithm is employed to exploit the high dimensional inputs alongside with some additional features extracted from phase 1 to make real-time navigation decisions that ensure a safe distance with the leader without losing his detection or crashing with other objects.

The proposed framework is simulated on the CARLA simulator and trained to navigate in a way to follow a leader-vehicle in several predefined scenarios using seven different maps and four weather conditions, then tested with three other maps. The results of the model are very promising for the autonomous car-following approach. Indeed, the model is capable of following the leader along the whole trajectory in almost 97.5% of the testing episodes without crashing, losing the leader visibility, or going out of the road. Our vision of autonomous driving is characterized by multiple use-cases; the vehicle should recognize the specific situation it is going to drive on and act based on this use-case. These promising results of this work support our vision and encourage further investigation of the problem by involving other environmental actors and different sensors inputs, such as LIDAR and RADAR, that will enhance the efficiency of car-following driving.

II. Computer Vision Phase

The computer vision phase consists of collecting and extracting the necessary features to be used later as an input for



(a) Deviation angle (b) AFoV

Fig. 2: Illustration of the deviation angle and the AFoV.

the RL phase. This part, which is composed of three sub-tasks is entirely based on RGB-D images, which include information about the depth-map, the distance separating the follower and the leader, and the navigation environment. The first task is to detect objects on the road (e.g., cars, pedestrians, etc.). This task is performed using the YOLOv3 algorithm fine-tuned to detect only the objects that influence the decision making of the follower. The YOLOv3 algorithm returns, for every detected object, four coordinates of a bounding box that locates the object in the image. For every frame, we use these outputs to map the object pixels to the depth feature pixels, and consequently, calculate the distance and the deviation angle between the follower vehicle and the leader assuming that the most representative pixel for the object is the pixel in the center of the bounding box.

a) YOLOv3: The third version of the multi-object detection algorithm YOLO, composed of 106 layers with 75 convolutional layers and 31 other layers, has overtaken the most of other object detection algorithms with being capable of detecting small objects in a very accurate and fast way [14]. The YOLOv3-608 is outperforming all the two-stage detectors. It is almost two times faster than all other algorithms. Indeed, YOLOv3 performs detection on three scales to be able to detect many objects with all sizes. Therefore, it can detect distant and close vehicles. As the network goes deeper, its feature map gets smaller. Furthermore, YOLOv3 is a feature-learning based network and can deal with images from any size due to the use of no fully-connected layer, which emphasizes

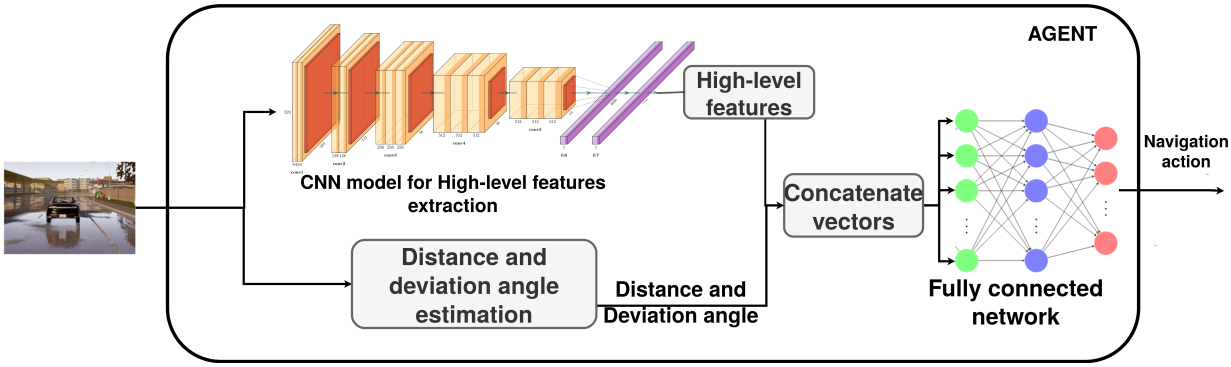


Fig. 3: Architecture of the RL agent.

efficiency for real-time object detection.

b) Distance and deviation angle calculation: The distance and deviation angle are calculated using the RGB-D frames provided by simulated sensory inputs (e.g. the Orbbec 3D camera) attached to the follower vehicle. Using the bounding box center point provided by YOLOv3 as a reference of the object location on the image, it is possible to accurately estimate the distance and the deviation angle. This method is a way better than using the bounding box coordinates or the 3D object detection due to its complexity or limited accuracy. To this end, we need to express the angular size of each pixel horizontally, denoted by Γ_x, as follows:

$$\Gamma_x = \frac{AFoV}{ImW}, \tag{1}$$

where ImW is the image width and $AFoV$ is the horizontal Angular Field of View (AFoV). The latter is a constant specific for every camera. In practice, AFoV is a measure of the vantage point from the lens distance as shown in Fig. 2. For the Orbbec 3D sensor, $AFoV$= 60°.

The camera is positioned at (0,0,0) in the Cartesian coordinate system and the central pixel of the bounding box is positioned at (i, j) without taking into consideration the third dimension. As a start, we proceed by calculating the angle of deviation, denoted by θ, using the following expression:

$$\theta = \Gamma_x \times \left(i - \frac{ImW}{2}\right). \tag{2}$$

Then, we compute the distance D separating the follower and the leader as $D = \frac{Z}{cos(\theta)}$, where Z is the distance between the sensor plane and the object, which is directly extracted from the depth feature of the sensor. Each depth sensor has a depth scale value that translates pixels to meters.

III. Reinforcement Learning Phase

In this section, we present the RL approach to address the car-following problem. The architecture of the RL model is given in Fig. 3.

a) States: In our model, every state is defined by the distance D and the deviation angle θ, between the leader and the follower vehicle, in addition to the high-level features extracted by the Convolutional Neural Network (CNN) from every frame. Hence, the autonomous follower extracts the previous features from the frames and concatenates them with the output of the CNN that contains all semantic information about the environment to identify the state.

b) Actions: Our model is trained to choose an action from an action-space composed of eight possible behaviours as shown in Table 1. The possible actions include deceleration, acceleration, and steering left and right.

TABLE I: Actions space of the model

Action	Description
a_0	Accelerate
a_1	Steer left at same speed
a_2	Steer right at same speed
a_3	Accelerate and steer left
a_4	Accelerate and steer right
a_5	Decelerate
a_6	Steer left and Decelerate
a_7	Steer right and Decelerate

c) Reward Function: The reward function in our case should consider the distance D, the deviation angle θ, and the collision risk with the leader-vehicle. Consequently, the autonomous vehicle will be forced to follow the leader. Therefore, we suggest a piecewise function that considers the maximum distance, D_{max}, to successfully detect the leader vehicle and a danger distance, D_{min}, that refers to the distance where the probability of crashing with the leader is significant. The reward function of the RL algorithm for the proposed car-following approach, denoted by $r(D, \theta)$, can be expressed as follows:

$$r(D,\theta) = \begin{cases} \zeta - \alpha((\beta \times D - \delta)^2 + \theta^2) & \text{if } D_{\text{min}} \leq D \leq D_{\text{max}}, \\ -\infty & D < D_{\text{min}}, \end{cases} \tag{3}$$

where α is the coefficient of proportionality between the reward and the variables (i.e Distance D and angle θ), $\frac{\delta}{\beta}$ is the ideal distance between the leader and the follower, and ζ is the maximum reward.

As we can notice in Fig. 4, the reward function is maximized if θ is around 0° and the distance is equal to $\frac{D_{\text{max}}}{2}$ which means that the autonomous agent is attempting to maintain an angle to stay behind the leader within a safe distance to keep its detection and avoid any risk of crashing. Also, to decrease the probability of crashing with the leader, the model considers any case with distance D less than D_{min} as a potential crashing situation. Distances higher than D_{max} may lead to non-detection cases.

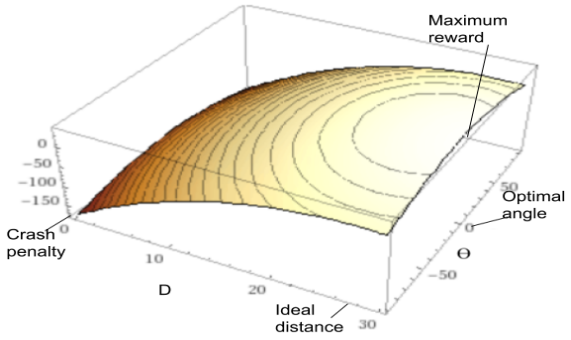


Fig. 4: Plot of an example of the reward function for $D_{\text{min}} < D < D_{\text{max}}$ where $\alpha = 0.005$, $\beta = 7$, $\delta = 190$, and $\zeta = 40$.

d) RL Algorithm: In DQN, we usually exploit the CNN to extract the relevant information in the image, with the goal of approximating the Q-value function, defined later in (5), that will be used for the decision making process. In our proposed model, we suggest to change the architecture of the CNN model inside the DQN algorithm by adding additional features that help the model avoid crashing with the leader and ensure a high-level safety driving. The RGB-D frame alongside the extra features are given as the input and the maximum Q-values of all possible actions are generated as the output.

At each step t, in Algorithm 1, the agent inspects the current state S_t of the environment, decides which action to be taken from the predefined action-space according to a policy π, and finally observes a reward signal r_t to evaluate this action. The goal of the agent is to identify the best policy that maximizes the expected sum of discounted rewards R_t expressed as follows:

$$R_t \doteq r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + ... + \gamma^T r_{t+T} \doteq \sum_{k=0}^{T} \gamma^k r_{t+k}, \quad (4)$$

where $\gamma \in [0, 1]$ is a discount factor that determines the importance of future rewards and T is the number of steps. The Q-function of a given policy π is defined as the expected return from executing an action a in a state S as follows:

$$Q^{\pi}(s, a) = \mathbb{E}\left[R_t \mid s_t = s, a_t = a, \pi\right]. \quad (5)$$

In the algorithm, we stock last agent experiences (s_t, a_t, r_t, s_{t+1}) into a buffer B. Then, we take samples already stocked to train the deep network. The idea behind taking samples from the replay buffer is that the data becomes almost independent and identically distributed (i.i.d.). Consequently, we obtain a more generalized model. In particular, DQN uses deep learning into the aim of obtaining an estimate of the Q-function for the current policy which is close to the optimal Q-function Q^* defined as the highest return we can expect to achieve by following any strategy. Hence, to successfully reach the Q^* we should find the optimal model weights, denoted by Θ. To reach a more stable input and output, we create two deep networks, the target network (weighted with Θ') and the standard network (weighted with Θ). We use the first one to retrieve Q values while the second one includes all updates in the training. After C steps, we synchronize Θ' with Θ that fixes Q-value targets temporarily, in consideration of the fact that the data is not 100% independent and identically distributed.

Algorithm 1 DQN Algorithm for the Car-following Driving

Initialize replay memory $\mathcal{B}$,
Initialize number of episodes M,
Initialize action-value function Q with two random sets of weights Θ and Θ',
ε = 1,
for $i = 1, \cdots, M$ **do**
  for $t = 1, \cdots, T$ **do**
    Choose $N \in [0..1]$,
    Set $a_t = \begin{cases} \arg\max_a Q(s_t, a; \Theta), & \text{if } N > \epsilon \\ \text{random action}, & \text{otherwise,} \end{cases}$
    Execute action a_t, collect reward r_{t+1}, and observe next state $s_{t+1} = (Frame_{t+1},\ D_{t+1},\ \theta_{t+1})$,
    Store the transition $(s_t, a_t, r_{t+1}, s_{t+1})$ in $\mathcal{B}$,
    Sample mini-batch of transitions $(s_j, a_j, r_{j+1}, s_{j+1})$ from $\mathcal{B}$,
    Set $y_j = \begin{cases} r_{j+1}, & \text{if } s_{j+1} \text{ is terminal} \\ r_{j+1} + \gamma \max_{a'} Q(s_{j+1}, a'; \Theta'), & \text{otherwise,} \end{cases}$
    Perform a gradient descent step using targets y_j with respect to the online parameters Θ,
    Every C steps, set $\Theta' \leftarrow \Theta$,
  end for
  $\epsilon = \epsilon + (\epsilon_{max} - \epsilon_{min}) \times \exp(-k \times i)$,
end for

At every training episode in Algorithm 1, the decision to explore the environment or exploit what is learned in previous episodes is made using an ϵ-greedy strategy [15] based on the following equation:

$$\epsilon_{t+1} = \epsilon_t + (\epsilon_{max} - \epsilon_{min}) \times \exp(-k \times i), \quad (6)$$

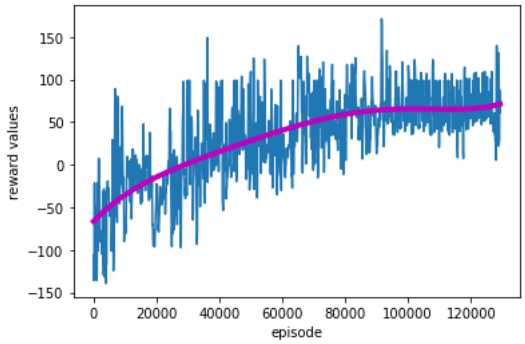
where ϵ_{min} is the minimum exploration probability, ϵ_{max} is the first exploitation probability, k is the rate at which our exploration factor decays, and i refers to the ith episode of the training phase. It is important to choose the way that we balance between them to make sure that the agent tries the maximum number of possible decisions. The idea is that we must have a large ϵ at the beginning of the training of the Q-function and then, we gradually reduce it since the follower becomes more confident at estimating the optimal Q-values.

IV. Selected Simulation Results

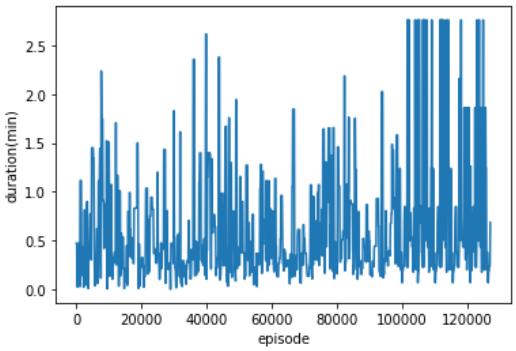
In this section, we investigate the performance of the proposed algorithm. Indeed, to train our model and test it, we use the CARLA simulator, as shown in Fig. 5. In fact, CARLA is very useful in this case due to the high cost of



Fig. 5: Snapshots of the simulation environment.

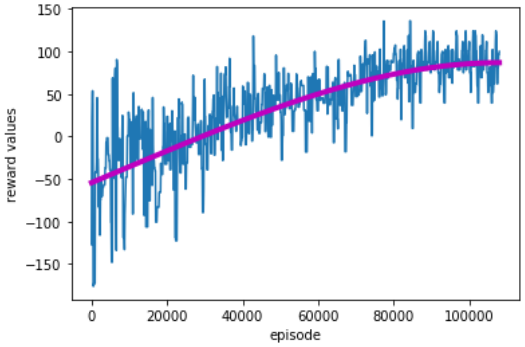


(a) Cumulative reward

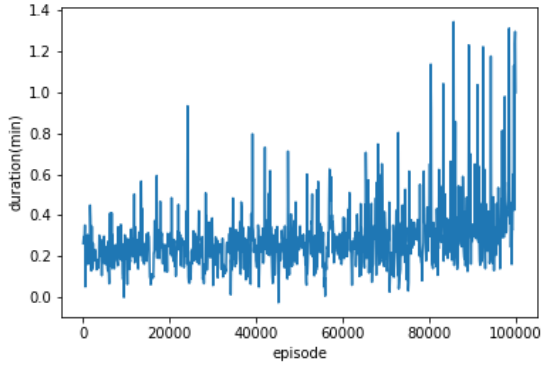


(b) Duration of episodes

Fig. 6: Training results of the autonomous car-following framework using DQN and RGB-D frames.



(a) Cumulative reward



(b) Duration of episodes

Fig. 7: Training results of the autonomous car-following framework using DDPG algorithm and RGB-D frames.

testing such a solution in a realistic environment. CARLA is an open-source simulator that has been designed to support the development, training, and validation of autonomous driving systems. In addition to the open-source code and protocols, CARLA provides open digital assets (urban layouts, buildings, vehicles) that were created for this purpose and can be used freely. For the training process, we set D_{min} to 10% of D_{max}. This value is sufficiently high to avoid crashes with the leader. The discount factor is set to 0.4 and, for the exploration-exploitation trade-off, we set ϵ_{max} to 1 and ϵ_{min} to 0.01.

In Figs. (6) and (7), we plot the training results of the DQN model based on the RGB-D frames and the Deep Deterministic Policy Gradient (DDPG) model as described in [9]. We can notice that the cumulative reward function is generally increasing over episodes until convergence at the end of the training phase where the model is not learning new possibilities from its exploration decisions. Hence, less autonomous driving and car-following errors occur due to the actions taken by the vehicle. In Fig. 6(b), we plot the duration of training episodes which reflects the driving duration of the vehicle. At early episodes, the training time is short as the vehicle crashes very early by taking wrong actions. The model makes some random mistakes during the exploration phase, which explains the fluctuation of the duration of episodes. However, with the decrease of the training loss (increase of achieve reward), i.e., the exploitation phase, the duration of the training episodes increases on average as fewer crashes occur.

During the training phases, both algorithms converge to the same range of values. However, the duration of episodes for the algorithms is significantly different. Indeed, the two models do not learn autonomous driving and car-following in the same manner, as we can notice from Fig. 7(b) where the duration of the DDPG episodes are increasing smoothly. Hence, the model is learning from its previous experience without knowing any new navigation decision or possibilities. On the contrary, the duration of the DQN episodes is wildly changing from an episode to another, and this behavior is explained by the fact that the model is attempting to explore all available navigation decisions in the environment. This attitude during the training is the success key of the algorithm that will increase, considerably, the efficiency of the model in navigating in any unknown environment.

In order to evaluate the model, we proceed by testing the capability of the model to avoid crashing with the leader car or any other object in the simulated environment. In every testing episode, the leader will travel from a point A to another point B and the autonomous vehicle will follow the leader through the entire trajectory. Every time the follower crashes into the leader or the surrounding environment, the test is considered a failure. If the follower can successfully follow the leader across the whole trajectory without incident, the test is considered

TABLE II: DDPG testing Statistics

Testing Result	Total
Successful driving	967
Crash with the leader	18
Crash on the sidewalk	5
Detection lost with leader	10

TABLE III: DQN testing Statistics

Testing Result	Total
Successful driving	975
Crash with the leader	2
Crash onto the sidewalk	3
Detection lost with leader	20

a success. On the other hand, the performances of the DQN model we generated were compared to those of the DDPG RL algorithm. DQN and DDPG had similar performance. DQN held an advantage, however, by minimizing crashes with the leader – a key characteristic for safe operation in real-world applications. We present the results of the comparison between DDPG and DQN in Tables II and III. These tables show statistics about the failing and successful incidents during the testing episodes.

In the testing phase, we utilize three maps, and 1000 runs split amongst the three maps for both the DQN and the DDPG approaches. In each map, the leader vehicle traverses from point A to point B. The DQN approach achieves success in following the leader vehicle without incident for 975 of the 1000 runs. In 20 of the failures, the follower loses the leader vehicle. In the remaining cases, the follower crashed into the leader only twice and leaves the road in three runs.

On the other hand, the DDPG approach succeeds in 967 of the runs, but crashes into the leader at a much higher rate of 18 runs, versus the two runs while testing the DQN approach. This DQN achieves a higher level of safety compared to DDPG since most of its failed tests correspond to a detection loss while the DDPG leads to higher crash rate either with the leader or the sidewalk. We show that the DQN model rapidly learns that collisions are a very high-priority objective. Loss of the leader vehicle is not as critical, because autonomous vehicles are equipped to drive based on the surrounding environment (i.e. lane detection and collision avoidance).

V. Conclusion

In this study, we implemented an autonomous car-following framework using the DQN algorithm. In our model, every state is identified using high-dimensional inputs and other additional image-based features related to the leader-vehicle. In the experimental phase, the model is tested using CARLA simulator over multiple environments. The model showed promising preliminary results in following the leader by avoiding crashes with it and the sidewalk and while using RGB-D frames only as inputs. These results can be significantly improved by taking into account other inputs, such as LiDAR, to feed to the model. Moreover, the model is currently designed to follow the leader in a straight line. In the future extension of this work, a more advanced RL method will be developed with larger action spaces to consider more complicated scenarios.

References

[1] H. Marzbani, H. Khayyam, C. N. TO, V. Quoc, and R. N. Jazar, "Autonomous vehicles: Autodriver algorithm and vehicle dynamics," *IEEE Transactions on Vehicular Technology*, vol. 68, no. 4, pp. 3201–3211, Jan. 2019.

[2] M. C. Lucic, X. Wan, H. Ghazzai, and Y. Massoud, "Leveraging intelligent transportation systems and smart vehicles using crowdsourcing: An overview," *Smart Cities*, vol. 3, no. 2, pp. 341–361, May 2020.

[3] Z. Wang, X. Ma, J. Liu, D. Chigan, G. Liu, and C. Zhao, "Car-following behavior of coach bus based on naturalistic driving experiments in urban roads," in *IEEE International Symposium on Circuits and Systems (ISCAS'19)*, Sapparo, Japan, May 2019.

[4] M. Masmoudi, H. Ghazzai, M. Frikha, and Y. Massoud, "Autonomous car-following approach based on real-time video frames processing," in *IEEE International Conference on Vehicular Electronics and Safety (ICVES'19)*, Cairo, Egypt, Sept. 2019.

[5] X. Gao, L. Gao, and G. Dong, "Research on intelligent driving behavior based on cognitive science and scene simulation," in *2011 International Conference on Intelligence Science and Information Engineering*, 2011, pp. 226–229, Aug. 2011.

[6] O. Buffet, O. Pietquin, and P. Weng, "Reinforcement learning," arXiv:2005.14419, June 2020.

[7] S. Mohseni, M. Pitale, V. Singh, and Z. Wang, "Practical solutions for machine learning safety in autonomous vehicles," arXiv:1912.09630, 2019.

[8] Q. Xue, K. Wang, J. J. Lu, and Y. Liu, "Rapid driving style recognition in car-following using machine learning and vehicle trajectory data," *Journal of Advanced Transportation*, vol. 2019, p. 9085238, Jan 2019.

[9] M. Zhu, X. Wang, and Y. Wang, "Human-like autonomous car-following model with deep reinforcement learning," *Transportation Research Part C: Emerging Technologies*, vol. 97, pp. 348 – 368, Dec. 2018.

[10] H. Gao, G. Shi, G. Xie, and B. Cheng, "Car-following method based on inverse reinforcement learning for autonomous vehicle decision-making," *International Journal of Advanced Robotic Systems*, vol. 15, no. 6, Dec, 2018.

[11] D. Yang, L. Zhu, Y. Liu, D. Wu, and B. Ran, "A novel car-following control model combining machine learning and kinematics models for automated vehicles," *IEEE Transactions on Intelligent Transportation Systems*, vol. 20, no. 6, pp. 1991–2000, June 2019.

[12] M. Kim, J. Park, and I. S. Kweon, "RGBD sensor and mirrors: A practical setup for 3D reconstruction of dynamic objects," in *11th International Conference on Ubiquitous Robots and Ambient Intelligence (URAI), Kuala Lumpur, Malaysia*, Nov. 2014.

[13] M. Masmoudi, H. Ghazzai, M. Frikha, and Y. Massoud, "Object detection learning techniques for autonomous vehicle applications," in *IEEE International Conference on Vehicular Electronics and Safety (ICVES), Cairo, Egypt*, Sept. 2019.

[14] J. Redmon and A. Farhadi, "YOLOv3: An incremental improvement," *arXiv:1804.02767v1*, Apr. 2018.

[15] W. C. Cheung, "Exploration-exploitation trade-off in reinforcement learning on online markov decision processes with global concave rewards," *arXiv:1905.06466*, May 2019.